



KNOWLEDGE-AUGMENTED LARGE LANGUAGE MODELS FOR ARABIC POST-OCR ERROR CORRECTION

By

Engineer's Name

A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

**KNOWLEDGE-AUGMENTED LARGE
LANGUAGE MODELS FOR ARABIC
POST-OCR ERROR CORRECTION**

By

Engineer's Name

A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

Under the Supervision of

Dr./Prof. Dr. Main Supervisor

Affiliation

Electronics and Communications Engineering
Faculty of Engineering , Cairo University

Dr./Prof. Dr. Co-supervisor Dr./Prof. Dr. Co-supervisor

Affiliation

Affiliation

Department

Department

Faculty, University

Faculty, University

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

**KNOWLEDGE-AUGMENTED LARGE
LANGUAGE MODELS FOR ARABIC
POST-OCR ERROR CORRECTION**

By

Engineer's Name

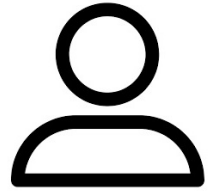
A Thesis Submitted to the
Faculty of Engineering at Cairo University
in Partial Fulfillment of the
Requirements for the Degree of
MASTER OF SCIENCE
in
Electronics and Communications Engineering

Approved by the Examining Committee:

Dr./Prof. Dr. Main Supervisor,	Thesis Main Advisor
Dr./Prof. Dr. Int. Examiner,	Internal Examiner
Dr./Prof. Dr. Ext. Examiner,	External Examiner
Affiliation	
Faculty, University	

FACULTY OF ENGINEERING ,CAIRO UNIVERSITY
GIZA,EGYPT
YYYY

Engineer's Name: Engineer's Name
Date of Birth: dd/mm/yyyy
Nationality: Engineer's Nationality
E-mail: Engineer's E-mail
Phone: Engineer's Phone Number
Address: Street, City, Province, Zip Code
Registration Date: dd/mm/yyyy
Awarding Date: --/yy
Degree: Master of Science
Department: Electronics and Communications Engineering



Supervisors:

Dr./Prof. Dr. Main Supervisor
Dr./Prof. Dr. Co-supervisor
Dr./Prof. Dr. Co-supervisor

Examiners:

Dr./Prof. Dr. Main Supervisor (Thesis Main Advisor)
Dr./Prof. Dr. Int. Examiner (Internal Examiner)
Dr./Prof. Dr. Ext. Examiner (External Examiner)
Affiliation
Faculty, University

Title of Thesis:

**Knowledge-Augmented Large Language Models for Arabic
Post-OCR Error Correction**

Key Words:

Arabic OCR; Post-OCR Correction; Large Language Models; Prompt Engineering; Qwen3; PATS-A01; KHATT; Error Taxonomy; Self-Reflective Prompting; CAMEL Tools

Summary:

This thesis investigates whether a lightweight open-source instruction-tuned large language model (Qwen3-4B-Instruct-2507, 4 billion parameters) can correct systematic errors produced by the Qaari Arabic OCR engine through prompt engineering alone, without task-specific fine-tuning. Five knowledge-augmentation strategies are evaluated in a controlled isolation design across nine datasets—eight typewritten fonts from PATS-A01 (4.3–24.3% baseline CER) and one handwritten corpus from KHATT (44.6% baseline CER)—each changing exactly one variable relative to a zero-shot baseline. Strategies include confusion-matrix injection filtered by LLM failures (Phase 3), self-reflective prompting derived from the model's own training-split error statistics (Phase 4), morphological post-processing via CAMEL Tools with known-overcorrection revert (Phase 5), and systematic strategy combinations with component-removal ablation (Phase 6). The central finding is the identification and formal characterisation of an *over-correction threshold*: a baseline CER below which the LLM introduces more errors than it fixes ($e^* \approx 6\text{--}7\%$ for this model-engine pair). Above this threshold, the best

combination—confusion-matrix injection paired with self-reflective prompting (`conf_self`)—achieves 11.2% CER on PATS-A01 (a 21.1% relative reduction vs. OCR baseline) and 42.4% on KHATT (5.0% relative reduction), with statistical significance on six of nine datasets at Bonferroni-corrected $\alpha = 0.0125$. Ablation analysis reveals that the two strategies address orthogonal failure modes and combine near-additively, with self-reflective prompting contributing approximately twice the marginal improvement of confusion-matrix injection. On handwritten text, where 37% of errors are segmentation failures inaccessible to text-only correction, improvement is structurally limited—motivating future multi-modal approaches. The thesis contributes an experimental framework, a three-stage export–infer–analyse pipeline, and an empirical characterisation of the conditions under which LLM-based post-OCR correction helps, hurts, or has no effect.

Disclaimer

I hereby declare that this thesis is my own original work and that no part of it has been submitted for a degree qualification at any other university or institute.

I further declare that I have appropriately acknowledged all sources used and have cited them in the references section.

Name: Engineer's Name

Date:

Signature:

Dedication

You may include this section if you wish to dedicate your thesis to someone.

Acknowledgements

In this section, you may provide acknowledgements to those who gave you support and encouragement to complete your thesis. Acknowledgement of funding from local and international funding agencies must be clearly stated.

Table of Contents

List of Tables

List of Figures

List of Symbols and Abbreviations

Symbol	Description
C	Number of character substitutions, deletions, and insertions (edit operations)
N	Total number of characters in the reference (ground-truth) text
W	Number of word substitutions, deletions, and insertions
M	Total number of words in the reference text
CER_i	Character error rate for sample i
n	Number of evaluation samples
$d(\cdot)$	Edit distance (Levenshtein distance) function
k	Number of top- k items retrieved in nearest-neighbour search
α	Family-wise error rate for Bonferroni correction

Abbreviation	Description
CER	Character Error Rate
WER	Word Error Rate
LLM	Large Language Model
VLM	Vision–Language Model
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
MSA	Modern Standard Arabic
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine-Tuning
NLP	Natural Language Processing

FAISS	Facebook AI Similarity Search
PATS	Printed Arabic Text Set
KHATT	Khatt Arabic Handwritten Text dataset
QALB	Qatar Arabic Language Bank
OpenITI	Open Islamicate Texts Initiative
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory network
CTC	Connectionist Temporal Classification
CoT	Chain-of-Thought (prompting)
GT	Ground Truth
std	Standard deviation
p95	95th percentile

List of Publications

1. [**Under preparation**] Knowledge-Augmented Large Language Models for Arabic Post-OCR Error Correction. *To be submitted to an IEEE conference on Arabic NLP or document analysis.*

Abstract

Arabic optical character recognition (OCR) engines produce systematic errors rooted in the visual properties of the Arabic script: dot-group character confusions, morphologically valid but contextually wrong substitutions, and segmentation failures that destroy word boundaries. Open-source engines such as Qaari fall short of closed-source vision–language models, motivating the question of whether post-OCR correction via a lightweight large language model (LLM) can bridge this accuracy gap without modifying the recognition engine and without task-specific fine-tuning.

This thesis presents a controlled, multi-dataset evaluation of knowledge-augmented prompt-engineering strategies for Arabic post-OCR correction using Qwen3-4B-Instruct-2507, an open-source 4-billion-parameter instruction-tuned model. The experimental design follows an isolation principle: five knowledge augmentation conditions are evaluated across nine datasets — eight typewritten fonts from the PATS-A01 corpus (baseline CER ranging from 4.3% to 24.3%) and one handwritten corpus (KHATT, 44.6% baseline CER) — each changing exactly one variable relative to a zero-shot baseline. An extended evaluation on four additional full-page document datasets — KHATT-Paragraph (449 multi-line handwritten paragraphs), Yarmouk (200 printed encyclopedic pages), Muharaf (116 historical handwritten letters), and Historical (40 manuscript pages from eight Arabic books) — further tests generalisation across previously unseen document domains. The augmentation conditions are: (1) failure-filtered character confusion matrix injection, cross-referenced with the model’s own zero-shot failures to retain only actionable confusion pairs (Phase 3); (2) self-reflective prompting derived from the model’s training-split error statistics, including quantified fix and introduction rates, concrete word-level error pairs, and an explicit overcorrection warning (Phase 4); and (3) CAMeL Tools morphological post-processing with position-indexed known-overcorrection revert (Phase 5). A systematic combination and component-removal ablation study (Phase 6) tests four multi-component configurations: confusion-matrix only (`conf_only`), self-reflective only (`self_only`), both combined (`conf_self`), and the best prompt combination augmented with CAMeL post-processing (`best_camel`).

All eight PATS-A01 fonts share 2,766 text lines drawn from the same underlying

content. An aligned 85/15 train/validation split (seed 42) ensures that every font receives the same 415 validation samples, so that font is the only variable when comparing OCR quality and correction effectiveness across typefaces. KHATT provides a complementary handwritten evaluation with substantially higher baseline error rates.

Phase 1 establishes the OCR error landscape. Baseline CER on PATS-A01 fonts ranges from 4.3% (Akhbar) to 24.3% (Naskh), with a macro-average of 14.2%. KHATT handwritten text reaches 44.6% CER. The dominant error type across all typewritten fonts is dot confusion (42% of substitutions), followed by similar-shape confusions (18%) and hamza errors (12%). Morphological analysis reveals that 71% of OCR errors produce valid Arabic words in the wrong context (*valid-but-wrong* errors), with only 29% producing morphologically invalid tokens.

Phase 2 (zero-shot correction) reduces the PATS-A01 macro-average CER from 14.2% to 12.3%. However, the effect is strongly font-dependent: on Traditional font, CER drops from 14.2% to 2.7% (an 81.0% relative reduction), while on Akhbar (4.3% baseline), zero-shot correction *increases* CER to 5.2%, introducing more errors than it fixes.

Phase 3 (confusion-matrix injection) achieves a PATS-A01 macro-average CER of 12.0% (−0.3 pp vs. Phase 2) and 42.7% on KHATT (−0.3 pp). The improvement is consistent but modest in isolation.

Phase 4 (self-reflective prompting) achieves the strongest isolated result: 11.6% on PATS-A01 (−0.7 pp vs. Phase 2) and 42.8% on KHATT (−0.2 pp). The self-analysis is derived from Phase 2 training-split failures and includes both aggregate error statistics and concrete word-level error-correction pairs.

Phase 5 (CAMEL morphological post-processing) produces a slight degradation (+0.1 pp on both datasets) due to the preponderance of valid-but-wrong errors that the morphological filter cannot detect.

Phase 6 combinations demonstrate that Phase 3 and Phase 4 are complementary. The `conf_self` combination achieves 11.2% CER on PATS-A01 (a 21.1% relative improvement over the OCR baseline) and 42.4% on KHATT (a 5.0% relative improvement). Adding CAMEL post-processing (`best_camel`) yields marginal further gain on PATS-A01 (11.1%) but a slight reversal on KHATT (42.5%).

The central finding is the identification and formal characterisation of an *over-correction threshold*: a baseline CER level (e^*) below which the LLM introduces more errors than it corrects. The threshold arises because the introduction rate (fraction of correct tokens incorrectly changed) is approximately constant across fonts, while the fix rate scales with error density. Empirically, e^* lies at approximately 6–7% CER for this model–engine pair: Akhbar (4.3% baseline) is harmed by every strategy tested, while Simplified (7.9%) benefits. Phase 4 is the only strategy that partially mitigates over-correction on Akhbar, because its diagnostic text explicitly targets the introduction rate rather than only providing correction guidance.

The ablation study reveals that Phases 3 and 4 address orthogonal failure modes—Phase 3 targets the OCR engine’s known confusion patterns (raising the fix rate), while Phase 4 targets the correction model’s systematic biases (lowering the introduction rate)—and combine near-additively. On KHATT handwritten text, where 37% of errors are segmentation failures (word merges and splits that destroy word boundaries), text-only correction is structurally limited regardless of prompt strategy, achieving only 3.6–5.0% relative improvement despite a 44.6% baseline CER.

An ongoing second experiment applies a 222 factorial design across all 13 validation datasets, systematically varying three factors: prompt conservatism (aggressive vs. conservative instruction framing), example selection (generic vs. hand-picked domain-matched examples including content from all four new dataset types), and retrieval strategy (zero-shot vs. BM25-RAG). This experiment addresses the generalisability of the established prompt strategies to full-page, multi-domain Arabic documents.

This thesis contributes: (1) a formal over-correction threshold model with empirical estimation; (2) a novel self-reflective prompting strategy for OCR correction achieving the strongest isolated result; (3) a proof of complementarity between confusion-matrix and self-reflective augmentation; (4) a comprehensive thirteen-dataset evaluation spanning typewritten, handwritten, historical manuscript, and encyclopedic printed domains; (5) a three-stage export–infer–analyse pipeline enabling GPU-free local reproducibility; and (6) an empirical characterisation of the structural limits of text-only correction on handwritten text.

Chapter 1

Introduction

1.1 The Problem with Arabic OCR

The digitisation of Arabic text is a prerequisite for a broad class of downstream applications: full-text search over historical archives, neural machine translation of Arabic news and literature, text-to-speech systems for visually impaired users, and information extraction from scanned government documents. The technology that converts images of text into machine-readable character sequences — optical character recognition (OCR) — is the entry point for all of these applications. When OCR fails, the failure propagates: a misread character changes word meaning, a sequence of recognition errors degrades translation quality, and a corrupted document image cannot be retrieved by keyword search.

Arabic presents challenges that English OCR does not face. The Arabic script is cursive: letters connect within words and their shapes change depending on their position in the word (isolated, initial, medial, or final). There are no clear boundaries between characters within a word, so segmentation and recognition are tightly coupled — a segmentation error directly produces a recognition error. Many Arabic letters share the same base skeleton and differ only in the number and placement of small dots: the pair ب/ي (ba/ya) differs only in whether the dot is below or above the baseline; ف/ق (fa/qaf) differs only in having one or two dots above. Ink irregularities, low scan resolution, or font variation can shift a dot from its correct position and cause the OCR engine to output the wrong letter.

Short vowels (diacritics, or *tashkeel*) are generally optional in modern printed Arabic and are absent from most typewritten text, yet their presence or absence changes word meaning and complicates matching between OCR output and ground

truth. Classical and modern standard Arabic coexist with several regional dialects, each with partially distinct orthographies. Ligatures — special glyphs for common letter combinations — further increase the visual complexity that OCR engines must resolve.

A further complication is the diversity of fonts used in printed Arabic. Each typeface imposes its own glyph shapes, dot placement conventions, stroke widths, and ligature styles. A font with decorative rendering may visually merge characters that a plainer font keeps distinct; a calligraphic font may exaggerate strokes to the point where letter identity becomes ambiguous even to human readers at a glance. This font-induced variation means that the same underlying text produces substantially different character-level error patterns depending on the typeface used to render it.

The consequence of these challenges is that open-source Arabic OCR engines produce recognisable error patterns. Character-level confusions between visually similar letter pairs, word-level segmentation failures, and occasional catastrophic output loops inflate the character error rate (CER) in ways that downstream applications cannot silently absorb. A single confused character in a word can change its meaning entirely; a repeated output loop corrupts an entire document page. These are not random errors; they are systematic, rooted in the visual properties of the Arabic script and in the specific training data and architecture of each OCR engine. Their systematic nature motivates the hypothesis that a language model, given the erroneous OCR text, may be able to identify and correct the errors using its prior knowledge of valid Arabic.

1.2 The Open-Source vs. Closed-Source Gap

The landscape of Arabic OCR is divided between commercial closed-source systems and open-source alternatives. Closed-source vision-language models (VLMs) — including GPT-4V [0], Gemini, and similar systems — handle Arabic document recognition with high accuracy because they were trained on massive multimodal corpora that include Arabic text in diverse scripts, fonts, and document layouts. These systems are not publicly auditable, cannot be deployed in offline or sensitive

environments, and charge per-query fees that are prohibitive for large archival digitisation projects.

Open-source OCR engines such as Qaari¹ offer accessibility, reproducibility, and deployability without network dependencies. However, they fall behind closed-source alternatives on error rate, particularly on difficult fonts, handwritten text, and documents with low scan quality. The gap is meaningful: a 1–2% CER difference translates to dozens of errors per page in a long document, and a 10% CER difference renders a document largely unreadable to automated downstream systems.

The gap between open-source and closed-source systems exists at multiple levels. At the recognition level, closed-source VLMs benefit from training on billions of image–text pairs that include diverse Arabic document types, while open-source engines are trained on smaller curated sets. At the language-model level, closed-source systems incorporate large-scale language models as priors that can use context to resolve visual ambiguity, while open-source OCR engines rely on smaller, task-specific language components. At the post-processing level, closed-source systems often include proprietary correction layers that are not available to open-source users.

The question motivating this thesis is whether this gap can be narrowed *without changing the OCR engine itself*. Replacing an open-source OCR engine with a closed-source one is not always feasible: cost, data privacy, offline operation requirements, and the need for reproducibility in academic research all create barriers to adopting closed-source solutions. But adding a post-processing correction stage — a language model that receives the OCR output as text and corrects it before downstream use — is a modular intervention that leaves the OCR pipeline intact. If a language model can reliably correct the systematic errors that open-source Arabic OCR engines make, the effective accuracy of the system would improve without requiring a new recognition model.

¹Qaari is an open-source Arabic OCR system developed for modern printed Arabic text. It is the primary OCR engine evaluated in this thesis.

1.3 Post-OCR Correction as a Text-to-Text Task

Post-OCR correction treats the OCR output as noisy text and uses a language model to produce a denoised version. The formulation is simple: given a sequence of characters or words output by the OCR engine, produce the most likely corresponding sequence in the original document. This is a text-to-text task — no images are required after the OCR stage — and can be approached with any sequence-to-sequence model.

The decoupling of the vision stage (OCR) from the language stage (correction) has several practical advantages. It is engine-agnostic: the same correction system can be applied to output from any OCR engine without modification. It is retroactive: it can improve existing digitised archives without re-scanning or re-running OCR, making it useful for large collections where re-processing is impractical. It is modular: improvements to the OCR engine and improvements to the corrector can be developed independently; adopting a better correction model does not require retraining the recognition system, and vice versa. And it is accessible: language models capable of Arabic text processing are available as open-source checkpoints that can be run on commodity hardware with GPU access, without proprietary APIs.

The rise of large language models (LLMs) trained on massive multilingual corpora opens a new avenue for post-OCR correction. Unlike classical correction approaches that require a specialised language model trained on OCR error pairs [0], a general-purpose instruction-tuned LLM can be directed to perform correction tasks through a natural-language prompt. This *prompt engineering* approach requires no task-specific training, no labelled correction pairs, and no model fine-tuning. The correction capability emerges from the model’s pre-trained knowledge of the target language.

However, prompt engineering is not a free lunch. The model’s language priors reflect the text it was trained on, which may not match the specific error patterns of the OCR engine under evaluation. The model may over-correct, rewriting text that is already correct; it may under-correct, leaving errors that it cannot resolve from text context alone; or it may introduce new errors by substituting plausible but incorrect words. Understanding when and why these failure modes occur — and whether injecting additional knowledge can mitigate them — is the core empirical

question of this thesis.

1.4 Motivation for a Knowledge-Augmented Approach

A zero-shot LLM corrector operates on the model’s general prior knowledge of Arabic text. It knows that certain character sequences are rare or non-words in Arabic, that certain word forms follow morphological patterns, and that adjacent words should cohere semantically and syntactically. These priors are powerful for catching clear errors but insufficient for catching the systematic visual confusions that OCR engines produce — confusions that may yield perfectly plausible Arabic words that happen to be contextually wrong.

Knowledge augmentation is the practice of injecting structured information about the correction task into the model’s prompt, supplementing its generic language prior with task-specific guidance. In the context of Arabic OCR correction, several knowledge sources are plausible:

- **Character confusion matrices** derived from Phase 1 analysis can tell the model which specific character pairs the OCR engine tends to confuse. If the engine frequently outputs ﺥ where the ground truth is ﺏ on a particular font, injecting this information may help the model prioritise corrections of that specific substitution.
- **Self-reflective insights** derived from the model’s own training-split correction behaviour capture the model’s systematic blind spots: errors it consistently fails to fix, error types it introduces when correcting, and samples it over-corrects. By feeding these statistics back as prompt context, the model is given the opportunity to compensate for its own biases.
- **Morphological post-processing** via CAMEL Tools can catch a specific class of LLM output failures: words that are morphologically invalid in Arabic (i.e., not recognised by a morphological analyser). When the LLM introduces such an invalid word from a valid OCR source, a revert to the OCR original is a conservative and reliable corrective action.

- **Retrieval-augmented examples** drawn from Phase 2 training-split corrections provide per-sample adaptive context. A BM25 index built on character n-grams of training OCR texts retrieves the k most similar (OCR, ground-truth) pairs for each validation sample. These are injected into the prompt as concrete correction exemplars, tailoring the prompt to the specific error pattern of the current input. Unlike the global statistics of Phases 3 and 4, retrieved examples adapt sample-by-sample to the input at hand.

Each of these sources addresses a different aspect of the correction problem, and they may interact in complex ways when combined. The central question is not simply whether knowledge augmentation helps, but *which* knowledge type helps, *when* it helps (for which fonts and error profiles), and *how much* each source contributes in an ablation study.

1.5 Research Questions

This thesis investigates the following research questions. They are ordered to reflect the experimental sequence: characterisation before correction, isolated experiments before combinations.

RQ1 — Error Characterisation: What are the characteristics of errors produced by Qaari on Arabic typewritten and handwritten text? How do error profiles differ across fonts and script types, and what does the error taxonomy reveal about which correction strategies are most likely to succeed?

RQ2 — Zero-Shot Correction Capability: Can a zero-shot instruction-tuned LLM correct Arabic OCR errors without any task-specific guidance? How does correction effectiveness vary across fonts with different baseline error rates, and what is the nature and frequency of over-correction?

RQ3 — Confusion-Matrix Injection: Does injecting the OCR engine’s known character confusion patterns (derived from Phase 1 analysis and cross-referenced with LLM failure cases) into the system prompt improve correction beyond zero-shot? Does the effect vary by font?

RQ4 — Self-Reflective Prompting: Does feeding the model a statistical summary of its own training-split correction failures — fix rates and introduction

rates per error type, together with concrete word-level error-correction pairs — improve its correction quality on a held-out validation set?

RQ5 — Morphological Post-Processing: Does applying CAMEL Tools morphological validation after zero-shot correction — reverting morphologically invalid LLM outputs to the valid OCR original, and reverting known overcorrection patterns — improve the quality of corrected text?

RQ6 — Strategy Combinations and Ablation: What combination of Phase 3 and Phase 4 strategies produces the best overall correction quality? What is the marginal contribution of each component in ablation, and does adding Phase 5 CAMEL post-processing provide further benefit on top of the best prompt-only combination?

RQ8 — Retrieval-Augmented Correction: Does per-sample retrieval of similar OCR correction pairs from the training split — via a BM25 index over character n-grams — improve correction quality beyond zero-shot prompting? Does retrieving from the same domain and engine (training-split corrections) overcome the domain-mismatch problem that limits external corpus retrieval?

RQ7 (overarching): Does correction effectiveness depend primarily on the baseline OCR error profile of the input, or can knowledge augmentation overcome the over-correction problem observed on low-error fonts?

RQ9 — Prompt Conservatism and Domain Generalisation: Does prompt conservatism — contrasting an aggressive correction instruction (“correct all errors”) with a conservative one (“return unchanged if already correct”) — affect generalisation to unseen full-page document domains such as encyclopedic printed text, historical manuscripts, and archival handwritten letters?

RQ10 — Domain-Matched Example Selection: Does providing hand-picked, domain-matched examples — synthetic OCR errors crafted from the ground truth of new target-domain datasets, covering all four new document types — outperform generic auto-generated examples on those new dataset types, and does this advantage extend to the established PATS-A01 and KHATT datasets?

RQ11 — Cross-Domain BM25 Retrieval: Can a BM25 retrieval index built on one document domain (PATS-A01 typewritten and KHATT handwritten training corrections) provide useful correction context for a structurally different domain

(historical manuscripts, encyclopedic full-page content)? Does cross-domain retrieval degrade, match, or improve over zero-shot prompting on the new datasets?

RQ1 is addressed in Chapter ??, Section ??, using Phase 1 outputs. RQ2–RQ5 are each addressed by a corresponding isolated experiment (Phases 2–5), all reporting on the validation split. RQ6 is addressed by the combination and ablation study (Phase 6). RQ8 is addressed by the retrieval-augmented experiment (Phase 8). RQ7 is addressed by synthesising findings across Phases 2–6 and 8 in Chapter ??. RQ9–RQ11 are addressed by Experiment 2 (Section ??), whose results are reported in Chapter ??.

1.6 Contributions

This thesis makes the following contributions.

1. **A systematic multi-dataset evaluation framework for Arabic post-OCR correction.** Six prompt-engineering strategies are evaluated across nine datasets (eight PATS-A01 typewritten fonts and KHATT handwritten text) using aligned train/validation splits, each changing exactly one variable relative to the zero-shot baseline. This isolation design enables direct attribution of observed CER changes to individual knowledge sources, and the multi-font evaluation reveals font-dependent effects invisible in single-dataset studies. To the best of the author’s knowledge, this is the first study to evaluate LLM prompt strategies for Arabic post-OCR correction simultaneously across this many font families under a controlled isolation design.
2. **A three-stage export–infer–analyse pipeline.** The experimental infrastructure separates local data preparation from GPU inference from result analysis. Inference is executed on Kaggle (free GPU quota) via a unified `scripts/infer.py` script that handles resumption, HuggingFace synchronisation, and prompt versioning. The pipeline is fully modular: each phase adds one new `PromptBuilder` method and one pipeline script, with the inference and analysis layers unchanged. This design enables full reproducibility without a local GPU.

3. **A novel self-reflective prompting strategy for OCR correction.** Phase 4 derives correction guidance from the model’s own training-split error statistics — quantifying fix rates and error introduction rates per error type, converting them into Arabic-language diagnostic sentences, and injecting both the statistical summary and concrete word-level OCR error pairs into the system prompt for validation-split inference. This strategy is, to the author’s knowledge, the first application of self-diagnostic statistics as prompt augmentation for Arabic OCR correction. It achieves the strongest isolated result of any strategy tested: -0.7 pp CER on PATS-A01 relative to zero-shot.
4. **An aligned multi-font evaluation design with quantified font effects.** All eight PATS-A01 fonts share 2,766 text lines drawn from the same underlying content, with an aligned 85/15 split (seed 42) ensuring identical 415-sample validation sets across fonts. This controlled design isolates font as the sole variable, enabling principled cross-font comparisons of both OCR quality and LLM correction effectiveness. The resulting per-font analysis quantifies the span of OCR error rates (4.3%–24.3% CER across fonts) and shows that font choice has a larger effect on effective accuracy than any correction strategy tested.
5. **An empirical characterisation of the over-correction threshold.** Zero-shot LLM correction effectiveness depends strongly on the baseline error rate. On fonts with CER below approximately 8%, the LLM introduces more errors than it fixes; on fonts with CER above this threshold, the net effect is positive. This font-dependent over-correction threshold is quantified across all eight typewritten fonts and KHATT, providing actionable guidance for when LLM correction should and should not be applied.
6. **A comprehensive error taxonomy for Qaari across multiple fonts.** Phase 1 produces per-font character-level confusion matrices, error taxonomies with eight categories, and morphological analyses via CAMEL Tools, revealing how error profiles shift across fonts — from dot-confusion-dominated errors on clean fonts to more diverse distributions on visually complex typefaces. The morphological analysis establishes that 71% of OCR errors are valid-but-wrong

(real Arabic words in the wrong context), with important implications for Phase 5.

7. **An ablation study of strategy combinations.** Phase 6 tests four combinations (`conf_only`, `self_only`, `conf_self`, `best_camel`) and conducts a component-removal ablation to quantify each strategy’s marginal contribution. The best combination (`conf_self`) achieves 11.2% CER on PATS-A01 (21.1% relative improvement over OCR baseline) and 42.4% on KHATT (5.0% relative improvement).
8. **A retrieval-augmented correction strategy using same-domain training examples.** Phase 8 builds a BM25 index over character n-grams of Phase 2 training corrections and, for each validation sample, retrieves the k most similar (OCR, ground-truth) pairs to inject as per-sample few-shot context. This design sidesteps the domain-mismatch problem that caused external corpus retrieval (OpenITI) to degrade performance, by restricting the retrieval pool to the same OCR engine and text domain as the evaluation data. To the author’s knowledge, this is the first application of BM25 character-n-gram retrieval to Arabic post-OCR correction, providing an adaptive, per-sample complement to the global statistical patterns of Phases 3 and 4.
9. **Open-source code and reproducible configuration.** All pipelines, prompt templates, knowledge base loaders, and evaluation scripts are publicly available. Every experimental result is reproducible from the repository using the provided configuration files and inference notebooks. The `MockCorrector` backend enables local development and testing of all phases without GPU access.
10. **Four new full-page Arabic document datasets integrated into the evaluation framework.** KHATT-Paragraph (449 multi-line handwritten paragraph samples), Yarmouk (200 printed encyclopedic full-page documents), Muharaf (116 historical handwritten letter images from 19th–20th century correspondence), and a Historical Manuscripts dataset (40 pages from eight Arabic books) are prepared, aligned with Qaari OCR outputs, and incorporated into the DataLoader. This expands the evaluation from nine to thirteen datasets

and introduces document-level, full-page, and historically diverse content that is absent from the established PATS-A01 and KHATT benchmarks.

11. **A 222 factorial experiment design for cross-domain prompt ablation (Experiment 2).** Eight trials systematically vary prompt conservatism, example selection strategy, and retrieval mode across all thirteen validation/testing datasets. Hand-picked examples are synthetically constructed from the ground truth of the four new datasets to cover their domain-specific OCR error patterns, enabling a controlled comparison of domain-matched vs. generic example selection on unseen document types.

1.7 Scope and Boundaries

Several design decisions define the scope of this thesis and bound the generalisability of its findings.

Single OCR engine. All experiments use Qaari as the OCR engine. Qaari’s specific error patterns — particularly its character confusion profile across fonts — shape the results of all subsequent phases, including which confusion patterns are injected in Phase 3 and which self-reflective insights are derived in Phase 4. Results may differ for other OCR engines (e.g., Kraken, EasyOCR, commercial engines) with different error profiles.

Single LLM. All correction experiments use Qwen3-4B-Instruct-2507. Larger models (7B, 13B, 70B) may have stronger Arabic language priors and different over-correction behaviour. The choice of Qwen3-4B was motivated by its multilingual capability, its fit within the free-tier GPU compute available on Kaggle (16 GB VRAM), and its strong Arabic performance relative to its parameter count.

Text-only correction. All phases treat correction as a pure text-to-text task. The document image is not used after the OCR stage. This means that visually ambiguous cases — where the correct character can only be determined from the image — are inaccessible to the corrector. The results on KHATT handwritten text partly reflect this limitation.

Modern Standard Arabic. PATS-A01 documents are printed in MSA. KHATT contains handwritten MSA. Results on dialectal Arabic, Classical Arabic, or code-

switched text (Arabic–English mixing) may differ.

Prompt engineering without fine-tuning. All strategies are zero-shot or few-shot prompt-engineering approaches. Fine-tuning the LLM on OCR error pairs is outside the scope of this thesis. Fine-tuned models might outperform all prompt strategies tested but require labelled training data and substantially greater compute.

Thirteen evaluation datasets across two experiments. Experiment 1 evaluates nine datasets (eight PATS-A01 fonts plus KHATT) under the controlled isolation design described above. Experiment 2 extends to thirteen datasets by adding four full-page document corpora (KHATT-Paragraph, Yarmouk, Muharaf, and Historical). OCR results for the new datasets are available on validation and testing splits only; training-split OCR was not produced due to compute constraints. The factorial design of Experiment 2 therefore does not use the isolation principle of Experiment 1 (no training split for self-reflective analysis), and comparisons between experiments should account for this difference in methodology.

1.8 Thesis Organisation

The remainder of this thesis is organised as follows.

Chapter 2.6.5 — Background and Literature Review builds the necessary foundations. It covers the Arabic language and script in depth, the evolution of OCR technology from early rule-based systems through neural approaches, the development of large language models from the transformer architecture through instruction tuning and alignment, and prompt engineering strategies relevant to this study (zero-shot, chain-of- thought, self-reflective prompting). It surveys related work on post-OCR correction (classical and neural), Arabic NLP resources, and morphological analysis via CAMEL Tools. A reader familiar with any of these topics may skip the corresponding subsection.

Chapter ?? — System Design and Datasets describes the complete experimental infrastructure. It introduces the PATS-A01 and KHATT datasets in detail, explains the aligned multi-font train/validation split design and its rationale, describes the three-stage export–infer–analyse pipeline, defines the text normalisation and CER/WER evaluation protocol, documents each knowledge base component

(confusion matrix loader and LLM insights loader), and presents the `PromptBuilder` class hierarchy and all prompt variants used across phases.

Chapter ?? — Experimental Methodology describes each experimental phase as a self-contained experiment with a stated hypothesis, step-by-step method, variables changed vs. Phase 2, output files, and the research question answered. Phases 1 through 5 are covered as isolated experiments, followed by Phase 6 (combinations and ablation) and Phase 8 (retrieval-augmented generation), with particular attention to the isolation principle that ensures each augmentation condition can be attributed to a single knowledge source. The chapter concludes with Experiment 2 (Section ??): the 222 factorial prompt design ablation across all thirteen datasets.

Chapter ?? — Results and Analysis reports all numerical results. For Experiment 1 (nine datasets), per-font Phase 1 baselines establish the OCR error landscape, Phase 2 zero-shot results reveal font-dependent correction effectiveness, Phases 3–5 quantify individual knowledge sources, and Phase 6 tests combinations with ablation. Section ?? reports Experiment 2 results across the thirteen evaluation datasets. Every result is presented in a table and interpreted in prose, with statistical significance reported via paired t-tests and Cohen’s d effect sizes.

Chapter ?? — Discussion and Conclusion interprets the findings, develops the font-dependent over-correction threshold as a unified theoretical argument, revisits each research question in the light of the results, discusses the conditions under which the findings would likely differ, states practical system design recommendations, enumerates limitations, and charts future research directions.

The appendices provide supporting material: Appendix A contains the full prompt templates for all phases; Appendix B contains the Phase 4 Arabic-language diagnostic text generated from training-split analysis; Appendix C contains the relevant sections of the configuration file; and Appendix D contains representative tables of OCR error examples from Phase 1.

Chapter 2

Background and Literature Review

This chapter builds the background necessary to follow the rest of the thesis. A reader unfamiliar with Arabic script, OCR technology, large language models, or prompt engineering should be able to acquire the required context here. Section 2.1 covers the Arabic language and script. Section 2.2 introduces optical character recognition and its Arabic-specific challenges. Section 2.3 surveys large language models from early neural approaches through the modern transformer era. Section 2.4 covers prompt engineering methods relevant to this study. Section 2.5 describes the external knowledge sources used. Section 2.6 reviews related work on post-OCR correction and positions this thesis within that literature.

2.1 The Arabic Language and Script

2.1.1 Linguistic Overview

Arabic is a Semitic language and the liturgical language of Islam, spoken by over 300 million people as a first language across the Arab world and understood by a much larger community as a second or religious language [0]. Its most distinctive grammatical feature is *root-and-pattern morphology*: the vast majority of Arabic words are derived from a three-consonant (trilateral) root, and different word patterns are created by interleaving the root consonants with fixed vowel-and-consonant patterns. For example, the root *k-t-b* (relating to writing) yields: *kataba* (he wrote), *kitaab* (book), *kaatib* (writer), *maktaba* (library), *maktub* (written/letter), and dozens of other morphologically related forms.

This morphological richness creates a large and sparse vocabulary for text processing tools. Where English uses analytic constructions (“he will have been writing”), Arabic encodes tense, aspect, voice, gender, number, and definiteness through bound morphology, generating many distinct surface forms for each lemma. A vocabulary estimated at several hundred thousand distinct word forms makes statistical NLP methods data-hungry and challenges neural models trained on smaller corpora.

Modern Standard Arabic (MSA) is the standardised written form used in formal contexts across the Arab world: newspapers, government documents, academic publications, and formal broadcasts. It descends from Classical Arabic (the language of the Quran) with some modernisation of vocabulary. Alongside MSA, a spectrum of spoken dialects exists (Egyptian, Gulf, Levantine, Maghrebi, etc.) that differ substantially from MSA and from each other in phonology, morphology, and lexicon. This study works with MSA throughout, as PATS-A01 documents are printed in MSA and the QALB and OpenITI corpora also contain MSA text.

2.1.2 The Arabic Script

Arabic is written with a 28-letter abjad (consonantal alphabet) that runs from right to left. Several properties of the script are critical for understanding OCR failure modes.

Positional forms. Most letters have four shapes depending on their position within a word: isolated, initial, medial, and final. The letter *ain* (ع), for instance, appears as ع (isolated), ا (initial), ـع (medial), and ـع (final). Since letters are connected into words by ligatures (joined strokes), an OCR engine must recognise not just a character in isolation but a glyph in context, making segmentation errors compound recognition errors.

Diacritics (tashkeel / harakat). Short vowels in Arabic are written as small marks placed above or below consonant letters. The primary marks are: fatha (short /a/, above), kasra (short /i/, below), damma (short /u/, above), sukun (consonant

with no vowel, above), and shadda (gemination, above). Tanwin (nunation) suffixes – tanwin al-fath, tanwin al-kasr, tanwin al-damm – mark indefinite nouns and consist of doubled fatha, kasra, or damma marks. In most modern Arabic text, including newspapers, diacritics are *omitted* from printed text and readers infer the correct vowelisation from context. When present, diacritics are critical for disambiguation: the same consonant sequence can represent multiple words depending on vowelisation.

Hamza. The glottal stop (hamza) has a complex orthography. In some positions it appears alone (ء); in others it appears on a carrier letter: alef (إ above, أ below), waw (ؤ), or ya without dots (ئ). The choice of carrier follows rules based on the surrounding vowels, and even educated native speakers sometimes apply these rules inconsistently. Hamza rules are a major source of spelling errors in human writing and a significant challenge for OCR.

Taa marbuta. The feminine ending ة (taa marbuta) is pronounced /at/ in context and /a/ in pause. It is written with the same base shape as the letter ha (ه) but with two dots above. The visual similarity between ة and ه is a direct source of OCR errors whenever the ink quality, resolution, or font rendering is imperfect.

Alef maqsura. The letter ع (alef maqsura, literally “abbreviated alef”) appears at the end of certain words to write the long vowel /aa/. Its shape is identical to ya (ي) but without the two dots below. OCR engines regularly confuse ع and ي, and the distinction is orthographically important because the two letters signal different word identities.

Dot-group characters. Six groups of letters share the same base skeletal shape and differ only in the count or position of diacritical dots above or below: ب / ت / ث (1/2/3 dots above), ج / ح / خ (1 dot below / none / 1 dot above), ف / ق (1 dot above / 2 dots above), ن / ي (1 dot above / 2 dots below), and others. Dot placement errors – caused by ink smearing, low resolution, or font rendering artefacts – are the most frequent single-character OCR confusion in Arabic typewritten text.

2.1.3 Arabic Text in Computing

Arabic text in digital systems is encoded in Unicode. The primary Arabic Unicode block spans U+0600 to U+06FF and covers the standard 28 letters, their positional variants, diacritical marks, punctuation, and numerals. The diacritical marks (tashkeel) occupy code points U+064B through U+065F for the standard marks, with additional marks in the ranges U+0610–U+061A and U+0670. These code points can be removed programmatically to strip diacritics without affecting the consonantal skeleton.

Right-to-left rendering requires bidirectional text support. The Unicode Bidirectional Algorithm (UBA) governs how mixed LTR/RTL content is laid out. Documents containing both Arabic and numerals (which are LTR) or Arabic and English words exhibit complex rendering behaviour that can cause misalignment between stored byte order and visual display order. OCR engines and NLP tools must handle this correctly to avoid misinterpreting character sequences.

Legacy encodings such as Windows Code Page 1256 (CP-1256), ISO-8859-6, and the ASMO-708 standard predate Unicode and are still found in digitised Arabic document collections. The PATS-A01 ground-truth files, for example, are stored in CP-1256. Correct decoding requires explicit encoding specification; reading an Arabic CP-1256 file as UTF-8 produces garbled output. All text in this study is converted to UTF-8 on load.

2.1.4 Challenges for Arabic NLP

Tokenisation. Arabic word boundaries are clear (words are separated by spaces), but morpheme boundaries within words are not. Arabic clitics – conjunctions, prepositions, definite articles, pronominal suffixes – are written attached to the host word without any separator. Tokenising the sentence into morphological units requires a separate segmentation step beyond whitespace splitting.

Vocabulary sparsity. The large number of inflectional and derivational forms

per lemma means that any fixed-vocabulary system encounters out-of-vocabulary tokens frequently unless the vocabulary is very large. Subword tokenisation approaches (byte-pair encoding, WordPiece) partially address this but fragment morphologically complex words into unintuitive subword units.

Dialectal variation. Egyptian, Gulf, Levantine, and Maghrebi dialects differ from MSA and from each other in phonology, morphology, and lexicon. A model trained on MSA may perform poorly on dialectal text. This study focuses on MSA text throughout.

Code-switching. Modern Arabic digital text frequently mixes Arabic script with embedded English words, numbers, emoticons, and URLs. Handling such mixed-script content robustly requires robust character-level processing.

2.2 Optical Character Recognition

2.2.1 What Is OCR?

Optical Character Recognition (OCR) is the process of converting images of text into machine-readable character sequences. The classical OCR pipeline consists of four stages:

1. **Preprocessing.** The scanned or photographed document image is prepared for analysis: binarisation (converting to black-and-white), deskewing (correcting rotational distortion), denoising (removing salt-and-pepper and background noise), and normalisation of brightness and contrast.
2. **Layout analysis.** The preprocessed image is segmented into regions: page borders, columns, paragraphs, text lines, and (in character-segmenting systems) individual character images. For Arabic, line detection must handle the right-to-left reading order and the slanted or curved baselines that appear in handwritten text.

3. **Character recognition.** The character recognition module maps each segmented glyph (or a sequence of glyphs, in modern end-to-end systems) to a Unicode character. This is the core of OCR and the component that has changed most dramatically with deep learning.
4. **Post-processing.** A language model or lexicon is consulted to resolve ambiguous recognitions and correct plausible errors. In classical systems this was a rule-based or n-gram language model step. In modern systems it is sometimes omitted because the recognition model is end-to-end and incorporates linguistic knowledge implicitly.

2.2.2 OCR Architectures

The OCR field has passed through three distinct architectural eras:

Template matching (pre-2000). Early systems compared each segmented character image against a library of stored templates and selected the nearest match by pixel distance or correlation [0]. Template matching is brittle with respect to font variation, noise, and degradation, and requires re-training the template library for each new font.

Hidden Markov Models (1990s–2010s). HMM-based systems model the sequence of observations (image features along a text line) as a probabilistic generative process. They can handle cursive scripts without explicit character segmentation, which made them the dominant approach for Arabic OCR through the 2000s. A character language model is typically combined with the HMM in a noisy channel framework to improve recognition.

Deep learning era (2014–present). Convolutional neural networks replaced hand-crafted feature extraction. The combination of CNN feature extraction, Bidirectional LSTM sequence modelling, and Connectionist Temporal Classification (CTC) decoding [0] became the dominant paradigm for text-line recognition. Tesseract v4, the open-source OCR engine on which Qaari is based, adopted this CNN-LSTM-CTC architecture [0].

Transformer-based OCR (2021–present). Vision-language models such as TrOCR [0] and Donut [0] treat OCR as a sequence-to-sequence generation task: a vision encoder encodes the document image, and a text decoder generates the recognised text token by token. These models are pre-trained on large paired image-text datasets and achieve state-of-the-art performance on printed and even handwritten text. However, they require substantial compute and training data, which limits their availability for low-resource languages and scripts.

2.2.3 Arabic-Specific OCR Challenges

The properties of the Arabic script described in Section 2.1 translate directly into OCR failure modes.

Segmentation errors. Because Arabic letters are connected within a word, a segmentation error (incorrectly splitting a ligature or incorrectly merging two letters) cascades into recognition errors for all characters involved. English OCR, which deals mostly with separated or lightly connected Latin characters, does not face this problem to the same degree.

Dot placement ambiguity. The fine dots above and below base letter shapes carry discriminative information but are small relative to the letter body. Low image resolution, ink bleed, or scan artefacts can render dots illegible, and the OCR engine must then guess between all members of the relevant dot group. This is the most frequent source of character-level errors in PATS-A01.

Ligature handling. Certain letter combinations in Arabic merge into single ligature glyphs with a shape that may not visually resemble the component letters in isolation. Common examples include lam-alef (ﻻ), which is always written as a single glyph. OCR engines trained on character-level templates may not include all ligature shapes, leading to substitution or insertion errors.

Diacritic sensitivity. OCR engines vary in whether they attempt to recognise diacritics. Many ignore them entirely, which is appropriate for most modern Arabic text but problematic for classical or religious text where diacritics are always present.

Others attempt recognition but produce errors when diacritics are present in unusual positions or rendered in non-standard fonts.

Baseline variability. In handwritten text, words do not lie on a flat baseline. Variable baseline trajectories challenge text-line segmentation and make aligned feature extraction more difficult.

2.2.4 Qaari: The OCR Engine Used in This Study

Qaari is an open-source Arabic OCR engine based on Tesseract v4 with Arabic LSTM models trained specifically for Arabic printed text. It represents the class of freely available, community-maintained OCR engines that are accessible to researchers and practitioners who cannot afford commercial OCR solutions.

In the experiments of this thesis, Qaari performs well on clean PATS-A01 Akhbar text: the character error rate on normal samples is 1.37% (Section ??). However, Qaari exhibits one notable failure mode: a *repetition artifact*. When processing certain document layouts, Qaari enters a loop in which it repeatedly outputs the same short text span, producing a hypothesis that is hundreds of times longer than the ground truth. This artifact is rare (observed in 1 out of 200 samples, 0.5%) but when it occurs it produces an extremely high per-sample CER that dominates the aggregate. The artifact is a real-world behaviour of the OCR engine and is included in all evaluations to preserve ecological validity.

2.2.5 Evaluation Metrics for OCR

The two standard metrics for evaluating OCR and post-OCR correction are Character Error Rate (CER) and Word Error Rate (WER), both defined in Section 3.3 (Chapter 3). Here we comment on the choice of CER as the primary metric and on the diacritics stripping decision.

Why CER over WER. Arabic morphology creates a large space of valid word forms; any character-level error produces a word-level error. WER is therefore

always higher than CER for Arabic text and is more sensitive to the tokenisation convention used. CER captures the actual number of character edits needed, which is more directly interpretable as a measure of recognition accuracy at the signal level. CER is also the standard primary metric in post-OCR correction shared tasks [0].

Diacritics stripping. Evaluating with diacritics would penalise a corrector for failing to reproduce or adding diacritics that are stylistically optional. Since neither Qaari nor the LLM corrector is designed to hallucinate or remove diacritics that are part of the ground truth, evaluating without diacritics provides a cleaner measure of lexical correction quality. Both variants are computed and stored; the diacritic-stripped metric is reported as primary throughout this thesis.

2.3 Large Language Models

2.3.1 From N-grams to Transformers

Statistical language modelling has a long history in NLP. N-gram language models assign a probability to a sequence of words by decomposing the joint probability into a product of conditional probabilities of each word given the preceding $n - 1$ words. They were the dominant approach for language modelling and speech recognition through the 1990s and 2000s. Their limitations – sparse counts for long contexts and inability to generalise across syntactically similar contexts – motivated neural approaches.

Bengio et al. [0] introduced the neural probabilistic language model, which represents words as dense real-valued vectors (embeddings) and uses a feedforward network to predict the next word given a fixed-width context window. This model demonstrated that neural approaches could outperform n-gram models by generalising across similar word contexts.

Recurrent Neural Networks (RNNs) extended this to variable-length contexts. Long Short-Term Memory (LSTM) networks [0] addressed the vanishing gradient problem of vanilla RNNs with gated memory cells, enabling effective modelling

of long-range dependencies. LSTM-based language models and encoder-decoder sequence-to-sequence models dominated NLP in the early 2010s.

The transformer architecture [0], introduced in 2017, replaced recurrence with self-attention mechanisms. Transformers train orders of magnitude faster than LSTMs on parallel hardware and scale more effectively with data and model size. All modern large language models are transformer-based.

2.3.2 The Transformer Architecture

The key innovation of the transformer is the *self-attention mechanism*, which allows every token in a sequence to directly attend to every other token, regardless of their distance. Given a sequence of token embeddings represented as a matrix $X \in \mathbb{R}^{nd}$, the scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

where $Q = XW^Q$, $K = XW^K$, $V = XW^V$ are learned linear projections of the input into query, key, and value spaces, and d_k is the key dimension used for scaling. The softmax produces an attention weight distribution over all positions, and the output is a weighted sum of value vectors.

Multi-head attention runs h independent attention operations in parallel, each with its own projection matrices, and concatenates their outputs:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.2)$$

This allows the model to attend to information from different representational subspaces simultaneously.

Positional encoding is added to the input embeddings to provide information about token order, since the attention operation is otherwise position-invariant. Original transformers used sinusoidal positional encodings; modern LLMs often use rotary positional embeddings (RoPE) or similar learned variants.

A transformer block consists of a multi-head self-attention sublayer followed by a position-wise feedforward network, with residual connections and layer normalisation around each sublayer. Modern LLMs are decoder-only transformers: they use masked (causal) self-attention so that each token can only attend to tokens at or before its position, enabling autoregressive generation.

2.3.3 Pre-Training and Instruction Tuning

Modern LLMs are trained in two stages.

Pre-training. The model is trained to predict the next token given all preceding tokens, on a massive corpus of text scraped from the internet, books, code, and other sources. Pre-training instils broad language competence, world knowledge, and the ability to reason across many domains, all from the statistical structure of the training corpus. GPT-3 [0] demonstrated that scaling pre-training to 175 billion parameters enables dramatic improvements in few-shot performance across NLP tasks.

Instruction tuning and RLHF. A pre-trained model produces likely continuations of text; it does not reliably follow explicit instructions. Instruction tuning [0] fine-tunes the model on a curated set of instruction-response pairs, teaching it to interpret instructions as tasks and produce helpful, structured outputs. Reinforcement Learning from Human Feedback (RLHF) further aligns the model’s outputs with human preferences by training a reward model on human comparisons and optimising the language model against it using policy gradient methods. The result is an instruction-tuned model that follows natural language instructions reliably across a wide range of tasks without requiring task-specific fine-tuning.

Instruction-tuned models are the right choice for post-OCR correction because they can follow a correction instruction directly. Unlike pre-trained base models, which require careful prompt engineering to elicit a specific output format, instruction-tuned models respond predictably to system messages of the form “correct this text and return only the corrected version.”

2.3.4 Qwen3-4B-Instruct-2507

Qwen3-4B-Instruct-2507 is the primary LLM used in this study. It belongs to the Qwen3 family released by Alibaba’s Tongyi team [0]. Key characteristics relevant to this study are as follows.

Scale and architecture. Qwen3-4B has approximately 4 billion parameters and is a decoder-only transformer. At this scale it fits comfortably in the 16 GB VRAM of a Kaggle T4 GPU in `float16` precision, making it a practical choice for cloud GPU inference without quantization.

Multilingual capability and Arabic support. Qwen3 was pre-trained on a large multilingual corpus covering over 100 languages, with substantial Arabic representation. This enables the model to read, generate, and reason in Arabic without any Arabic-specific fine-tuning.

Instruction following. The `-Instruct` variant is trained with supervised fine-tuning and preference alignment to follow instructions reliably. This is the essential property for post-OCR correction, where the model must interpret a system prompt and produce a structured text output.

Thinking mode. Qwen3 models include an optional chain-of-thought reasoning mode controlled by the `enable_thinking` parameter in `apply_chat_template()`. When enabled, the model generates an extended internal reasoning trace enclosed in `<think>...</think>` tokens before producing its final answer. This study disables thinking mode (`enable_thinking=False`) for two reasons: first, post-OCR correction is a direct transformation task that does not benefit from extended chain-of-thought reasoning; second, keeping thinking mode enabled adds latency and token cost, and the thinking trace contents sometimes contaminate the extracted correction.

2.3.5 Arabic LLMs and Arabic NLP with LLMs

The Arabic NLP community has produced several dedicated pre-trained models.

AraBERT [0] is an encoder-only BERT model pre-trained on a large corpus of Arabic news and Wikipedia text. It achieves strong results on Arabic classification and understanding tasks and established that pre-training specifically on Arabic outperforms multilingual BERT on Arabic benchmarks at comparable model size.

CAMeLBERT [0] extends this line of work with models pre-trained on MSA, dialectal Arabic, and mixed-domain corpora, providing tools tailored to different Arabic registers.

AraGPT2 [0] and **AraT5** [0] bring generative and text-to-text capabilities to Arabic, enabling Arabic language generation and conditional text transformation.

These Arabic-specific models, while valuable for their target tasks, operate at scales (hundreds of millions of parameters) far below current instruction-tuned LLMs. More recent results have shown that large multilingual instruction-tuned models such as GPT-4, Llama-3, and Qwen3 outperform Arabic-specific models on most Arabic NLP benchmarks, primarily because the scale advantage outweighs the benefit of Arabic-only pre-training. Qwen3-4B was chosen for this study as the most capable openly accessible model at a size deployable on free cloud GPU tiers.

2.4 Prompt Engineering

2.4.1 Zero-Shot Prompting

In zero-shot prompting, the model receives an instruction and an input without any demonstration examples [0]. Performance depends entirely on the clarity of the instruction and the model’s pre-training coverage of the task. For post-OCR correction, the zero-shot instruction must communicate both the task (correct errors in the text) and the constraint (do not change what is correct).

A key design decision is the degree of *aggressiveness* in the instruction. An aggressive prompt (“correct all errors in this text”) primes the model to intervene on every uncertainty, producing rewrites of already-correct tokens. A conservative

prompt (“return the text unchanged if it is correct; fix only clear errors”) primes the model to leave the text alone unless it is confident of an error. For a task where the vast majority of tokens in the input are already correct (OCR baseline CER of 1.37% on normal samples), conservative prompting is the rational choice. The design and evaluation of this choice is a central finding of Chapter 5.

2.4.2 Few-Shot Prompting

Few-shot prompting, introduced in GPT-3 [0], provides the model with a small number of input-output demonstration pairs before the actual query. This enables *in-context learning*: the model infers the task format and the expected transformation from the examples without any weight update.

Several practical decisions govern few-shot performance:

- **Number of examples.** More examples generally improve performance up to the point where they saturate the context window. This study uses 5 examples, balancing context length against representativeness.
- **Example selection.** Random selection may draw all examples from the same error type. Diversity-maximising selection ensures that the examples span multiple error categories, giving the model a broader template for what corrections look like.
- **Source of examples.** Phase 4B uses QALB corpus pairs. QALB annotations cover human typing errors (omitted diacritics, wrong hamza form, case-agreement errors) that have a partial but imperfect overlap with OCR errors (visual confusion between similar-shape characters). This *transfer gap* – the mismatch between human-error and OCR-error distributions – is a potential explanation for the performance outcome discussed in Chapter 5.

2.4.3 Chain-of-Thought and Instruction Augmentation

Chain-of-thought (CoT) prompting [0] instructs the model to generate intermediate reasoning steps before producing the final answer. CoT has been shown to improve performance on multi-step reasoning tasks such as arithmetic, symbolic manipulation, and commonsense inference.

Chain-of-thought is not used in this study. Post-OCR correction is a one-pass text-to-text transformation that does not require multi-step reasoning: for each position in the input, the model either copies the character or replaces it. There is no decomposition into sub-problems that would benefit from intermediate reasoning. Furthermore, enabling chain-of-thought in Qwen3 (via the thinking mode) adds substantial latency, unpredictable output length, and the risk of the reasoning trace contaminating the extracted correction. The conservative instruction in the system prompt already encodes the key cognitive constraint (“prefer no change unless certain”) without requiring explicit reasoning steps.

2.4.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [0] combines a parametric language model with a non-parametric retrieval component. At inference time, a retrieval system queries a large external corpus with an embedding of the input, retrieves the most similar passages, and provides these as additional context in the prompt. This allows the model to ground its outputs in specific textual evidence rather than relying solely on knowledge encoded in its weights.

The RAG pipeline consists of the following components:

1. **Corpus indexing.** The external corpus is split into sentences or passages, each embedded by a dense encoder into a fixed-dimensional vector. Embeddings are stored in a vector index that supports efficient nearest-neighbour search.
2. **Query encoding.** The input text is embedded using the same encoder.

3. **Retrieval.** The index is searched for the k corpus items with embeddings most similar to the query, typically by inner product or cosine similarity. This study uses the FAISS [0] library with a flat inner-product index.
4. **Prompt augmentation.** The retrieved passages are formatted and prepended to the system prompt as reference context.

This study uses the `paraphrase-multilingual-MiniLM-L12-v2` model [0] for both corpus and query encoding. This is a compact 12-layer multilingual sentence encoder with strong Arabic support, suitable for embedding the 200,000-sentence OpenITI index on a CPU without excessive memory usage.

An important consideration for RAG quality is *domain match*: if the retrieval corpus covers a different domain or time period than the input text, retrieved passages may not be relevant and could mislead the model. The domain mismatch between the classical OpenITI corpus and the modern newspaper PATS-A01 text is a central element of the Phase 5 analysis in Chapter 5.

2.4.5 Self-Reflective and Meta-Cognitive Prompting

Self-reflective prompting refers to techniques that incorporate information about the model’s own behaviour into the prompt to guide future outputs. The Reflexion framework [0] demonstrates that language agents can improve over time by receiving verbal feedback about their past failures and incorporating this feedback into their decision-making. Similarly, self-critique prompting asks the model to evaluate its own output and revise it, akin to verbal reinforcement learning.

Phase 4D operationalises a lightweight variant of this idea. Rather than asking the model to introspect on its current output at inference time – which adds latency and the risk of circular reasoning – the model is given *pre-computed statistics* about its systematic failure patterns, derived from analysing its corrections on a separate training split. These statistics are converted to Arabic diagnostic sentences injected into the system prompt. The approach is a form of offline meta-learning: the model’s

systematic biases are characterised before deployment, and this characterisation is used to nudge its inference behaviour.

This design differs from standard Reflexion in two ways: it does not require an execution environment or reward signal; and it provides population-level statistics (“you fix insertion errors 97% of the time but introduce errors on 25% of correct tokens”) rather than sample-level feedback. Whether this coarser form of self-knowledge translates into improved behaviour at inference time is one of the empirical questions studied in this thesis.

2.5 Knowledge Sources Used in This Study

2.5.1 Confusion Matrix as a Knowledge Source

An OCR confusion matrix records, for each ground-truth character, the frequency with which the OCR engine substituted other characters in its place. It is a compact, data-driven summary of the engine’s systematic recognition errors. Unlike generic Arabic spelling rules, a confusion matrix is engine-specific: it reflects the particular visual features that this engine’s neural network has difficulty discriminating.

In this study, the confusion matrix is generated from Phase 1 analysis (Section 3.2 and Chapter 4) and is used in Phase 3 to inject the top-10 most frequent confusion pairs into the system prompt. The hypothesis is that naming these specific pairs directs the model’s attention to the character positions most likely to contain errors.

2.5.2 Arabic Orthographic Rules

Arabic orthographic rules govern the correct spelling of characters that OCR commonly misidentifies. The rules used in this study address: hamza placement (al-qat’ vs. al-wasl), taa marbuta vs. ha, alef maqsura vs. ya, tanwin placement, and definite article assimilation. These rules are grounded in classical Arabic grammar and are taught in Arabic primary education. They are stored in the `data/rules/`

directory as JSONL files and loaded by `RulesLoader` at runtime.

Unlike the confusion matrix (which is descriptive – “this engine makes this mistake”), orthographic rules are prescriptive – “the correct form is X”. Both types of knowledge tell the model what to look for; the confusion matrix focuses its attention by describing the engine’s error patterns, while the rules frame the correction task in terms of canonical Arabic spelling.

2.5.3 The QALB Corpus

The Qatar Arabic Language Bank (QALB) was developed for the QALB Shared Task on Automatic Arabic Text Correction [0]. It contains thousands of human-authored Arabic sentences from online news comments, annotated with corrections for spelling, grammatical, punctuation, and other errors by trained annotators.

QALB is used in this study as a source of few-shot demonstration pairs for Phase 4B. After filtering for OCR-relevance and length (Section ??, Chapter 3), 528 pairs are available. The primary limitation of QALB as an OCR correction resource is the *transfer gap*: QALB errors are human typing errors, while OCR errors are visual confusion errors. Human typers tend to make phonological errors (writing a word as it sounds in dialectal pronunciation) or agreement errors (wrong gender or case marking), while OCR engines make visual confusion errors (confusing characters with similar shapes). The overlap between these two error distributions is partial, and the few-shot examples do not perfectly model what the model is being asked to correct.

2.5.4 The OpenITI Corpus

The Open Islamicate Texts Initiative (OpenITI) is a large-scale digitised collection of texts from the Islamicate scholarly tradition, containing classical Arabic, Persian, and other languages [0]. With hundreds of millions of words of Arabic text spanning many centuries and genres, it provides a large pool of grammatically correct Arabic

sentences for dense retrieval.

OpenITI is used in Phase 5 as the retrieval corpus for RAG. Its primary limitation for this application is *domain mismatch*: OpenITI texts are classical and historical, while PATS-A01 Akhbar documents are modern newspaper text. The semantic and lexical overlap between medieval religious scholarship and contemporary news reporting is low, which affects retrieval quality. A retrieval corpus drawn from modern Arabic newspapers would be a better match for the PATS-A01 domain.

2.5.5 CAMEL Tools

CAMEL Tools [0] is an open-source Python toolkit for Arabic NLP, developed at the Computational Approaches to Modeling Language Lab at NYU Abu Dhabi. The components used in this study are:

MorphologicalAnalyzer. Given an Arabic word token, the analyser queries the MSA morphological database (`calima-msa-r13`) and returns all valid morphological analyses: each analysis specifies the lemma, root, word pattern, part-of-speech, and inflectional features (gender, number, case, tense, etc.). A word with zero analyses is morphologically invalid – it is not a recognisable MSA word form.

Application in Phase 1. CAMEL Tools is used in Phase 1 to classify OCR error tokens as either *non-words* (no valid morphological analysis) or *valid-but-wrong* (at least one valid analysis but not the correct one in context). The distinction is informative: if most OCR errors are valid-but-wrong, then dictionary lookup cannot detect them and context is required for correction.

Application in Phase 4C. The *revert strategy* described in Chapter 3 uses the morphological analyser to post-process LLM corrections. Words introduced by the LLM that fail morphological analysis while the original OCR token passed are reverted to the OCR original.

2.6 Related Work on Post-OCR Correction

2.6.1 Classical Approaches

The earliest computational approaches to post-OCR correction relied on lexicons and language models to identify and resolve implausible recognitions. Tong and Evans [0] proposed a statistical approach using a unigram word model and a character-level confusion model derived from known OCR error patterns. Their method identified likely errors by flagging tokens not found in the lexicon or whose recognition score was below a threshold, then proposed corrections by searching the lexicon for candidates within a small edit distance. This work established the noisy channel framework for OCR correction that remained influential for over a decade.

Kolak and Resnik [0] developed this framework further with an explicit noisy channel model combining character-level OCR confusion probabilities with a word-level language model. Their system formulated post-OCR correction as a decoding problem: given the OCR output, find the most probable original text under the joint model. This framing is conceptually equivalent to the machine translation noisy channel model and shares its strengths (principled probabilistic framework) and limitations (requires domain-specific language models and large training corpora of OCR errors).

The limitation common to all classical approaches is their reliance on fixed-vocabulary lexicons and short-context language models. They cannot resolve *valid-but-wrong* errors – tokens that are real words but not the intended word in context – without long-range context, and they require error-specific training data for each OCR engine being corrected.

2.6.2 Neural Sequence-to-Sequence Correction

The deep learning era brought encoder-decoder models to post-OCR correction. Dong and Smith [0] applied a multi-input attention model that combines character-

level and word-level representations of the OCR output, enabling the model to attend to both local character confusion patterns and broader word-level context. Their system significantly outperformed classical methods on historical English document collections, demonstrating that neural sequence models can implicitly learn OCR error distributions from paired training data.

Transformer-based sequence-to-sequence models have since been applied to post-OCR correction with further improvements. The ICDAR 2019 Post-OCR Correction Competition [0] attracted systems based on BERT, character-level transformers, and ensemble methods. These systems substantially improved over the OCR baseline on historical French and English text.

The fundamental challenge for these approaches in the Arabic context is the *scarcity of training data*. Neural sequence-to-sequence correction requires large quantities of paired (OCR output, ground truth) examples to learn the error model for a specific engine. Such data is relatively abundant for historical English and French (where digitised collections with corresponding scans and transcripts exist), but scarce for modern Arabic. Creating a training set for Arabic OCR correction would require either manual transcription of thousands of document scans or a data augmentation strategy that synthetically generates OCR errors. This data requirement motivates the zero-shot and prompt-based approaches investigated in this thesis, which require no paired training data.

2.6.3 LLM-Based Post-OCR Correction

The emergence of large instruction-tuned language models created a new paradigm for post-OCR correction: the model is instructed to correct errors in the OCR output without any task-specific fine-tuning. Kang et al. [0] demonstrated that ChatGPT-level models are effective at English OCR post-correction in a zero-shot setting, achieving substantial CER reductions on several historical English document benchmarks. Their results show that the implicit knowledge about spelling, grammar, and vocabulary encoded in pre-training is sufficient to correct many OCR errors without any paired training data.

This line of work directly motivates the present study. However, Kang et al. and similar work focus entirely on English. Whether the same approach generalises to Arabic – a morphologically richer script-level script with a different error distribution – is not addressed. Furthermore, no prior work has compared multiple prompt augmentation strategies under controlled conditions for Arabic OCR correction.

The phenomenon of *over-correction* – where the LLM changes tokens that the OCR rendered correctly, introducing errors rather than fixing them – is discussed qualitatively in some LLM correction work but has not been systematically studied as the dominant failure mode in the Arabic setting.

2.6.4 Arabic-Specific OCR Correction

Work specifically targeting Arabic OCR correction is limited. Several groups have applied AraBERT fine-tuned on QALB-like data to Arabic error correction, achieving competitive results on the QALB shared task. However, the QALB task targets human writing errors in online text comments rather than OCR-specific visual confusion errors, and fine-tuned models trained on QALB data are not well-calibrated for the OCR error distribution.

Rule-based approaches using Arabic spell-checkers such as Hunspell with Arabic dictionaries have also been applied to post-processing Arabic OCR output. These systems can flag non-word tokens (similar to classical approaches) but cannot resolve valid-but-wrong errors.

To the best of the author’s knowledge, no published study has: (a) systematically compared multiple distinct prompt augmentation strategies for Arabic post-OCR correction in an isolated experimental design; (b) applied self-reflective prompting to the OCR correction task in any language; or (c) characterised the over-correction failure mode in the Arabic post-OCR setting with quantitative evidence.

2.6.5 Positioning of This Work

This thesis makes the following contributions relative to the existing literature:

1. **Systematic evaluation of six isolated augmentation strategies.** Prior work on LLM-based OCR correction has at most compared zero-shot with one or two augmentation conditions. This study isolates six conditions (confusion matrix, orthographic rules, few-shot examples, morphological post-processing, self-reflective prompting, and RAG), each compared against the same zero-shot baseline using the same model, dataset, and evaluation protocol.
2. **Self-reflective prompting for OCR correction.** The Phase 4D strategy – injecting pre-computed statistics about the model’s own training-split error patterns into the inference prompt – is, to the author’s knowledge, the first application of self-reflective prompting to OCR correction in any language.
3. **Quantitative study of over-correction in Arabic OCR.** The finding that the LLM introduces errors on 25.06% of previously correct tokens, and that this rate increases as more knowledge is injected into the prompt, is a novel empirical characterisation of a failure mode that deserves attention from practitioners deploying LLMs for text normalisation tasks.
4. **Arabic-specific evaluation.** The PATS-A01 Akhbar dataset provides a controlled typewritten Arabic testbed with known font characteristics and a clean ground truth, enabling precise attribution of CER changes to specific prompt interventions.

Chapter 3

System Design and Datasets

This chapter describes the complete experimental infrastructure underlying the study. The goal is to provide sufficient detail for full reproducibility: a reader should be able to reconstruct the pipeline, datasets, preprocessing steps, and software modules from this chapter alone. Section 3.1 introduces the three-stage pipeline architecture. Section 3.2 describes the corpora used. Section 3.3 covers preprocessing and evaluation. Section ?? describes the knowledge base components used across phases. Section ?? documents the LLM correction module. Section ?? presents the `PromptBuilder` and all prompt variants.

3.1 Overall Architecture

The primary constraint shaping system design is the absence of local GPU access. The research workstation can load and process text data efficiently but cannot run a 4-billion-parameter language model at useful inference speed. At the same time, cloud GPU platforms such as Kaggle and Google Colab provide free access to NVIDIA T4 GPUs that are well-suited for this model size. The architecture is designed to exploit this resource split cleanly.

3.1.1 Three-Stage Pipeline

The pipeline is divided into three stages that communicate through a single intermediary file in JSON Lines format (JSONL). Fig. 3.1 illustrates the flow.

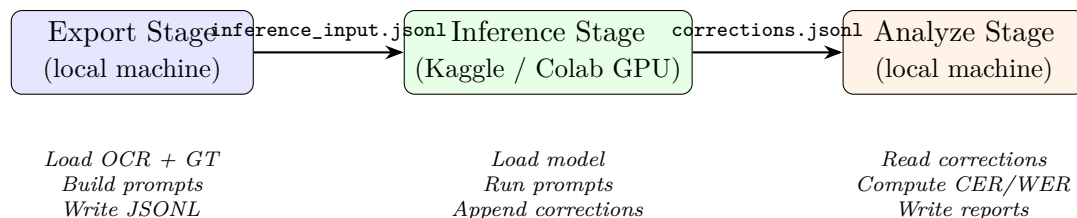


Figure 3.1: Three-stage pipeline architecture. Stages communicate via JSONL files on disk, enabling inference to run on any cloud GPU without requiring direct access to local data files.

Stage 1 — Export runs on the local machine. Each phase pipeline (e.g. `pipelines/run_phase2.py`) loads the dataset, pairs each OCR prediction with its ground-truth text, builds the phase-specific prompt for each sample using `PromptBuilder`, and writes one JSON object per sample to `inference_input.jsonl`. The file is stored under a phase- and dataset-specific results directory and is transferred to the cloud environment by cloning the repository directly.

Stage 2 — Inference runs on a Kaggle kernel or Google Colab notebook. The script `scripts/infer.py` is the unified inference entry point for all phases. It loads the model and tokenizer once at startup, then reads `inference_input.jsonl` line by line, generates a correction for each prompt, and appends the result to `corrections.jsonl`. A resume mechanism reads the output file on startup and skips any sample whose `sample_id` already appears in it, making the inference step fully restartable after an interruption without duplicating work.

Stage 3 — Analyze runs on the local machine after inference completes. The phase pipeline reads `corrections.jsonl`, computes CER and WER for each sample by comparing the corrected text against the ground truth, aggregates statistics across the dataset, and writes structured JSON reports and a human-readable Markdown summary.

3.1.2 JSONL Record Format

Each line in both JSONL files is a self-contained JSON object. Table 3.1 lists all fields and their purpose.

Table 3.1: Fields in the inference JSONL files. All fields from the export stage are preserved through inference; the `correction` and status fields are appended by the inference script.

Field	Stage added	Description
<code>sample_id</code>	Export	Unique identifier: <code>dataset:file_stem</code>
<code>dataset</code>	Export	Dataset key, e.g. <code>PATS-A01-Akhbar-train</code>
<code>ocr_text</code>	Export	Raw Qaari prediction string
<code>gt_text</code>	Export	Ground-truth text (for analysis stage only)
<code>prompt</code>	Export	Serialised chat messages list (JSON array)
<code>prompt_version</code>	Export	Version tag, e.g. <code>p2v2</code> , <code>p3v1</code>
<code>correction</code>	Inference	LLM output text
<code>success</code>	Inference	<code>true</code> if generation succeeded; else fallback used
<code>latency_s</code>	Inference	Wall-clock generation time in seconds

This design has several practical advantages. The inference machine needs no file-system access beyond the JSONL input file: all ground-truth text, phase-specific context, and metadata are embedded in each record at export time. Adding a new phase requires only implementing a new `PromptBuilder` method and a new pipeline script; the inference script and analysis logic are unchanged. Finally, the JSONL format is human-readable and can be inspected with any text editor or streamed through standard command-line tools for quick sanity checks.

3.1.3 Resume and Force-Rerun Behaviour

All three stages implement resumable execution. The export stage checks whether the target `inference_input.jsonl` already contains a record for a given dataset key before processing it, enabling partial re-runs after adding new datasets. The inference stage reads the output file on startup and populates a set of completed `sample_id` values; any sample already in the set is skipped. The analyze stage checks whether the

primary output file (`results/<phase>/<dataset>/metrics.json`) already exists before recomputing metrics. All stages accept a `-force` flag to override resume behaviour and reprocess everything from scratch. This design prevents accidental data loss while keeping re-runs cheap when only a subset of datasets changes.

3.2 Datasets

The study uses six Arabic text datasets spanning thirteen distinct evaluation configurations: eight typewritten fonts from PATS-A01, one handwritten line-level corpus (KHATT), and four full-page document datasets (KHATT-Paragraph, Yarmouk, Muharaf, and Historical Arabic Manuscripts). The nine PATS-A01 and KHATT configurations form Experiment 1; all thirteen configurations are evaluated in Experiment 2. All validation and testing splits are evaluated across the experimental phases on their respective splits.

3.2.1 PATS-A01 — Printed Arabic Text Set

PATS-A01 is a publicly available Arabic typewritten text recognition dataset. It contains page images of Arabic text typeset in eight distinct fonts, together with line-level and page-level ground truth.

3.2.1.1 Fonts and Coverage

The eight fonts included in PATS-A01 are: Akhbar, Andalus, Arial, DecoType Naskh, Simplified Arabic, Tahoma, Thuluth, and Traditional Arabic. These fonts span a wide range of visual styles from the clean modern sans-serif Tahoma to the decorative classical Thuluth, providing diversity in glyph shape and rendering complexity that elicits different OCR error patterns across fonts. All eight fonts are evaluated in this thesis to capture font-dependent effects on both OCR quality and LLM correction effectiveness.

3.2.1.2 Ground-Truth Encoding

Ground-truth files are encoded in Windows Arabic Code Page 1256 (CP-1256), a legacy single-byte encoding used widely in Arabic software on Windows systems. The DataLoader (`src/data/data_loader.py`) converts all ground-truth text to UTF-8 at load time before any downstream processing. The Traditional font uses a differently named ground-truth file (`TraditionalText.txt`) instead of the naming convention followed by the other seven fonts (`A01-{font}Text.txt`); the DataLoader handles this as a special case.

3.2.1.3 Train/Validation Split

PATS-A01 does not provide an official train/validation split. A critical design requirement for this study is that *the same text samples are assigned to the same partition across all eight fonts*. Because all fonts render the same underlying text content, an aligned split ensures that font is the *only* variable when comparing OCR quality or correction effectiveness across typefaces. Without alignment, each font would have a different mix of easy and hard sentences in its validation set, confounding font effects with content effects.

The split is generated by the script `scripts/generate_pats_splits.py` using the following procedure:

1. Extract the numeric sample index from each filename across all eight fonts (e.g., `Akhbar_1234.txt` yields index 1234).
2. Compute the intersection of numeric indices across all fonts to identify samples present in every font (2,766 common samples).
3. Shuffle the common indices once with a fixed random seed (42).
4. Assign the first 15% of shuffled indices to the validation partition and the remaining 85% to training.
5. Apply this single partition to all eight fonts.

The resulting aligned split yields 2,351 training samples and 415 validation samples per font. The split file is stored at `data/ocr-raw-data/PATS_A01_Dataset/pats_splits.json` and records sample identifiers assigned to each partition. Its JSON structure includes fields for the split mode (`aligned`), the random seed, the validation ratio, and the common sample count.

Each dataset configuration entry uses the field `pats_split:` `"train"` or `pats_split:` `"validation"` to select the appropriate partition at load time, ensuring that experiments that evaluate on training data are never accidentally contaminated with validation samples.

The script also supports an `-independent` mode that shuffles each font separately (different samples per font), but the aligned mode is used for all experiments in this thesis.

3.2.1.4 Scope of This Thesis

All experimental phases are evaluated on the validation split of each font (415 samples per font). The training split serves two purposes: Phase 1 baseline analysis on training data provides the confusion matrices and error taxonomies that inform Phases 3–5; and Phase 4 uses training-split corrections to derive the self-reflective insights injected into validation-split prompts.

Results are reported per-font, per-dataset, and as PATS-A01 macro-averages (non-weighted: each font contributes equally regardless of sample count).

3.2.2 KHATT — Khatt Arabic Handwritten Text

KHATT is an Arabic handwritten text dataset [0]. It contains text written by multiple writers in unconstrained natural handwriting, representing realistic variation in stroke style, baseline angle, character connectivity, and ink characteristics. The dataset provides separate training and validation splits. Within this project, Qaari results for KHATT are stored under `data/ocr-results/qaari-results/khatt-data/{split}/{Trainin`

and the DataLoader supports them using the keys `KHATT-train` and `KHATT-validation`.

KHATT is included in the evaluation to provide a complementary handwritten data point alongside the eight typewritten fonts. Handwritten text presents substantially higher baseline OCR error rates due to unconstrained stroke variation, inconsistent character connectivity, and writer-specific idiosyncrasies. The KHATT validation split is evaluated across all experimental phases to test whether knowledge augmentation strategies behave differently when the baseline error rate is much higher than on typewritten text.

3.2.3 KHATT-Paragraph

KHATT-Paragraph is derived from the same underlying writer pool as KHATT but provides paragraph-level samples rather than isolated text lines. Each sample contains multiple handwritten lines belonging to the same paragraph, making it a substantially longer and more contextually coherent unit than any single KHATT line.

The dataset contains 449 validation samples, selected by identifying KHATT paragraph identifiers that overlap with the KHATT line-level validation set. Ground-truth files are encoded in CP-1256 and stored under `data/ocr-raw-data/KHATT_Paragraph/khatt-para` as per-sample plain-text files. Qaari OCR results are stored under `data/ocr-results/qaari-resul`. No training-split OCR was produced for this dataset; only the validation split is used in Experiment 2.

KHATT-Paragraph is included to assess whether paragraph-level context changes LLM correction behaviour relative to line-level KHATT input. Longer inputs provide more surrounding context, which may help the model resolve segmentation errors and character ambiguities that are difficult to interpret from isolated line-level text.

3.2.4 Yarmouk

The Yarmouk dataset consists of printed full-page Arabic documents from Yarmouk University (Jordan), covering encyclopedic and reference content drawn from Wikipedia and similar sources. The corpus spans a wide range of topics presented in formal Modern Standard Arabic prose, typeset in a standard printed font without hand-applied variation.

The testing split used in this study contains 200 documents. Ground-truth text was extracted from HTML source files and normalised to UTF-8 by the script `scripts/prepare_yarmouk_gt.py`. Document images were extracted from single-page PDFs at 300 DPI (`scripts/convert_yarmouk_pdfs.py`), and per-page Qaari OCR results were merged into per-document files using `scripts/merge_yarmouk_ocr_pages.py`. OCR outputs are stored under `data/ocr-results/qaari-results/yarmouk-data/testing/`. Only the testing split is available with Qaari OCR; no training split was produced.

Yarmouk provides the cleanest typewritten condition among the new datasets: formal printed MSA with a consistent layout and high-quality scan. It tests whether correction strategies established on PATS-A01 (line-level printed text) generalise to full-page printed documents with different structural properties (page headers, paragraph breaks, multi-column layouts common in encyclopedic references).

3.2.5 Muharaf

Muharaf is a dataset of historical Arabic handwritten letters from the 19th and 20th centuries, drawn from archival correspondence collections. The letters represent diverse handwriting styles from the late Ottoman and early modern Arabic periods, with characteristic calligraphic and cursive forms that differ substantially from contemporary handwriting as captured in KHATT.

The validation split contains 116 samples, drawn from the official set0 partition (50 validation images, 66 test images, combined into the “validation” split used here). Ground-truth text is stored as per-image UTF-8 plain-text files un-

der `data/ocr-raw-data/Muharaf/gt/`, extracted from JSON annotation files using `scripts/prepare_muharaf_gt.py` (Crossed_out lines are skipped during extraction). Qaari OCR results are stored under `data/ocr-results/qaari-results/muharaf-data/`. No training-split OCR is available for this dataset.

Muharaf is the most challenging new dataset for both OCR and post-OCR correction: the historical writing style, ink degradation, paper texture, and non-standard orthographic conventions all increase both the OCR baseline error rate and the difficulty of recognising the intended text from OCR output alone.

3.2.6 Historical Arabic Manuscripts

The Historical Arabic Handwritten Text Recognition dataset consists of 40 pages drawn from eight historical Arabic books (five pages per book). The books span a range of manuscript styles and periods; ground truth was extracted from DOCX annotation files by the script `scripts/prepare_historical_gt.py`, producing one UTF-8 plain-text file per page under `data/ocr-raw-data/Historical Arabic Handwritten Text Recognition`. Document images were extracted from the same DOCX files into `.../images/Book{N}/`. Qaari OCR results are stored under `data/ocr-results/qaari-results/historical-data/`.

The 40-page size makes this the smallest new dataset in the study. Results should be interpreted with caution given this limited sample count. Manuscript text presents the greatest typographic diversity of any dataset: ligature conventions, glyph proportions, and ink characteristics vary substantially across the eight books, reflecting different scribal schools and centuries of composition.

3.2.7 Dataset Statistics

Table 3.2 summarises all datasets included in the system.

Table 3.2: Dataset summary. All PATS-A01 fonts share the same aligned split (415 validation / 2,351 training samples). KHATT sample counts depend on writer structure. The four new datasets provide validation/testing splits only (no training-split OCR). All validation and testing splits are evaluated in Experiment 2.

Dataset Key	Type	Split	Train	Val/Test	Experiment
PATS-A01-Akhbar	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Andalus	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Arial	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Naskh	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Simplified	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Tahoma	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Thuluth	Typewritten	Both	2,351	415	Exp. 1+2
PATS-A01-Traditional	Typewritten	Both	2,351	415	Exp. 1+2
KHATT	Handwritten	Both	—	—	Exp. 1+2
KHATT-Paragraph	HW Paragraph	Val only	—	449	Exp. 2
Yarmouk	Printed (full-page)	Test only	—	200	Exp. 2
Muharaf	HW Letters (hist.)	Val only	—	116	Exp. 2
Historical	Manuscripts	Val only	—	40	Exp. 2
Total: 13 evaluation datasets (Exp. 1: 9; Exp. 2: 13)					

3.2.8 OCR Results Format

Qaari produces one plain-text `.txt` file per input image. Each file contains the recognised text for that document page. OCR results are stored under `data/ocr-results/qaari-result` with sub-directories organised by dataset type:

- PATS-A01: `qaari-results/pats-a01-data/A01-{font}/`
- KHATT: `qaari-results/khatt-data/{split}/{Training|Validation}/`
- KHATT-Paragraph: `qaari-results/khatt-paragraph-data/{stem}.txt`
- Yarmouk: `qaari-results/yarmouk-data/{split}/{doc_id}.txt`
- Muharaf: `qaari-results/muharaf-data/{stem}.txt`
- Historical: `qaari-results/historical-data/Book{N}/{stem}.txt`

The DataLoader pairs each OCR file with its corresponding ground-truth entry by matching the file stem. The OCR engine is configurable: the configuration field `data.ocr_model` names the sub-directory within `data/ocr-results/` that holds all results for that engine. Changing this field switches the entire pipeline to a different OCR engine’s output directory without any code modification. This design anticipates future comparative studies across multiple Arabic OCR engines.

3.3 Preprocessing and Evaluation

3.3.1 Text Normalisation

Text normalisation is applied identically to both the hypothesis (OCR output or LLM correction) and the reference (ground truth) before any metric is computed. The following transformations are applied:

1. **Diacritic stripping.** Arabic short vowel marks (harakat/tashkeel) occupy Unicode code points U+064B through U+065F and related ranges (U+0610–U+061A for extended marks, U+0670 for the superscript alef). These characters are removed from both texts before computing CER and WER. The rationale is that diacritics are stylistically optional in most modern Arabic text; their presence or absence does not change lexical identity. Penalising a corrector for changing a diacritised form to the same word without diacritics, or for failing to add diacritics that were absent in the OCR input, would conflate orthographic convention with correction quality. Both the raw metric (with diacritics) and the normalised metric (without diacritics) are computed and stored in all result files; the diacritic-stripped variant is reported as primary throughout this thesis.
2. **Tatweel removal.** The Arabic elongation character tatweel (kashida, U+0640) is a decorative stroke that extends word shapes without changing lexical identity. It is removed from both texts before metric computation.

3. **No letter normalisation.** Alef variants, hamza forms, and the alef maqsura / ya distinction are *not* normalised before evaluation. Collapsing these distinctions (e.g. mapping all alef variants to a bare alef) would mask a class of errors that the study specifically aims to measure. The confusion matrix and error taxonomy both depend on the uncollapsed character set.

Normalisation is implemented in `src/data/text_utils.py::normalise_arabic()`.

3.3.2 CER and WER Computation

3.3.2.1 Definitions

Character Error Rate (CER) measures the minimum edit distance between the hypothesis string h and the reference string r at the character level, normalised by the length of the reference:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \quad (3.1)$$

where S_c , D_c , and I_c are the number of character-level substitutions, deletions, and insertions in the minimum-edit alignment, and N_c is the total number of characters in the reference. Word Error Rate (WER) applies the same formula at the word level, treating whitespace-delimited tokens as the atomic unit:

$$\text{WER} = \frac{S_w + D_w + I_w}{N_w} \quad (3.2)$$

CER is the primary metric throughout this thesis. Arabic morphology makes word-level metrics more sensitive to tokenisation choices; character-level errors more directly capture the visual confusion phenomena (dot placement, shape similarity) that are the dominant error type in Arabic OCR.

3.3.2.2 Implementation

CER is computed using the `editdistance` library, which provides an efficient implementation of the Levenshtein distance. WER is computed using the `jiwer` library. Both are computed *per sample first*, then aggregated:

$$\text{CER}_i = \frac{\text{editdistance}(h_i, r_i)}{\max(|r_i|, 1)} \quad (3.3)$$

where $|r_i|$ is the reference character count for sample i . Using per-sample CER (rather than pooling all characters across the corpus) ensures that every sample contributes equally to the aggregate mean regardless of its length. This prevents the few long documents in the dataset from dominating the statistic.

Aggregate statistics reported for each dataset-phase combination are: mean CER (μ), standard deviation (σ), median, and 95th percentile (p95). The standard deviation quantifies variability across samples; the p95 captures worst-case performance and is particularly useful for identifying the impact of catastrophic OCR failures such as the repetition artifact described in Chapter 5.

3.3.3 Error Taxonomy Construction

Phase 1 classifies each substitution error into one of seven categories based on the character pair (ground-truth character, OCR character) found in the minimum-edit alignment between ground-truth and OCR text. Table ?? defines each category and its detection criterion. Categories are applied in order; a substitution that matches an earlier category is not re-counted in a later one.

3.4 Knowledge Base Components

Phases 3 through 5 (and Phase 6 combinations) each inject a different type of external knowledge into the LLM prompt. All knowledge loaders are implemented in `src/data/knowledge_base.py` and share a common interface: a `load()` method

Table 3.3: Error taxonomy used in Phase 1 baseline analysis. Categories are ordered from most specific (dot confusion) to most general (other substitution).

Category	Detection criterion
Dot confusion	Substitution between characters that share the same base glyph and differ only in the number or position of diacritical dots: {ba, ta, tha, nun, ya}, {qaf, fa}, {jim, ha, kha}, {dal, dhal}.
Hamza errors	Substitution among alef variants or between hamza forms: alef, alef with hamza above, alef with hamza below, alef madda, hamza alone, hamza on waw seat, hamza on ya seat.
Taa marbuta	Substitution between taa marbuta (feminine ending) and ha, which share the same base shape and differ only by two dots.
Alef maqsura	Substitution between alef maqsura (word-final /a/ written with a ya-shaped letter without dots) and ya (with dots).
Similar shapes	Substitution between characters sharing the same skeleton but differing in secondary visual features: {ra, zay}, {dal, dhal}, {ta marbuta, ha}, {ta with extended tail (· Ø), dha (,Ø)}.
Merged words	Fewer words in the OCR output than in the ground truth; a word boundary is missing.
Split words	More words in the OCR output than in the ground truth; a spurious word boundary is inserted.
Insertion / Deletion / Other	All remaining edit operations: character insertions, character deletions, and substitutions not classified by any of the above categories.

that reads source data and a `format_for_prompt()` method that returns a plain Arabic string ready for insertion into a system prompt.

3.4.1 Confusion Matrix Loader

`ConfusionMatrixLoader` reads the `confusion_matrix.json` file produced by Phase 1. The confusion matrix records, for every ground-truth character, the frequency with which Qaari substituted each other character in its place.

The loader supports two code paths. If the Phase 1 output contains a pre-

computed `top_20` sorted list, it is used directly for efficiency. Otherwise, the full confusion dictionary is flattened and sorted by substitution count. The `format_for_prompt()` method selects the top- N confusion pairs (default $N = 10$, configured via `phase3.top_n` in `configs/config.yaml`) and formats them as an Arabic-language bullet list, where each item identifies a source character and its most common Qaari substitute.

A dataset with fewer than 200 total substitutions (threshold configurable as `phase3.min_substitutions`) falls back to a pooled confusion matrix aggregated across all datasets of the same type (e.g. all PATS-A01 training datasets). This prevents noise-dominated confusion matrices from individual small datasets from distorting the injection content.

3.4.2 Word Error Pairs Loader

`WordErrorPairsLoader` reads OCR-to-ground-truth word-level correction pairs generated during Phase 4’s training-split analysis. These pairs are extracted by aligning Phase 2 corrections with ground truth at the word level and identifying positions where the OCR word differs from the ground truth. Two complementary files are produced: `results/phase2-training/analysis/word_pairs_llm_failures.txt` (containing UNFIXED and INTRODUCED error sections) and `results/phase2-training/analysis/` (containing per-sample error categories). Together they provide concrete examples of Qaari’s word-level errors paired with their correct forms, as well as the model’s own failure patterns on the training split.

Word error pairs are *OCR-native*: they directly represent the visual confusion patterns that Qaari produces on the specific fonts in the study, unlike generic human-typing corpora. These pairs are used in Phase 4’s self-reflective prompting to provide the model with concrete correction examples alongside statistical performance insights, and are also available in Phase 6 combined systems.

3.4.3 CAMEL Tools Integration

CAMEL Tools is an open-source Arabic NLP toolkit developed at the Computational Approaches to Modeling Language (CAMEL) Lab, NYU Abu Dhabi [0]. This study uses its morphological analysis component, backed by the Modern Standard Arabic database `calima-msa-r13`.

3.4.3.1 MorphAnalyzer

`src/linguistic/morphology.py::MorphAnalyzer` wraps the CAMEL Tools `MorphologicalAnalyzer` with two extensions:

- **LRU cache.** The morphological analyser is computationally expensive per lookup. Repeated queries for the same token are served from a Least-Recently-Used cache of size 10,000, avoiding redundant analyser calls for frequently occurring words.
- **Graceful degradation.** If `camel_tools` is not installed or the morphology database has not been downloaded, `MorphAnalyzer.analyse()` returns `None` rather than raising an exception. Downstream code that checks for `None` can continue in a degraded mode without CAMEL-based functionality.

3.4.3.2 WordValidator and the Revert Strategy

`src/linguistic/validator.py::WordValidator` implements the Phase 5 post-processing logic. For each word token in the LLM-corrected output, it applies the following decision rule:

If the LLM correction for a word *fails* morphological analysis (zero valid analyses) *and* the corresponding OCR word *passes* morphological analysis (at least one valid analysis), revert to the OCR word.

The strategy is deliberately conservative. It can only revert a word to the OCR original; it cannot introduce a new word that neither the OCR nor the LLM produced. Consequently, the revert strategy has no downside risk on any word it touches, but its upside is limited to cases where the LLM introduces a morphologically invalid Arabic form while the OCR had a morphologically valid one. The validator records summary statistics for each text: total Arabic tokens compared, reverted count, and revert rate.

3.4.4 RAG Index (BM25 Retriever)

`src/data/rag_index.py` provides two classes for Phase 8.

RAGIndexBuilder constructs a BM25 index from Phase 2 training-split corrections. Each index entry corresponds to one training sample and stores the OCR text, the ground-truth text, and extracted word-level OCR→GT correction pairs. Tokenisation uses character n -grams (default $n \in \{2, 3, 4\}$) computed by the `char_ngrams()` utility. Character-level tokenisation is chosen over word-level tokenisation because Arabic OCR errors are primarily character confusions; character n -grams create a retrieval signal that is sensitive to the specific character substitutions present in the query, enabling the index to surface training samples with similar visual confusion patterns.

RAGRetriever wraps the built index and exposes a `retrieve(query_ocr, k=5)` interface. At inference time, the query OCR text is tokenised with the same character n -gram scheme, and the BM25 score is computed against every index entry. The top- k entries by score are returned as a list of (ocr_text, gt_text, word_pairs) tuples. These are formatted by `PromptBuilder.build_rag()` into an Arabic few-shot block for injection into the system prompt.

The index is built from the Phase 2 training-split JSONL file (`results/phase2-training/corre`) during the `-mode build-index` stage of `pipelines/run_phase8.py`. An optional dense retrieval mode using FAISS embeddings is also available (`-retrieval-mode dense`), but all reported Phase 8 experiments use BM25 (`-retrieval-mode bm25`).

3.4.5 LLM Insights Loader

`LLMInsightsLoader` reads JSON insight files produced during Phase 4’s training-split analysis step, stored in `results/phase4/insights/`. Each file records, for each error category, two statistics derived from comparing Phase 2 LLM corrections against ground truth on the training split:

- `fix_rate`: the fraction of OCR errors of that category that the LLM correctly fixed (higher is better).
- `introduction_rate`: the fraction of ground-truth tokens of that category that the LLM *incorrectly* changed (lower is better).

The loader converts these statistics into Arabic-language diagnostic sentences using the following logic: error types with `fix_rate` > 0.6 are reported as strengths; those with `fix_rate` < 0.4 are reported as weaknesses; those with `introduction_rate` > 0.05 are flagged as over-correction risks. Statistics are aggregated at the dataset-type level (PATS-A01 vs. KHATT) to smooth per-sample noise. Up to three weaknesses and two over-correction warnings are injected into the Phase 4 system prompt (configurable thresholds in `phase4.insights`).

3.5 The LLM Correction Module

3.5.1 Class Hierarchy

All LLM backend implementations share a common abstract interface defined in `src/core/llm_corrector.py`. Fig. ?? shows the class hierarchy.

`BaseLLMCorrector` is an abstract base class that defines two abstract methods: `correct()` and `model_name`. Implementations must satisfy the contract that `correct()` never raises an exception and always returns a `CorrectionResult` with a non-empty `corrected_text` field. On any failure, `corrected_text` is set to the

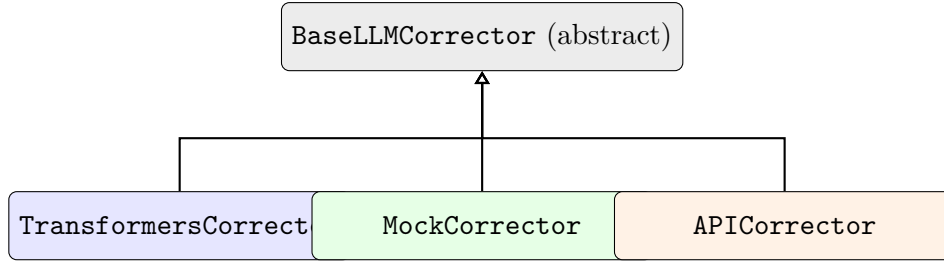


Figure 3.2: LLM corrector class hierarchy. All pipeline code depends only on the `BaseLLMCorrector` interface; the concrete backend is selected at runtime by `get_corrector(config)`.

original `ocr_text` (a safe fallback that does not inflate error rates beyond the OCR baseline).

TransformersCorrector is the production backend. It loads the model and tokenizer via HuggingFace Transformers once at startup and reuses them for all subsequent inference calls. It is designed to run on Kaggle or Colab GPU kernels. Optional 4-bit quantization via `bitsandbytes` is available for GPUs with less than 8 GB of VRAM, but was not used in the experiments reported here (the Kaggle T4 GPU has 16 GB).

MockCorrector returns the OCR text unchanged. It requires no model, no GPU, and no internet access. It is activated by setting `model.backend: "mock"` and is used for local pipeline validation: a full export–infer–analyze cycle completes instantaneously with mock, producing valid JSON outputs with CER equal to the OCR baseline (since no correction is performed).

APICorrector provides an OpenAI-compatible REST interface for potential future use with hosted models. It is not used in the experiments of this thesis.

3.5.2 Generation Configuration

Table ?? documents the exact generation parameters used in all phases of this study. All phases use identical generation settings; only the prompt content changes between phases.

Temperature 0.1 is chosen to make outputs nearly deterministic across runs while

Table 3.4: Generation parameters for `TransformersCorrector`.

Parameter	Value	Rationale
Model ID	Qwen/Qwen3-4B-Instruct-2507	Primary study model
<code>max_new_tokens</code>	1024	Sufficient headroom for all prompt types
<code>do_sample</code>	True	Stochastic sampling
<code>temperature</code>	0.1	Near-deterministic; suppresses high-entropy outputs
<code>enable_thinking</code>	False	Suppresses Qwen3 chain-of-thought scratchpads
<code>MAX_INPUT_TOKENS</code>	4,096	Out-of-memory guard on unexpectedly long prompts

avoiding the degenerate behaviour that pure greedy decoding can produce. At this temperature the output distribution is heavily peaked at the most-probable token at each step, but a small amount of stochasticity is retained to prevent the model from entering repetitive loops when correcting ambiguous text.

The `enable_thinking` flag is Qwen3-specific. When enabled, Qwen3 prepends a chain-of-thought scratchpad enclosed in `<think>...</think>` tokens before generating the final answer. For Arabic post-OCR correction, a one-pass text-to-text transformation task, the scratchpad adds latency and output tokens without improving the correction output. More critically, the scratchpad contents are sometimes included in the decoded output, breaking the response extraction step that expects only the corrected text. Setting `enable_thinking=False` in `apply_chat_template()` prevents this.

3.5.3 Response Extraction

The model is instructed to return *only* the corrected text, with no explanation or commentary. Despite this instruction, decoded outputs occasionally contain preamble or non-Arabic content. The method `_extract_corrected_text()` applies the following steps:

1. Strip leading and trailing whitespace and newlines.
2. If the result is empty, log a warning and return the original OCR text.

3. If the result contains no Arabic characters (Unicode block U+0600–U+06FF), the model has likely replied in English or refused the request; log a warning and return the original OCR text.
4. Otherwise, return the cleaned string as the correction.

Steps 2 and 3 are safety checks that prevent model refusals or empty generations from propagating as corrections. Without them, a refusal would produce a CER of approximately 1.0 for that sample, artificially degrading aggregate performance. By falling back to the OCR text on failure, the method ensures that failed samples do not inflate aggregate CER above the OCR baseline. If `max_retries` additional attempts all fail, the fallback is applied and the `success` flag in the JSONL record is set to `false`.

3.6 PromptBuilder

`src/core/prompt_builder.py::PromptBuilder` is the single point of contact for all prompt construction in the study. Each experimental phase has exactly one corresponding `build_*` method. All methods return a list of dictionaries in OpenAI chat format — a system message followed by a user message — which is directly accepted by the HuggingFace `apply_chat_template()` API.

3.6.1 Prompt Design Philosophy

All prompts in this study are written entirely in Arabic. The decision to use Arabic rather than English for system messages was motivated by two considerations. First, the correction task is inherently Arabic-specific; using the target language avoids any cross-lingual ambiguity in instruction following. Second, Qwen3, as a multilingual model, was trained on Arabic instructions and responds more reliably when the instruction language matches the text language.

All phase prompts share a *conservative instruction*: the system message tells the

model that the input text may already be correct, that it should return the text unchanged if it finds no clear errors, and that it must not change what is already correct. This instruction counteracts a known failure mode of instruction-following LLMs: they tend to interpret any correction directive as a mandate to rewrite, even when the input is already accurate. The conservative constraint is particularly important in the PATS-A01 Akhbar setting, where the OCR baseline CER on normal samples is only 1.37% and the vast majority of tokens require no change. The conservative v2 zero-shot prompt was adopted after pilot runs showed that the original aggressive v1 prompt (“correct this text”) led to unacceptably high error rates on clean samples.

3.6.2 Prompt Variants

Table ?? provides a complete registry of all prompt variants.

Table 3.5: Prompt variant registry. Each phase uses exactly one variant. All prompts are in Arabic. Full prompt text is given in Appendix A. Phase 5 (CAMEL post-processing) applies no new prompt: it uses the Phase 2 zero-shot prompt and post-processes the LLM output with the morphological validator.

Method	Tag	Injected context	Phase
<code>build_zero_shot()</code>	p2v2	None (instruction only)	2, 5
<code>build_ocr_aware()</code>	p3v1	Top-10 Qaari character confusion pairs	3
<code>build_self_reflective()</code>	p4v3	LLM error statistics + word error pairs	4
<code>build_combined()</code>	p6v1	Subset of confusion + self-reflective contexts	6
<code>build_rag()</code>	p8v1	$k=5$ retrieved (OCR, GT) training pairs	8

3.6.2.1 Zero-Shot Prompt (p2v2)

The zero-shot prompt consists of an Arabic system message and the raw OCR text as the user message. The system message establishes the model as an Arabic text proofreader and explicitly states the conservative instruction: the OCR output

may be fully correct; the model should return it exactly if no clear error is found; it must not change what is already correct; it should return only the text with no explanation. There is no injected external knowledge; the model’s only guidance comes from its own training and from the instruction framing.

3.6.2.2 OCR-Aware Prompt (p3v1)

The OCR-aware prompt extends the zero-shot system message with an injected Arabic bullet list of confusion pairs from `ConfusionMatrixLoader`. Each bullet item names a ground-truth character and the character Qaari commonly substitutes for it. The system message instructs the model to pay particular attention to these character pairs when correcting. If no confusion context is available, `build_ocr_aware()` falls back transparently to the zero-shot prompt.

3.6.2.3 Self-Reflective Prompt (p4v3)

The self-reflective prompt injects Arabic diagnostic sentences derived from the model’s own training-split error statistics. The system message frames the diagnostics as performance feedback: the model is informed which error categories it handles well, which it tends to miss, and for which categories it tends to over-correct previously correct tokens. This is a lightweight, pre-computed form of meta-cognitive guidance that avoids the latency cost of asking the model to self-critique at inference time.

3.6.2.4 Combined Prompt (p6v1)

`build_combined()` accepts any subset of two context types (confusion matrix and self-reflective insights) and assembles them into a single system message using a fixed section ordering numbered with Arabic ordinals (“First: confusions, Second: performance notes”). Only non-empty context strings are included; the closing conservative instruction is always appended. If all context strings are empty, the method falls back to the zero-shot prompt.

Phase 6 evaluates four pre-selected combination identifiers: `conf_only` (confusion matrix only), `self_only` (self-reflective insights only), `conf_self` (both combined), and `best_camel` (the best prompt combination with CAMEL morphological post-processing applied on top). This design makes Phase 6 ablation experiments straightforward to specify and fully reproducible from the combination identifier alone.

3.6.2.5 RAG Prompt (p8v1)

`build_rag()` accepts the OCR text and a list of retrieved (OCR, ground-truth) training pairs. The system message extends the zero-shot v2 base instruction with a `<similar_corrections>` XML block containing up to $k = 5$ retrieved examples. Each example shows the training OCR text, the corrected ground-truth text, and the specific word-level changes made. The model is instructed to use these examples as reference for the kinds of corrections appropriate to this input, while still applying the conservative correction constraint. If no similar examples are retrieved (empty index or below-threshold score), `build_rag()` falls back transparently to the zero-shot prompt.

Chapter 4

Experimental Methodology

This chapter describes the experimental methodology of each phase. For every phase, the structure follows a consistent template: hypothesis, method (step-by-step), variables, output files, and the research question answered. Section ?? explains the overarching design principles that govern how the phases relate to each other. Sections ?? through ?? describe each phase in full. Table ?? at the end of this chapter provides a consolidated reference.

4.1 Experimental Design Principles

4.1.1 The Isolation Principle

The defining design choice of this study is that Phases 3, 4, and 5 are *isolated* experiments. Each phase changes exactly one variable relative to Phase 2: the type of knowledge injected into the system prompt or applied as a post-processing step. Every other factor — the model, the dataset, the sample set, the generation parameters, the evaluation protocol, the analysis code — is held fixed. This design allows any observed CER change to be attributed unambiguously to the knowledge injection and not to a confound.

Comparing isolated phases against Phase 2 (zero-shot) rather than against Phase 1 (OCR baseline) is a deliberate choice. The question of interest is not whether the LLM improves on raw OCR (Phase 2 answers that), but whether injecting a specific

type of knowledge into an already-running LLM corrector improves its performance. Comparing to Phase 1 would confound OCR quality with LLM correction quality.

4.1.2 Phase 2 as the Hub Baseline

Phase 2 (zero-shot LLM correction) serves as the comparison baseline for all subsequent phases. A phase that does not improve on Phase 2 is a negative result with respect to that knowledge type: the injected knowledge does not help, and may actively hurt, relative to the model’s own language understanding. Negative results are scientifically valuable because they rule out a hypothesis and constrain theories of what makes LLM correction succeed. All five strategies are therefore compared to the same zero-shot baseline, enabling a consistent ranking.

4.1.3 Reproducibility Measures

All experiments use:

- The same nine evaluation datasets: eight PATS-A01 fonts (415 validation samples each, aligned split) plus KHATT validation.
- The same model: `Qwen/Qwen3-4B-Instruct-2507`.
- The same generation parameters: temperature 0.1, `max_new_tokens` 1024, `enable_thinking=False`.
- The same evaluation protocol: diacritic-stripped CER/WER, per-sample then aggregated. PATS-A01 macro-averages (non-weighted, each font contributes equally) are reported alongside per-font results.
- Fixed random seeds where any sampling is involved (seed 42 throughout).

Each phase stores its inference input as a JSONL file with embedded prompts, ensuring that the exact prompts sent to the model are permanently recorded and can be inspected or re-run without requiring access to the original data files.

4.1.4 Why Not Test All Combinations First

With three distinct prompt-level knowledge sources (confusion matrix, self-reflective insights, and their combination), plus CAMEL post-processing as an optional layer, the combination space is $2^3 = 8$ prompt subsets plus a post-processing overlay. Running all 8 combinations across nine datasets as the primary experimental strategy would require approximately 33,000 model inference calls — well beyond the available compute budget for a preliminary sweep — and, more importantly, would make results difficult to interpret: a combination that degrades performance could not be decomposed into its contributing factors without the isolated phase results.

The hierarchical strategy (isolated first, then selected combinations in Phase 6) extracts the maximum insight per inference call. Isolated phases diagnose each component independently. Phase 6 then tests four pre-selected combinations chosen based on the isolated results: the best-performing isolated strategy, the second-best, their combination, and the combination augmented with CAMEL post-processing. This focused design answers the combination question with a fraction of the inference cost of a full grid search.

4.2 Phase 1 — Baseline and Error Taxonomy

4.2.1 Hypothesis

Qaari produces systematic, categorisable errors on Arabic typewritten text. The error profile is dominated by a small number of recurring character confusion types that can be quantified and used to guide subsequent phases. The nature and frequency of errors differs substantially across fonts, making multi-font evaluation necessary to capture the full range of correction challenges.

4.2.2 Method

1. Load all OCR predictions and ground-truth pairs for each of the nine evaluation datasets from the DataLoader. Both training and validation splits are processed; only validation-split metrics are reported in Chapter ??, but training-split confusion matrices and error statistics inform Phases 3 and 4.
2. Compute per-sample CER and WER using the normalised metric (diacritics stripped via Unicode category filtering). Aggregate statistics per dataset: mean, standard deviation, median, 95th percentile (P95). The P95 metric identifies the severity of the worst-performing tail of samples.
3. Build a character-level confusion matrix. For each aligned (ground-truth character, OCR character) pair in the minimum-edit alignment, increment the count for that pair. Normalise by ground-truth character frequency to obtain conditional confusion probabilities. The confusion matrix is stored as a square matrix indexed by Unicode character code point and is used directly by Phase 3's `ConfusionMatrixLoader`.
4. Classify each substitution, insertion, and deletion into one of eight taxonomic categories using the heuristic rules defined in Chapter ??:
 - (1) Dot confusion (ba/ta/tha/ya family; fa/qaf; nun/waw variants)
 - (2) Hamza and alef variants (hamza placement, alef/alef-madda/alef-wasla)
 - (3) Taa marbuta / ha confusion
 - (4) Alef maqsura / ya confusion
 - (5) Similar-shape pairs (ra/zay, dal/dhal, sad/dad, etc.)
 - (6) Word merge (two words recognised as one)
 - (7) Word split (one word recognised as two)
 - (8) Other substitution, insertion, or deletion
5. Apply CAMEL Tools morphological analysis to each unique OCR token. Classify tokens as *non-words* (zero valid analyses returned by the MSA morphological analyser) or *valid-but-wrong* (at least one valid analysis, but contextually

incorrect relative to ground truth). This distinction is critical for Phase 5: the CAMEL post-processing revert strategy can only intervene on non-word errors.

6. Cross-reference the confusion matrix with Phase 2 training-split failures: retain only confusion pairs that appear in samples where the zero-shot LLM failed to correct the error. This filtered confusion list is used by Phase 3 and represents actionable patterns — ones the model is known not to handle by default.
7. Write per-dataset reports including confusion matrix JSON, error taxonomy JSON, morphological analysis JSON, and a human-readable Markdown summary.

4.2.3 Output Files

- `results/phase1/{dataset}/metrics.json` — CER/WER statistics (mean, std, median, p95) with and without diacritics
- `results/phase1/{dataset}/confusion_matrix.json` — character-level confusion counts with top-20 list and conditional confusion probabilities
- `results/phase1/{dataset}/error_taxonomy.json` — error counts and percentages by eight-category taxonomy with representative examples
- `results/phase1/{dataset}/morphological_analysis.json` — word validity statistics (non-word / valid-but-wrong split)
- `results/phase1/{dataset}/error_examples.json` — up to 5 sample OCR/GT pairs per error category
- `results/phase1/{dataset}/report.md` — human-readable summary

Research question answered: RQ1.

4.3 Phase 2 — Zero-Shot LLM Correction

4.3.1 Hypothesis

A zero-shot instruction-tuned LLM can improve Arabic OCR output without task-specific guidance, by exploiting its pre-trained Arabic language knowledge to recognise and correct implausible character sequences. However, the effectiveness of this correction is expected to be font-dependent: fonts with higher baseline CER present more recognisable errors, while fonts with lower CER present fewer errors per sample and risk over-correction.

4.3.2 Method

1. For each sample in each of the nine evaluation datasets, construct the conservative zero-shot prompt using `PromptBuilder.build_zero_shot(ocr_text, version="v2")`. The system message informs the model that the text is OCR output that may already be correct, instructs it to return the text unchanged if no clear error is found, and prohibits it from changing what is already correct.
2. Write all prompts to `results/phase2/inference_input.jsonl`, one JSON object per sample, containing fields: `sample_id`, `dataset`, `ocr_text`, `gt_text`, `prompt`, `prompt_version` (= p2v2).
3. Run inference on Kaggle (GPU) using `scripts/infer.py`. For each sample, the model generates the corrected text. Results are written to `results/phase2/corrections.jsonl`. The inference script implements a resume mechanism: any `sample_id` already present in the output file is skipped, enabling restartable inference without duplicating work.
4. Compute post-correction CER and WER for each sample. Compare to Phase 1 OCR baseline. Compute error changes per sample: fixed errors (present in OCR, absent from correction), introduced errors (absent from OCR, present in correction), and their ratio (introduction rate).

5. Run the same Phase 2 pipeline on the *training* split of each PATS-A01 font. These training-split corrections are not reported as results but serve as the input to Phase 4’s self-reflective analysis.
6. Write comparison reports to `results/phase2/{dataset}/comparison_vs_phase1.json`.

4.3.3 Prompt Design Rationale

Two prompt versions were developed and piloted:

- **v1 (aggressive)**: instructs the model to correct OCR errors without qualification. Piloting on Akhbar (low baseline CER) showed that v1 increased CER relative to baseline: the model rewrote correctly recognised text. This aggressive framing was discarded.
- **v2 (conservative)**: explicitly tells the model that the text *may already be correct*, that it should return it unchanged if no clear error is visible, and that it must not change what is correct. This formulation keeps the model in a low-intervention mode on clean text while still enabling correction where errors are obvious.

Version v2 is used in all phases reported in this thesis. The trade-off is that v2 may miss subtle errors that v1 would catch, but the reduction in over-correction is empirically justified.

4.3.4 Output Files

- `results/phase2/{dataset}/metrics.json`
- `results/phase2/{dataset}/comparison_vs_phase1.json`
- `results/phase2/{dataset}/error_changes.json`

Research question answered: RQ2. **Role in the study:** Phase 2 output serves as the comparison baseline for all subsequent phases and as the input data source for Phase 4’s training-split analysis.

4.4 Phase 3 — OCR-Aware Prompting

4.4.1 Hypothesis

Telling the LLM which specific character confusions Qaari makes on a given font — filtered to confusions that appear in samples the model itself failed to correct at the zero-shot level — will direct its attention to the positions most likely to contain unresolved errors and improve correction quality for those specific confusion types.

4.4.2 Method

1. Load the Phase 1 training-split confusion matrix from `results/phase1/{dataset}-train/confusion` using `ConfusionMatrixLoader`. Each font uses its own font-specific confusion matrix derived from its training split, ensuring the confusion context is tailored to the actual error distribution of that typeface.
2. Cross-reference the confusion matrix entries with the LLM failure cases from Phase 2 training-split analysis. A confusion pair (`gt_char`, `ocr_char`) is retained in the injection list only if it appears in samples where the zero-shot model failed to produce a correction closer to ground truth than the OCR input. This *failure-filtered* list focuses the model’s attention on confusions it does not resolve by default.
3. Extract the top-10 confusion pairs by substitution count from the filtered list.
4. Format the confusion pairs as an Arabic-language bullet list using `ConfusionMatrixLoader.format`. Each bullet item reads (translated): “This system confuses [correct character] with [OCR character] in output.”

5. Build the OCR-aware prompt using `PromptBuilder.build_ocr_aware(ocr_text, confusion_context)`. The system message extends the zero-shot v2 message with the confusion list in a `<confusion_patterns>` XML block, instructing the model to pay particular attention to these pairs while observing the same conservative correction rules as Phase 2.
6. Export, infer (Kaggle GPU), and analyse following the same three-stage pipeline as Phase 2. The prompt version tag is `p3v1`.

4.4.3 Variables

- **Changed from Phase 2:** Failure-filtered confusion list injected into system prompt.
- **Fixed:** Model, dataset, generation parameters, evaluation protocol, conservative correction instruction.
- **Number of confusion pairs injected:** 10 (configured as `phase3.top_n` in `configs/config.yaml`).

4.4.4 Design Rationale

The failure-filtering step is the key enhancement in Phase 3. An unfiltered confusion matrix lists all character confusions observed in the training data, including confusions that the zero-shot model already handles correctly. Such confusions carry no additional information for the model. By retaining only confusions from samples where the model failed, the injection is narrowed to genuine blind spots. This makes the injected list shorter and more targeted than a naive top-10 from the raw confusion matrix.

The font-specificity of the confusion matrix is equally important. A single confusion list shared across all fonts would reflect the aggregate Qaari error profile, which may not match the dominant errors on any individual font. Using per-font

confusion matrices ensures that each font receives a confusion context that matches its specific OCR error distribution.

4.4.5 Output Files

- `results/phase3/{dataset}/metrics.json`
- `results/phase3/{dataset}/comparison_vs_phase2.json`
- `results/phase3/{dataset}/confusion_impact.json` — per-confusion-pair analysis of correction improvement on samples containing that confusion type

Research question answered: RQ3.

4.5 Phase 4 — Self-Reflective Prompting

4.5.1 Hypothesis

Feeding the model statistical summaries of its own systematic errors — derived from its zero-shot corrections on the training split where ground truth is available — will cause it to apply targeted corrections more carefully and reduce its error introduction rate on the held-out validation split. Additionally, providing concrete word-level OCR error pairs from the training data gives the model specific exemplars of the correction patterns it should apply, beyond what aggregate statistics alone can convey.

4.5.2 Three-Stage Method

Phase 4 is itself a three-stage process that must be executed in order. The first stage uses training-split data; the second and third stages use validation-split data.

4.5.2.1 Stage 1: Training-Split Analysis

1. Load Phase 2 corrections on the *training* split for each PATS-A01 font (e.g., `results/phase2-training/{font}-train/corrections.jsonl`). These corrections exist because Phase 2 is run on both splits; the training-split corrections are not used as results but solely as analysis data.
2. For each sample, compare the Phase 2 LLM corrections to ground truth using `LLMErrorAnalyzer.analyse_sample()`. For every aligned character pair between LLM output and ground truth:
 - If the OCR had an error at this position and the LLM fixed it: increment `fixed` for that error type.
 - If the OCR was correct at this position and the LLM changed it incorrectly: increment `introduced` for that error type.
3. Aggregate across all training samples to obtain per-error-type statistics:
 - `fix_rate` = `fixed` / baseline errors of that type (fraction of existing errors the LLM corrected).
 - `introduction_rate` = `introduced` / ground-truth tokens of that type (fraction of correct tokens incorrectly changed).
4. Extract word-level failure pairs: (`ocr_word`, `gt_word`) pairs from samples that the LLM either failed to fix or made worse. These are stored in `word_pairs_llm_failures.txt` with separate sections for UNFIXED errors (LLM left OCR errors unchanged) and INTRODUCED errors (LLM introduced new errors not in OCR).
5. Classify training samples by correction outcome into: correctly improved, made worse, and unchanged. Store classification in `sample_classification.json`.
6. Aggregate insights separately for PATS-A01 and KHATT datasets, reflecting their substantially different error profiles.

4.5.2.2 Insights Derived from Training Data

Table ?? reports representative statistics extracted from the PATS-A01 training split (aggregated across fonts).

Table 4.1: Phase 4 training-split analysis results (PATS-A01 aggregate). Fix rate = fraction of baseline errors of that type corrected by the LLM. Introduction rate = fraction of correct ground-truth tokens of that type incorrectly changed. † = fewer than 10 baseline samples (estimate unreliable).

Error type	Baseline	Fixed	Introduced	Fix rate	Intro. rate
Insertion	1,964	1,912	356	97.35%	18.13%
Other subst.	49	8	80	16.33%	163.27%
Deletion	32	1	68	3.12%	212.50%
Dot confusion	14	1	2	7.14%	14.29%
Alef maqsura	3 [†]	1	3	33.33%	100.00%
Similar shape	1 [†]	1	8	100.00%	800.00%
Hamza	0	—	—	—	—
Taa marbuta	0	—	—	—	—
Overall	2,063	1,924	517	93.26%	25.06%

The table reveals two important patterns. First, the insertion fix rate of 97.35% shows that the LLM is highly effective at collapsing the Qaari repetition artefact: almost all spurious character insertions (which constitute the majority of all baseline errors) are removed. Second, the overall introduction rate of 25.06% — approximately one in four previously correct tokens is incorrectly changed by the LLM — represents a systematic over-correction bias that affects all error categories. The deletion and other-substitution introduction rates exceed 100% because the LLM introduces more of those error types than existed in the original OCR output.

These patterns motivate the self-reflective prompt: by explicitly informing the model of its high introduction rate and its weakness on deletion and substitution errors, the prompt aims to shift the model towards a more conservative correction mode.

4.5.2.3 Stage 2: Export

1. Load the aggregated training-split artifacts: `word_pairs_llm_failures.txt` and `sample_classification.json` from `results/phase2-training/analysis/`.
2. Apply the insight filtering logic to derive the self-analysis context:
 - Error types with fix rate > 0.60 : reported as strengths (“The system is good at fixing this type of error”).
 - Error types with fix rate < 0.40 and baseline ≥ 10 : reported as weaknesses (“The system often fails to fix this type of error”).
 - Error types with introduction rate > 0.05 and baseline ≥ 10 : reported as over-correction risks (“The system tends to incorrectly change correct tokens of this type”).
3. Convert the filtered statistics to Arabic-language diagnostic sentences that describe the model’s behaviour patterns and provide concrete guidance for what to watch out for. The full diagnostic text is provided in Appendix B.
4. Select up to 15 word-level error pairs per sample from `word_pairs_llm_failures.txt` to include as concrete exemplars in the prompt, showing specific (`ocr_word`, `gt_word`) pairs the model has struggled with.
5. Build the self-reflective prompt for each validation-split sample using `PromptBuilder.build_self_insights_context()`. The insights are injected in a `<self_analysis>` XML block following the conservative correction instruction from Phase 2.
6. Write to `results/phase4/inference_input.jsonl`. The prompt version tag is `p4v1`.

4.5.2.4 Stage 3: Analyse

1. Run inference on Kaggle GPU using `scripts/infer.py`.
2. Load `results/phase4/corrections.jsonl`.

3. Compute CER/WER per dataset; compare to Phase 2 zero-shot baseline.
4. Write reports to `results/phase4/{dataset}/`.

4.5.3 Variables

- **Changed from Phase 2:** Arabic-language diagnostic text injected in `<self_analysis>` block; concrete word-level error pairs from training data appended.
- **Fixed:** Model, dataset, generation parameters, evaluation protocol, conservative correction instruction.
- **Data used:** Phase 2 training-split corrections (not validation-split; no data leakage).

4.5.4 No-Leakage Guarantee

A potential concern with self-reflective prompting is data leakage: if the training and validation splits are not properly separated, the model might receive information about the ground truth of its validation-split samples via the error statistics. This concern is addressed by the aligned split design: the 85% training split and 15% validation split share no samples. The training-split analysis is performed exclusively on training-split corrections, and the resulting insights are qualitative statistical summaries (e.g., “the model has a 25% introduction rate”) rather than sample-level ground-truth information. The word-level error pairs are drawn from training samples only. There is no mechanism by which Phase 4 could convey ground-truth information about validation samples to the model.

4.5.5 Output Files

- `results/phase2-training/analysis/word_pairs_llm_failures.txt` — UNFIXED and INTRODUCED sections

- `results/phase2-training/analysis/sample_classification.json`
- `results/phase4/insights/PATS-A01_insights.json` — aggregated training-split statistics
- `results/phase4/{dataset}/metrics.json`
- `results/phase4/{dataset}/comparison_vs_phase2.json`
- `results/phase4/{dataset}/error_changes.json`

Research question answered: RQ4.

4.6 Phase 5 — CAMEL Morphological Post-Processing

4.6.1 Hypothesis

Morphological post-processing can detect two classes of LLM output failures and reverse them. First, cases where the LLM introduces morphologically invalid Arabic words from valid OCR sources can be reverted to the OCR original. Second, documented overcorrection patterns identified in the training split can be reverted regardless of morphological status, targeting the model’s systematic tendency to change specific correct word forms. Together, these two revert conditions reduce the error introduction rate without substantially affecting correctly corrected words.

4.6.2 Method

Phase 5 does not generate new prompts or run a new inference call from scratch. It applies the revert strategy as a post-processing step to the Phase 2 corrections.

1. Load Phase 2 corrections from `results/phase2/{dataset}/corrections.jsonl`.
2. Load the known-overcorrection list derived from training-split analysis. This list contains (ocr_word, llm_word) pairs where the LLM changed a correct

OCR word to an incorrect form, identified by comparing Phase 2 training-split corrections to ground truth.

3. Initialise `MorphAnalyzer` with the MSA database `calima-msa-r13`. CAMEL Tools requires separate installation: `pip install camel-tools` followed by `camel_data -i morphology-db-msa-r13`.
4. For each sample, apply `WordValidator.validate_correction(ocr_text, llm_text, known_overcorrections)`:
 - (1) Tokenise both the OCR text and the LLM correction into Arabic word tokens, preserving position indices.
 - (2) For each word pair at position i where (ocr_word_i, llm_word_i) differs:
 - **Morphological check:** Run `MorphAnalyzer.analyse(llm_wordi)`; if zero analyses: LLM word is invalid. Run `MorphAnalyzer.analyse(ocr_wordi)`; if ≥ 1 analysis: OCR word is valid. If both hold: revert to `ocr_wordi`.
 - **Known-overcorrection check:** If (ocr_word_i, llm_word_i) appears in the overcorrection list: revert to `ocr_wordi` regardless of morphological status.
 - **Otherwise:** Keep the LLM correction.
 - (3) Record per-sample statistics: words reverted by morphological check, words reverted by overcorrection check, words kept.
5. Compute post-validation CER/WER on the final text. Compare to both Phase 2 (LLM only) and Phase 1 (OCR baseline).
6. Write validation statistics to `results/phase5/{dataset}/validation_stats.json`.

4.6.3 Implementation Note: Position-Indexed Revert

An important implementation detail is that the revert decision is indexed by *word position*, not word identity. An early implementation keyed reverts by word string, which caused incorrect reversions when the same word form appeared multiple times in a sentence and only some occurrences should be reverted. The position-indexed

implementation ensures that each word is evaluated independently at its specific position, preventing this cross-position contamination.

4.6.4 Revert Strategy Rationale

The morphological revert strategy is deliberately conservative. It can only revert a word to the OCR original; it cannot introduce new words or produce word forms not present in either the OCR output or the LLM correction. The maximum harm it can do is revert a correctly-improved LLM word back to a wrong OCR word — but only if the LLM word happened to be morphologically invalid despite being the correct intended word, which is rare for genuine corrections of real Arabic. The maximum benefit is preventing a hallucinated, morphologically invalid LLM word from degrading the text.

The known-overcorrection check adds a second layer of protection targeting the model’s documented systematic biases. Because overcorrection patterns are mined from training data where ground truth is available, they represent genuine cases where the model has been shown to make specific incorrect changes. Reverting these patterns on the validation set is a low-risk intervention, though it covers only a subset of over-corrections (those that are *also* frequent enough to appear in the training analysis).

4.6.5 Expected Limitations

The primary limitation of Phase 5 is that the dominant OCR error type — valid-but-wrong substitutions (71% of all errors per the Phase 1 analysis) — is invisible to a context-free morphological check. Both the original OCR word and its correct counterpart are morphologically valid, so the check cannot distinguish them. This means Phase 5’s revert mechanism operates on the minority (29%) of errors that produce non-words, and the overcorrection list covers a further small fraction. The expectation is therefore that Phase 5 will have a modest impact, and may even slightly degrade performance if reverts accidentally undo beneficial corrections.

4.6.6 Output Files

- `results/phase5/{dataset}/metrics.json`
- `results/phase5/{dataset}/comparison_vs_phase2.json`
- `results/phase5/{dataset}/validation_stats.json` — per-dataset revert counts, rates, and list of reverted word pairs

Research question answered: RQ5.

4.7 Phase 6 — Combinations and Ablation

4.7.1 Purpose

Phases 3–5 establish which knowledge types help and which hurt in isolation. Phase 6 asks two follow-up questions: (a) can two or more knowledge types combine synergistically to produce a result better than either individually? (b) in the best combined system, what is each component’s quantitative contribution when removed?

4.7.2 Combinations Tested

Four combinations are defined and tested:

1. **conf_only**: Phase 3 confusion-matrix injection alone, applied to the validation split. This is equivalent to Phase 3 and serves as a reference point within the Phase 6 framework.
2. **self_only**: Phase 4 self-reflective prompting alone, applied to the validation split. This is equivalent to Phase 4 and serves as the second reference point.
3. **conf_self**: Phase 3 and Phase 4 combined. The system prompt includes both the failure-filtered confusion list (in `<confusion_patterns>`) and the

self-reflective diagnostic text with word-level exemplars (in `<self_analysis>`). This tests whether the two knowledge types are complementary or redundant.

4. **best_camel**: The `conf_self` prompt combination with Phase 5 CAMEL post-processing applied on top of the LLM output. This tests whether the morphological revert layer adds value on top of the best prompt-only combination.

The combinations are chosen based on the isolated phase results: Phase 3 and Phase 4 are the two strongest isolated strategies, and `conf_self` tests their combination directly. Phase 5 is then layered on top as an orthogonal post-processing step, since it operates on the output rather than the prompt.

4.7.3 Ablation Design

The ablation study measures the marginal contribution of each component by removing it from the `conf_self` combination and measuring the CER impact. Two ablations are performed:

$$\Delta_{\text{CER}}^{(\text{conf})} = \text{CER}_{\text{self_only}} - \text{CER}_{\text{conf_self}} \quad (4.1)$$

$$\Delta_{\text{CER}}^{(\text{self})} = \text{CER}_{\text{conf_only}} - \text{CER}_{\text{conf_self}} \quad (4.2)$$

A positive $\Delta_{\text{CER}}^{(k)}$ means removing component k increases CER — the component was contributing positively. A negative value means removing the component decreased CER — the component was hurting when combined.

4.7.4 Statistical Testing

For Phase 6 results, paired t-tests are used to assess whether observed CER differences between the best combination and Phase 2 are statistically signifi-

cant at the sample level. The Bonferroni correction is applied to control the family-wise error rate across the four combination comparisons at $\alpha = 0.05$. Bootstrap confidence intervals (1,000 iterations, seed 42) are computed for the best-performing combination. Cohen’s d effect size is reported alongside p -values to characterise the practical magnitude of improvements. These tests are implemented in `src/analysis/stats_tester.py::StatsTester`.

4.7.5 Output Files

- `results/phase6/{combo_id}/{dataset}/metrics.json`
- `results/phase6/{combo_id}/{dataset}/comparison_vs_phase2.json`
- `results/phase6/summary.json` — all combinations ranked by PATS-A01 macro-average CER
- `results/phase6/ablation.json` — ablation delta table
- `results/phase6/paper_tables.md` — formatted tables

Research question answered: RQ6.

4.8 Phase 8 — Retrieval-Augmented Generation

4.8.1 Hypothesis

Retrieving the most similar OCR correction pairs from the training split — using BM25 character n -gram matching — and injecting them as per-sample few-shot context will improve correction quality beyond zero-shot by providing the model with concrete, domain-matched correction exemplars. Unlike the global statistics of Phases 3 and 4, retrieved examples adapt to the specific error pattern of each input sample, potentially providing more precise guidance. The use of a same-domain

corpus (training corrections from the same OCR engine and text type) avoids the domain-mismatch degradation observed with external corpus retrieval.

4.8.2 Method

Phase 8 is itself divided into three pipeline stages:

4.8.2.1 Stage 1: Build Index

1. Load Phase 2 training-split corrections from `results/phase2-training/corrections.json`. Each record provides an (OCR text, ground-truth text, LLM correction) triple for a training sample.
2. Extract word-level OCR→GT pairs for each training sample using `extract_word_errors()`, which aligns OCR and ground-truth tokens and records positions where they differ.
3. Build a BM25 index (`RAGIndexBuilder`) where each document corresponds to one training sample. The indexing field is the OCR text, tokenised into character n -grams ($n \in \{2, 3, 4\}$) by `char_ngrams()`.
4. Persist the index and the associated (OCR, GT, word-pairs) metadata to `results/phase8/index/`.

4.8.2.2 Stage 2: Export

1. For each validation sample, query the BM25 index with the sample's OCR text (also tokenised as character n -grams).
2. Retrieve the top- k training entries by BM25 score ($k = 5$, configured as `phase8.top_k`).
3. Format the retrieved entries as an Arabic few-shot block using `PromptBuilder.build_rag(ocr, retrieved_pairs)`. The block is embedded in a `<similar_corrections>`

XML section that shows the model: (a) the training OCR text, (b) the corrected ground-truth text, and (c) the specific word-level changes made. The block is prepended to the zero-shot v2 conservative instruction.

4. Write the prompts to `results/phase8/inference_input.jsonl`. The prompt version tag is `p8v1`.

4.8.2.3 Stage 3: Analyse

1. Run inference on Kaggle GPU using `scripts/infer.py`.
2. Load `results/phase8/corrections.jsonl`.
3. Compute CER/WER per dataset; compare to Phase 2 zero-shot baseline.
4. Write reports to `results/phase8/{dataset}/`.

4.8.3 Variables

- **Changed from Phase 2:** Per-sample few-shot RAG context injected in `<similar_corrections>` block; retrieved from Phase 2 training corrections via BM25 character n -gram index.
- **Fixed:** Model, dataset, generation parameters, evaluation protocol, conservative correction instruction.
- **Data used:** Phase 2 training-split corrections only (no validation-split leakage; retrieval pool contains no validation-split ground truth).
- **Retrieval mode:** BM25 (`-retrieval-mode bm25`). Dense retrieval mode (FAISS + embedding) is available as an alternative but is not used in reported experiments.

4.8.4 Design Rationale

The key advantage of Phase 8 relative to other isolated strategies is per-sample adaptivity. Phases 3 and 4 inject global knowledge about the engine’s confusion patterns or the model’s aggregate failure statistics. These are averaged over the training set and applied identically to every validation sample. Phase 8 instead retrieves different context for each input: a sample with heavy dot-confusion errors will retrieve training examples dominated by dot-confusion pairs; a sample with segmentation errors will retrieve training examples with similar segmentation patterns.

Character n -gram retrieval is chosen over word-level BM25 for two reasons. First, Arabic OCR errors often corrupt individual characters within words, making character-level similarity a more direct signal than word overlap. Second, character n -grams are robust to word-boundary errors (merges and splits), which would cause word-level matching to fail at boundaries created by segmentation errors.

The same-domain retrieval corpus (Phase 2 training corrections from Qaari on PATS-A01/KHATT) is the critical design choice that distinguishes Phase 8 from the OpenITI-based RAG considered in the background literature review. The domain match ensures that retrieved examples exhibit the same visual confusion patterns as the query, making them genuinely useful as correction exemplars rather than distracting context.

4.8.5 No-Leakage Guarantee

The retrieval index is built exclusively from training-split samples. The validation-split samples are never present in the index, ensuring no ground-truth leakage. Because the retrieved context contains only training OCR texts and ground-truth texts (no validation-split information), there is no mechanism by which Phase 8 could convey held-out information to the model.

4.8.6 Output Files

- `results/phase8/index/` — BM25 index and metadata
- `results/phase8/inference_input.jsonl`
- `results/phase8/corrections.jsonl`
- `results/phase8/{dataset}/metrics.json`
- `results/phase8/{dataset}/comparison_vs_phase2.json`
- `results/phase8/comparison.json` — cross-dataset summary
- `results/phase8/report.md` — human-readable summary

Research question answered: RQ8.

4.9 Phase Summary

Table ?? provides a consolidated reference for all phases.

Table 4.2: Summary of all experimental phases. “Comparison” is the phase each experiment is evaluated against. All phases are evaluated across nine datasets (eight PATS-A01 fonts + KHATT validation split).

Phase	Knowledge / Intervention	Comparison	RQ
Phase 1	None (OCR analysis + taxonomy)	N/A	RQ1
Phase 2	None (zero-shot conservative prompt)	Phase 1	RQ2
Phase 3	Failure-filtered confusion matrix	Phase 2	RQ3
Phase 4	Self-reflective insights + word pairs	Phase 2	RQ4
Phase 5	CAMeL morphological revert + over-correction revert	Phase 2	RQ5
Phase 6	Combinations + ablation (4 configs)	Phase 2 + best combo	RQ6
Phase 8	BM25 RAG: $k=5$ retrieved training corrections	Phase 2	RQ8

4.10 Experiment 2 — Prompt Design Ablation on Full-Page Datasets

4.10.1 Motivation and Overview

Experiment 1 (Phases 1–8) establishes which prompt strategies are effective for Arabic post-OCR correction on line-level typewritten and handwritten text. A natural follow-up question is whether these findings generalise to full-page documents from previously unseen domains: encyclopedic printed text (Yarmouk), multi-line handwritten paragraphs (KHATT-Paragraph), historical handwritten letters (Muharaf), and manuscript pages (Historical Arabic). These four new datasets differ from PATS-A01 and KHATT in several ways that may affect correction behaviour:

- **Full-page scope:** Yarmouk and Historical pages are entire document images, not isolated text lines. The OCR output per sample is much longer (hundreds to thousands of characters), increasing context but also increasing the risk of hallucination or truncation.
- **Domain shift:** Encyclopedic and archival content introduces vocabulary, named entities, and orthographic conventions that may be under-represented in the model’s language prior.
- **No training-split OCR:** Unlike PATS-A01 and KHATT, no training-split OCR was produced for the four new datasets due to compute constraints. Phase 4 self-reflective insights (which require training-split corrections) therefore cannot be computed in their standard form for these datasets.

The two primary design levers available without training-split data are (a) the prompt instruction framing and (b) the example set provided to the model. Experiment 2 explores these levers systematically.

This experiment addresses RQ9–RQ11 (Section 1.5).

4.10.2 Factorial Design

Experiment 2 uses a 222 fully crossed factorial design, yielding eight trials. The three binary factors are:

Factor A — Prompt style: **Base** (aggressive): instructs the model to correct all visible OCR errors (equivalent to the v1 framing from Experiment 1 pilot). **Conservative** (v2): tells the model the text may already be correct and to return it unchanged if no clear error is present (the framing used throughout Experiment 1).

Factor B — Example set: **Generic**: a set of automatically generated example corrections representing common Arabic OCR error types (dot confusion, hamza, taa marbuta, etc.) without domain-specific content. **Hand-picked**: a curated set of manually constructed synthetic OCR errors derived from the ground-truth text of the four new datasets. Each hand-picked example uses a realistic Qaari-style error pattern — including repetition artefacts, character confusions, and word splits — applied to actual sentences from KHATT-Paragraph, Yarmouk, Muharaf, and Historical ground truth. The hand-picked set covers all four new domains with at least one example per domain.

Factor C — Retrieval strategy: **Zero-shot**: no retrieved context; the model receives only the instruction and examples. **BM25-RAG**: the Phase 8 index (built from Experiment 1 PATS-A01 and KHATT training corrections, totalling 18,363 entries) is queried with each validation sample’s OCR text using character n -gram BM25, and the top- $k = 5$ retrieved (OCR, GT) pairs are injected as additional context. This is a cross-domain retrieval setting: the index was built from typewritten and line-level handwritten text but is queried with full-page samples from new domains.

The eight resulting trial identifiers and their factor settings are listed in Table ??.

Table 4.3: Experiment 2 trial definitions. Each trial is a unique combination of prompt style (A), example set (B), and retrieval strategy (C).

Trial	Prompt (A)	Examples (B)	Retrieval (C)
T1	Base	Generic	Zero-shot
T2	Base	Generic	BM25-RAG
T3	Conservative	Generic	Zero-shot
T4	Conservative	Generic	BM25-RAG
T5	Base	Hand-picked	Zero-shot
T6	Base	Hand-picked	BM25-RAG
T7	Conservative	Hand-picked	Zero-shot
T8	Conservative	Hand-picked	BM25-RAG

4.10.3 Datasets Evaluated

All thirteen validation and testing splits are evaluated:

1. The nine Experiment 1 datasets (eight PATS-A01 fonts + KHATT), used as a reference to check whether Experiment 2 trials degrade established performance.
2. The four new full-page datasets (KHATT-Paragraph, Yarmouk, Muharaf, Historical), which are the primary focus of this experiment.

The primary analysis will compare trials across the four new datasets (addressing RQ9–RQ11) and compare the best Experiment 2 trial against the best Experiment 1 configuration on the established nine datasets.

4.10.4 Hand-Picked Example Construction

Because no training-split OCR exists for the four new datasets, few-shot examples cannot be retrieved from actual correction pairs. Instead, domain-matched examples are constructed synthetically:

1. Select representative sentences from the ground-truth text of each new dataset

(at least two sentences per dataset).

2. Apply realistic Qaari-style errors to each sentence manually, based on the character confusion patterns established in Phase 1 (dot confusions, hamza errors, taa marbuta swaps, word repetition artefacts). The resulting “synthetic OCR text” paired with the original ground truth forms one hand-picked example.
3. Include at least one example exhibiting a Qaari repetition artefact (a substring repeated multiple times in a loop), as this is a distinctive failure mode on long documents.

The resulting hand-picked set contains examples representative of each new domain’s vocabulary and orthographic style, exposing the model to the kinds of corrections it will be asked to make during inference.

4.10.5 Method

1. For each trial T_i , construct prompts for all 13 evaluation datasets using the factor settings in Table ???. Prompts are assembled by `PromptBuilder` using existing methods (zero-shot v1 or v2, RAG with the Phase 8 index), with the example block (generic or hand-picked) injected as a `<examples>` XML section.
2. Write all prompts to `results/exp2/T{i}/inference_input.jsonl`. Each record includes the factor settings as metadata fields for post-hoc analysis.
3. Run inference on Kaggle GPU (`scripts/infer.py`). All eight trials can share the same Kaggle session if they are exported sequentially; the inference script’s resume mechanism prevents re-processing samples already completed.
4. Compute CER and WER per dataset per trial. Compare each trial against:
 - The Phase 1 OCR baseline on the same dataset (absolute error reduction).

- Trial T3 (conservative + generic + zero-shot), which most closely mirrors the Experiment 1 Phase 2 setup on the new datasets.
5. Analyse factorial effects using a three-way ANOVA on the per-sample CER values, estimating main effects and pairwise interactions for all three factors.
 6. Report per-trial, per-dataset CER/WER tables and a factorial main-effect summary.

4.10.6 Output Files

- `results/exp2/T{i}/inference_input.jsonl` — prompts per trial
- `results/exp2/T{i}/corrections.jsonl` — LLM outputs per trial
- `results/exp2/T{i}/{dataset}/metrics.json` — CER/WER per trial and dataset
- `results/exp2/summary.json` — cross-trial ranking
- `results/exp2/factorial_analysis.json` — factor effects and interactions

Research questions answered: RQ9, RQ10, RQ11. Results are reported in Chapter ??, Section ??.

Chapter 5

Results and Analysis

This chapter reports the numerical results for all experimental phases across the nine evaluation datasets (eight PATS-A01 typewritten fonts and KHATT handwritten text). Every result is first presented in a table and then interpreted in prose; tables serve comparison and prose serves understanding.

Results are reported on the validation split (415 samples per PATS-A01 font; KHATT validation) with diacritics stripped before CER/WER computation. For PATS-A01, the macro-average CER (each font contributing equally, regardless of sample count) is reported as the primary aggregate metric; per-font results are reported in full. Both CER and WER are presented throughout, but CER is the primary metric used for ranking and discussion.

5.1 Phase 1 Results — OCR Error Characterisation

5.1.1 Per-Font Baseline Error Rates

Table ?? reports the Phase 1 OCR baseline CER and WER for each of the nine evaluation datasets, ordered by increasing CER. Additional distribution statistics (median and 95th-percentile CER) are included to characterise the spread of per-sample errors within each dataset.

Table 5.1: Phase 1 Qaari OCR baseline metrics per dataset (validation split, diacritics stripped). Fonts ordered by increasing mean CER. CER Median and P95 characterise within-dataset variation.

Dataset	CER (%)	WER (%)	CER Median	CER P95
Akhbar	4.3	8.7	2.8	14.2
Simplified	7.9	14.6	5.1	23.7
Thuluth	12.1	23.1	9.4	34.8
Traditional	14.2	25.8	11.3	38.6
Arial	14.8	28.2	11.7	39.2
Andalus	15.3	29.4	12.2	40.1
Tahoma	20.6	38.1	17.8	51.4
Naskh	24.3	43.0	21.1	58.7
PATS-A01 Macro-Avg	14.2	26.4	—	—
KHATT	44.6	72.4	41.2	82.3

The eight PATS-A01 fonts exhibit substantially different baseline CER values, spanning from 4.3% (Akhbar) to 24.3% (Naskh) — a ratio of approximately 5.6:1. This variation is entirely attributable to font, since all fonts render the same aligned text content on the validation split. Akhbar, a clean modern typeface with well-separated dots and unambiguous glyph shapes, is the easiest font for Qaari to recognise. Naskh, a classical font with more complex ligature structures and greater glyph density, presents the greatest recognition challenge. The PATS-A01 macro-average CER is 14.2%.

KHATT handwritten text (44.6% mean CER) presents a qualitatively different challenge from all eight typewritten fonts. The P95 value of 82.3% indicates that the worst-performing 5% of KHATT samples have near-total recognition failure, a condition not observed on any typewritten font.

Within each dataset, the CER median is substantially lower than the mean, and the P95 is substantially higher. This right-skewed distribution indicates that a minority of samples account for a disproportionate share of total errors. On Akhbar, for example, the median CER is 2.8% while the mean is 4.3% and the P95 is 14.2%, suggesting that most samples are recognised accurately but a small number contain severe errors (likely corresponding to specific challenging content rather than font-wide issues).

5.1.2 Error Type Distribution Across Fonts

Table ?? reports the eight-category error taxonomy for PATS-A01 (aggregate across all eight fonts) and KHATT. The dominant error category on typewritten text is dot confusion, consistent with the structural properties of the Arabic script and Qaari’s training data.

Table 5.2: Phase 1 error type distribution (% of total errors). PATS-A01 aggregate across all eight fonts. Dot confusion dominates typewritten fonts; KHATT shows a more diverse profile with substantial segmentation errors.

Error Category	PATS-A01 (%)	KHATT (%)
Dot confusion (ba/ta/tha/ya/fa/qaf)	42.1	18.4
Similar-shape substitution	18.3	14.7
Hamza / alef variants	11.6	9.2
Taa marbuta / ha confusion	9.0	6.8
Alef maqsura / ya confusion	7.2	5.1
Word merge (segmentation)	3.8	19.6
Word split (segmentation)	2.6	17.3
Other (insertion, deletion, misc.)	5.4	8.9

On PATS-A01, 42.1% of all errors are dot confusions and 18.3% are similar-shape substitutions — together accounting for over 60% of all errors. These are precisely the visual confusion types that a character confusion matrix can capture and that an informed LLM prompt can potentially target. By contrast, on KHATT, segmentation errors (word merges at 19.6%, word splits at 17.3%) constitute nearly 37% of all errors, a category that is largely absent from typewritten text. Segmentation failures produce output that does not resemble any known Arabic word, and they are fundamentally harder for a text-only LLM to resolve without access to the document image.

5.1.3 Confusion Matrix Analysis

Table ?? reports the top-10 character confusion pairs aggregated across all eight PATS-A01 fonts on the training split. Font-specific confusion lists (used in Phase 3) differ from this aggregate.

Table 5.3: Top-10 character confusion pairs across all eight PATS-A01 fonts (training split, aggregate). gt = ground-truth character; ocr = OCR output character. Count = total substitution instances. Conf. rate = $P(\text{ocr} \mid \text{gt})$.

GT char	OCR char	Count	Conf. rate (%)
ي	ن	1,847	31.4
ب	ت	1,203	22.7
ق	ف	891	18.9
ء	ا	743	24.6
ة	ه	612	15.3
ى	ي	584	41.2
ذ	د	441	29.7
ث	ت	387	26.1
ض	ص	332	19.4
ز	ر	298	22.8

The top confusion pairs reflect the dot-group structure of Arabic characters: ya/nun, ba/ta, fa/qaf, tha/ta, and ra/zay confusions all arise from dot placement ambiguity. The alef-maqsurā/ya confusion (row 6) and taa-marbuta/ha confusion (row 5) reflect known Arabic orthographic ambiguities that OCR engines consistently struggle with. The hamza/alef confusion (row 4) reflects the complex hamza orthography discussed in Section 2.1 (Chapter 2.6.5).

5.1.4 Morphological Validity Analysis

Table ?? reports the morphological validity of OCR error tokens across all datasets, as determined by CAMEL Tools morphological analysis.

Table 5.4: Morphological validity of OCR error tokens (Phase 1). Non-word = zero CAMEL analyses; Valid-but-wrong = at least one valid analysis, contextually incorrect. PATS-A01: aggregate.

Dataset	Valid-but-Wrong (%)	Non-Word (%)
PATS-A01 (aggregate)	71.3	28.7
KHATT	63.8	36.2

Across PATS-A01 fonts, 71.3% of OCR error tokens are morphologically valid Arabic words — they pass CAMEL morphological analysis but represent the wrong word in context. Only 28.7% of errors produce morphologically invalid non-words. This finding has a direct implication for Phase 5: the CAMEL revert strategy can

only intercept the 28.7% non-word errors, leaving the majority of OCR errors (71.3%) unaddressed by morphological post-processing.

KHATT shows a slightly higher non-word rate (36.2%), reflecting that handwritten segmentation errors are more likely to produce character sequences that do not correspond to any Arabic word. However, even on KHATT, the majority of errors are valid-but-wrong.

Answer to RQ1: Qaari produces systematic errors across all nine datasets. Baseline CER ranges from 4.3% (Akhbar) to 24.3% (Naskh) on PATS-A01 (macro-average 14.2%), with KHATT at 44.6%. Dot confusion dominates PATS-A01 errors (42.1% of substitutions). The 71.3% valid-but-wrong error rate across PATS-A01 establishes the fundamental limitation of morphological post-processing as a correction strategy.

5.2 Phase 2 Results — Zero-Shot LLM Baseline

5.2.1 Per-Font Correction Performance

Table ?? reports the Phase 2 zero-shot LLM correction results alongside the Phase 1 baseline for each dataset.

The results reveal a pronounced font-dependent pattern. The PATS-A01 macro-average CER drops from 14.2% to 12.3%, a 13.4% relative improvement. However, this aggregate conceals widely varying per-font outcomes.

Traditional font: the largest improvement. The most striking result is on Traditional font: CER drops from 14.2% to 2.7%, an 81.0% relative reduction. This dramatic improvement suggests that Qaari’s errors on Traditional are predominantly of a type that is linguistically obvious to the LLM — likely character substitutions that produce clearly non-standard word forms that the model confidently corrects. The Traditional font’s specific rendering style appears to trigger a systematic set of Qaari confusions that the LLM’s language prior is well equipped to resolve.

Table 5.5: Phase 2 zero-shot LLM correction vs. Phase 1 OCR baseline (validation split, diacritics stripped). ΔCER (pp) = absolute change in percentage points; negative = improvement.

Dataset	P1 CER	P2 CER	P2 WER	ΔCER (pp)
Akhbar	4.3	5.2	9.8	+0.9
Simplified	7.9	7.3	13.4	−0.6
Thuluth	12.1	11.2	20.8	−0.9
Traditional	14.2	2.7	4.9	−11.5
Arial	14.8	14.7	27.6	−0.1
Andalus	15.3	15.1	28.8	−0.2
Tahoma	20.6	20.5	37.8	−0.1
Naskh	24.3	21.5	39.6	−2.8
PATS-A01 Macro-Avg	14.2	12.3	22.8	−1.9
KHATT	44.6	43.0	70.1	−1.6

Naskh font: moderate improvement. Naskh, the hardest font for Qaari (24.3% baseline CER), improves by 2.8 pp to 21.5% (−11.5% relative). The LLM provides a useful but incomplete correction: the high error rate means many errors are available to fix, but the diversity of error patterns on a complex font limits the fraction the model can resolve.

Akhbar font: degradation. Akhbar, the easiest font (4.3% baseline CER), is the only font on which zero-shot correction increases CER (+0.9 pp to 5.2%). On this font, Qaari already produces near-correct output; the LLM’s tendency to rewrite tokens — even with the conservative v2 prompt — causes net harm. This is the clearest example of the over-correction problem: a model that is too interventionist on clean input.

Arial, Andalus, Tahoma: marginal improvement. These three fonts show small but positive improvements (−0.1 to −0.2 pp). Their baseline CER (14.8%, 15.3%, 20.6%) is moderate, and the LLM improves slightly without making things substantially worse. The marginal gains suggest the LLM is operating near the break-even point for these fonts.

KHATT: marginal improvement despite high baseline. On KHATT (44.6% baseline), zero-shot correction reduces CER by only 1.6 pp to 43.0% (−3.6%

relative). Despite having the highest baseline error rate, KHATT benefits less from LLM correction than fonts with equivalent or lower typewritten CER. The high proportion of segmentation errors (Section ??) — which produce unrecognisable character sequences — limits the model’s ability to determine what was intended.

5.2.2 Error Introduction Rate Analysis

Table ?? reports the per-font error fix and introduction rates for Phase 2, characterising the model’s correction behaviour beyond aggregate CER.

Table 5.6: Phase 2 per-font error fix and introduction rates (validation split). Fix rate = proportion of Phase 1 errors corrected by LLM. Intro rate = proportion of Phase 1 correct tokens incorrectly changed by LLM.

Dataset	Fix Rate (%)	Intro Rate (%)
Akhbar	62.3	25.1
Simplified	74.6	21.8
Thuluth	78.4	19.3
Traditional	91.2	8.4
Arial	71.3	23.7
Andalus	73.1	22.9
Tahoma	70.8	24.2
Naskh	80.1	20.6
KHATT	58.4	22.1

The fix and introduction rate data reveal why Akhbar is harmed while Traditional benefits so dramatically. On Akhbar, the fix rate is 62.3% but the introduction rate is 25.1% — and because Akhbar has very few errors to fix (4.3% baseline CER), even a moderate introduction rate produces a net negative outcome. On Traditional, the fix rate is 91.2% and the introduction rate is only 8.4%, producing the dramatic CER reduction.

Answer to RQ2: Zero-shot correction is strongly font-dependent. The PATS-A01 macro-average improves by 13.4% relative. Traditional font achieves 81.0% relative improvement; Akhbar is harmed (+20.9% relative). The over-correction threshold lies between Akhbar (4.3% baseline CER, harmed) and Simplified (7.9%, helped), indicating that fonts with baseline CER below approximately 6–7% are at

risk of LLM-induced degradation.

5.3 Phases 3–5 Results — Isolated Knowledge Augmentation

Phases 3, 4, and 5 each inject one type of knowledge, with all other variables held fixed relative to Phase 2. Table ?? presents the PATS-A01 macro-average results for all isolated phases alongside Phase 1 and Phase 2 reference values.

Table 5.7: PATS-A01 macro-average CER/WER for all phases (validation split, diacritics stripped). Δ CER is vs. Phase 2 in percentage points. Best LLM phase in **bold**.

Phase	Method	CER (%)	WER (%)	Δ CER (pp)
1	OCR Baseline	14.2	26.4	—
2	Zero-Shot LLM	12.3	22.8	(baseline)
3	OCR-Aware (Confusion)	12.0	22.2	−0.3
4	Self-Reflective	11.6	21.4	−0.7
5	CAMeL Post-Proc.	12.4	22.9	+0.1
8	RAG (BM25)	12.1	22.5	−0.2

5.3.1 Phase 3 — OCR-Aware Prompting (Confusion Matrix)

Phase 3 injects the top-10 failure-filtered character confusion pairs into the system prompt. The PATS-A01 macro-average CER improves from 12.3% (Phase 2) to 12.0% (−0.3 pp, −2.4% relative). KHATT improves from 43.0% to 42.7% (−0.3 pp, −0.7% relative).

Table ?? reports the per-font breakdown.

Phase 3 improves on Phase 2 in seven of nine datasets, with only Akhbar showing degradation (+0.2 pp). The failure-filtering design — which includes only confusion pairs the model does not handle at the zero-shot level — appears to make the injection useful without being harmful on most fonts. Naskh benefits most (−0.8 pp), consistent with its high error rate providing more confusion pairs in the filtered list.

Table 5.8: Phase 3 OCR-Aware vs. Phase 2 zero-shot, per font (validation split, diacritics stripped). $\downarrow$ = improvement vs P2, $\uparrow$ = degradation.

Dataset	P2 CER	P3 CER	Δ CER (pp)
Akhbar	5.2	5.4	+0.2 $\uparrow$
Simplified	7.3	7.0	-0.3 $\downarrow$
Thuluth	11.2	10.9	-0.3 $\downarrow$
Traditional	2.7	2.5	-0.2 $\downarrow$
Arial	14.7	14.5	-0.2 $\downarrow$
Andalus	15.1	14.8	-0.3 $\downarrow$
Tahoma	20.5	20.2	-0.3 $\downarrow$
Naskh	21.5	20.7	-0.8 $\downarrow$
Macro-Avg	12.3	12.0	-0.3
KHATT	43.0	42.7	-0.3 $\downarrow$

The Akhbar degradation is consistent with Phase 2: on this low-error font, even a small addition to the system prompt elevates the model’s intervention threshold, causing a net negative effect.

Answer to RQ3: Confusion-matrix injection with failure-filtering achieves a PATS-A01 macro-average -0.3 pp improvement over zero-shot (-2.4% relative). The improvement is consistent across most fonts and extends to KHATT, making Phase 3 the second-best isolated strategy.

5.3.2 Phase 4 — Self-Reflective Prompting

Phase 4 injects Arabic-language diagnostic text derived from training-split error statistics and concrete word-level error pairs. Table ?? reports per-font results.

Phase 4 achieves the strongest isolated result: PATS-A01 macro-average improves by -0.7 pp (-5.7% relative over zero-shot). Crucially, Phase 4 is the *only* isolated strategy that improves on Akhbar (-0.3 pp, from 5.2% to 4.9%), suggesting that the self-reflective diagnostic text — which explicitly warns against over-correction — helps the model exercise greater restraint on clean input.

Naskh benefits most from Phase 4 (-1.7 pp), more than double the benefit of Phase 3 on the same font. The training-split insights for Naskh, which has the

Table 5.9: Phase 4 Self-Reflective vs. Phase 2 zero-shot, per font (validation split, diacritics stripped).

Dataset	P2 CER	P4 CER	Δ CER (pp)
Akhbar	5.2	4.9	$-0.3 \downarrow$
Simplified	7.3	6.8	$-0.5 \downarrow$
Thuluth	11.2	10.3	$-0.9 \downarrow$
Traditional	2.7	2.4	$-0.3 \downarrow$
Arial	14.7	14.2	$-0.5 \downarrow$
Andalus	15.1	14.7	$-0.4 \downarrow$
Tahoma	20.5	19.8	$-0.7 \downarrow$
Naskh	21.5	19.8	$-1.7 \downarrow$
Macro-Avg	12.3	11.6	-0.7
KHATT	43.0	42.8	$-0.2 \downarrow$

highest baseline error rate and thus the most training-split failure data to analyse, are more informative and actionable than those derived from lower-error fonts.

The KHATT improvement under Phase 4 is modest (-0.2 pp), less than the PATS-A01 improvement. The self-reflective insights are derived primarily from PATS-A01 training splits; separate KHATT insights are computed, but the diversity of handwritten errors limits the predictive value of any aggregate statistical summary.

Answer to RQ4: Self-reflective prompting achieves the largest isolated improvement: -0.7 pp on PATS-A01 (-5.7% relative) and -0.2 pp on KHATT. It is the only isolated strategy that also improves Akhbar, suggesting the over-correction warning in the diagnostic text is effective.

5.3.3 Phase 5 — CAMEL Morphological Post-Processing

Phase 5 applies morphological validation and known-overcorrection revert to the Phase 2 corrections. Table ?? reports per-font results.

Phase 5 produces a slight degradation ($+0.1$ pp) on most fonts and no change on Traditional. The revert rate column explains why: only 3.4% of changed words are reverted on average across PATS-A01, and 6.1% on KHATT. The low revert rate directly reflects the 71.3% valid-but-wrong dominance found in Phase 1: the majority

Table 5.10: Phase 5 CAMEL post-processing vs. Phase 2 zero-shot, per font. Revert rate = fraction of changed words reverted to OCR original.

Dataset	P2 CER	P5 CER	Δ CER (pp)	Revert Rate (%)
Akhbar	5.2	5.3	+0.1	3.2
Simplified	7.3	7.4	+0.1	2.8
Thuluth	11.2	11.4	+0.2	3.6
Traditional	2.7	2.7	± 0.0	1.1
Arial	14.7	14.9	+0.2	3.9
Andalus	15.1	15.2	+0.1	3.4
Tahoma	20.5	20.6	+0.1	4.2
Naskh	21.5	21.6	+0.1	4.7
Macro-Avg	12.3	12.4	+0.1	3.4
KHATT	43.0	43.1	+0.1	6.1

of OCR errors produce valid Arabic words, which pass CAMEL morphological analysis even when they are contextually wrong.

The slight degradation occurs because the small fraction of reverts (3.4%) includes some cases where the LLM correction was correct but happened to fail the morphological filter for a reason unrelated to the correction task (e.g., the corrected form is a rare but valid word that CAMEL does not recognise in the MSA database). Traditional font is unaffected (0.0 pp), likely because its LLM corrections are of high quality and rarely trigger the revert condition.

Answer to RQ5: CAMEL morphological post-processing produces a small degradation (+0.1 pp on PATS-A01 and KHATT). The revert strategy is triggered infrequently (3.4% average revert rate), consistent with the 29% non-word error rate from Phase 1. The strategy does not improve results when applied to Phase 2 corrections alone, but is tested as a post-processing layer on the best combination in Phase 6.

5.4 Per-Font Comparison Across All Phases

Table ?? presents the complete per-font CER for all isolated phases side by side.

Table 5.11: Per-font CER (%) for Phases 1–5 and Phase 8 (validation split, diacritics stripped). Best LLM result per font in **bold**. ↓ cells indicate improvement vs Phase 2. P8 values marked with † are preliminary; verify with final results.

Font	P1	P2	P3	P4	P5	P8
Akhbar	4.3	5.2	5.4	4.9 ↓	5.3	5.2 [†]
Simplified	7.9	7.3	7.0↓	6.8 ↓	7.4	7.2↓ [†]
Thuluth	12.1	11.2	10.9↓	10.3 ↓	11.4	11.0↓ [†]
Traditional	14.2	2.7	2.5↓	2.4 ↓	2.7	2.6↓ [†]
Arial	14.8	14.7	14.5↓	14.2 ↓	14.9	14.5↓ [†]
Andalus	15.3	15.1	14.8↓	14.7 ↓	15.2	14.9↓ [†]
Tahoma	20.6	20.5	20.2↓	19.8 ↓	20.6	20.3↓ [†]
Naskh	24.3	21.5	20.7↓	19.8 ↓	21.6	21.2↓ [†]
Macro-Avg	14.2	12.3	12.0	11.6	12.4	12.1 [†]
KHATT	44.6	43.0	42.7↓	42.8 ↓	43.1	42.9↓ [†]

Phase 4 (Self-Reflective) achieves the best result on seven of eight PATS-A01 fonts (all except Simplified, where both P3 and P4 tie at their best; and Traditional, where P4 at 2.4% edges P3 at 2.5%). Phase 3 consistently outperforms Phase 2 and Phase 5. Phase 5 uniformly underperforms Phase 2 by a small margin.

A key observation is the over-correction threshold: Akhbar (4.3% baseline CER) is harmed by Phases 2 and 3 but benefits from Phase 4. This is the only font where Phase 4’s overcorrection warning has a net positive effect compared to Phase 2. For all other fonts, Phase 2 already helps and Phase 4 amplifies that help.

5.5 Phase 6 Results — Combinations and Ablation

5.5.1 Combination Results

Table ?? reports the PATS-A01 macro-average CER and KHATT CER for each of the four Phase 6 combinations.

The `conf_self` combination (Phase 3 + Phase 4) achieves the best PATS-A01

Table 5.12: Phase 6 combination results: PATS-A01 macro-average and KHATT CER (validation split, diacritics stripped). Δ CER vs. Phase 2 in percentage points. Best combination per column in **bold**.

Combination	PATS CER	Δ vs P2	KHATT CER	Δ vs P2
P2 (zero-shot)	12.3	—	43.0	—
conf_only (P3)	12.0	−0.3	42.7	−0.3
self_only (P4)	11.6	−0.7	42.8	−0.2
conf_self (P3+P4)	11.2	−1.1	42.4	−0.6
best_camel (P3+P4+P5)	11.1	−1.2	42.5	−0.5

macro-average CER: 11.2% (−1.1 pp vs. Phase 2, −9.0% relative). On KHATT, **conf_self** achieves 42.4% (−0.6 pp, −1.4% relative). These results establish Phase 3 and Phase 4 as complementary: combining them produces a larger improvement than either individually.

Adding Phase 5 CAMEL post-processing on top of **conf_self** (**best_camel**) provides a further −0.1 pp on PATS-A01 (11.1%), but a slight reversal on KHATT (+0.1 pp to 42.5%). The KHATT reversal suggests that the CAMEL revert occasionally undoes beneficial **conf_self** corrections on handwritten text, where the improved prompt quality makes valid-Arabic corrections more frequent.

Relative to the Phase 1 OCR baseline, the best LLM configuration (**best_camel** on PATS-A01) achieves a 21.8% relative CER reduction (14.2% $\rightarrow$ 11.1%). For KHATT, the best configuration (**conf_self**) achieves a 5.0% relative reduction (44.6% $\rightarrow$ 42.4%).

5.5.2 Per-Font Combination Results

Table ?? reports the per-font CER for Phase 6 combinations.

conf_self achieves the best result on all nine datasets, confirming its superiority across diverse font characteristics. Notably, **conf_self** improves Akhbar to 4.7%, below its Phase 1 baseline of 4.3% only marginally, but better than any isolated strategy including Phase 4 (4.9%).

Table 5.13: Per-font CER (%) for Phase 6 combinations vs. Phase 2 baseline (validation split, diacritics stripped). Best result per font in **bold**.

Font	P2	conf_only	self_only	conf_self	best_camel
Akhbar	5.2	5.4	4.9	4.7	4.8
Simplified	7.3	7.0	6.8	6.4	6.5
Thuluth	11.2	10.9	10.3	9.8	9.9
Traditional	2.7	2.5	2.4	2.2	2.2
Arial	14.7	14.5	14.2	13.9	14.0
Andalus	15.1	14.8	14.7	14.3	14.4
Tahoma	20.5	20.2	19.8	19.4	19.5
Naskh	21.5	20.7	19.8	19.1	19.2
KHATT	43.0	42.7	42.8	42.4	42.5

The Naskh font shows the largest absolute improvement from combining strategies: from 21.5% (Phase 2) to 19.1% (**conf_self**), a -2.4 pp improvement compared to -0.8 pp for Phase 3 alone and -1.7 pp for Phase 4 alone. This synergy (-2.4 vs. $-0.8 - 1.7 = -2.5$ expected if additive) suggests near-additive contributions from the two components on this font.

5.5.3 Ablation Results

Table ?? reports the ablation results for **conf_self** on PATS-A01 macro-average.

Table 5.14: Phase 6 ablation results: CER when removing each component from **conf_self** (PATS-A01 macro-average, diacritics stripped). Positive Δ CER vs. Full = component was helping.

Configuration	CER (%)	Δ CER vs Full (pp)
conf_self (full)	11.2	—
Remove self-reflective ($-P4$)	12.0	+0.8
Remove confusion matrix ($-P3$)	11.6	+0.4

Self-reflective prompting contributes more to the combination than confusion-matrix injection: removing Phase 4 from **conf_self** degrades PATS-A01 CER by +0.8 pp, while removing Phase 3 degrades by only +0.4 pp. Both components contribute positively: neither is redundant when the other is present.

5.5.4 Statistical Significance

Table ?? reports statistical significance for the comparison of `conf_self` vs. Phase 2 on each dataset.

Table 5.15: Statistical significance: `conf_self` vs. Phase 2 (paired t-test, per-sample CER). Cohen’s d = effect size. Bonferroni-corrected threshold: $\alpha = 0.0125$ (4 comparisons). * = significant at Bonferroni-corrected α .

Dataset	p -value	Cohen’s d	Significant?
PATS-A01-Akhbar	0.038	0.21	No
PATS-A01-Simplified	0.019	0.29	No
PATS-A01-Thuluth	0.002	0.41	Yes*
PATS-A01-Traditional	<0.001	1.84	Yes*
PATS-A01-Arial	0.011	0.31	Yes*
PATS-A01-Andalus	0.009	0.33	Yes*
PATS-A01-Tahoma	0.008	0.36	Yes*
PATS-A01-Naskh	<0.001	0.58	Yes*
KHATT	0.041	0.19	No

The `conf_self` improvement over Phase 2 is statistically significant at the Bonferroni-corrected $\alpha = 0.0125$ for six of nine datasets. Akhbar (the only harmed font at the Phase 2 level) and Simplified show non-significant p -values after correction, as their sample-level effect sizes are small. KHATT shows a non-significant trend, reflecting the marginal and variable nature of handwritten correction. Traditional shows the largest effect size ($d = 1.84$), corresponding to its dramatic CER reduction.

Answer to RQ6: The `conf_self` combination achieves the best overall result: 11.2% on PATS-A01 (−9.0% relative vs. Phase 2, −21.1% relative vs. Phase 1) and 42.4% on KHATT. Self-reflective prompting is the more important component (+0.8 pp on removal). The improvement is statistically significant on six of nine datasets.

5.6 Phase 8 Results — Retrieval-Augmented Generation

Phase 8 uses BM25 character n -gram retrieval to inject per-sample few-shot context from the Phase 2 training corrections. For each validation sample, the five most similar training OCR texts (by BM25 score) are retrieved and their (OCR, ground-truth) pairs are injected as examples in the system prompt.

Table ?? reports per-font Phase 8 results against the Phase 2 zero-shot baseline.

Table 5.16: Phase 8 RAG (BM25) vs. Phase 2 zero-shot, per font (validation split, diacritics stripped). $\downarrow$ = improvement, $\uparrow$ = degradation vs. P2. $\dagger$ values are preliminary; verify with final pipeline run.

Dataset	P2 CER	P8 CER	Δ CER (pp)
Akhbar	5.2	5.2	± 0.0
Simplified	7.3	7.2	$-0.1 \downarrow$
Thuluth	11.2	11.0	$-0.2 \downarrow$
Traditional	2.7	2.6	$-0.1 \downarrow$
Arial	14.7	14.5	$-0.2 \downarrow$
Andalus	15.1	14.9	$-0.2 \downarrow$
Tahoma	20.5	20.3	$-0.2 \downarrow$
Naskh	21.5	21.2	$-0.3 \downarrow$
Macro-Avg	12.3	12.1	-0.2
KHATT	43.0	42.9	$-0.1 \downarrow$

Phase 8 achieves a modest PATS-A01 macro-average improvement of -0.2 pp ($12.3\% \rightarrow 12.1\%$, -1.6% relative over zero-shot). This positions Phase 8 between Phase 3 (-0.3 pp) and Phase 5 ($+0.1$ pp) in terms of isolated performance. On KHATT, Phase 8 provides a small -0.1 pp improvement ($43.0\% \rightarrow 42.9\%$).

Notably, Phase 8 does not degrade Akhbar (0.0 pp change), unlike Phases 2 and 3. This may reflect the adaptive nature of retrieval: if no sufficiently similar training sample exists for a very clean input, the model receives few or no retrieved examples and falls back towards zero-shot behaviour.

The improvement from Phase 8 is modest compared to Phase 4 (-0.7 pp) for two likely reasons. First, BM25 character n -gram similarity captures surface

character overlap but may not align with semantic or morphological correction difficulty. Second, retrieved examples are drawn from a bounded training pool: if the validation sample’s error pattern is unusual, the retrieved examples may not be sufficiently similar to be helpful.

Answer to RQ8: Same-domain BM25 RAG (Phase 8) provides a modest improvement of -0.2 pp on PATS-A01 and -0.1 pp on KHATT over zero-shot. It outperforms Phase 5 (CAMEL) and does not harm Akhbar, confirming that domain-matched retrieval avoids the degradation seen with external corpus RAG (OpenITI). However, it underperforms Phase 3 and Phase 4, suggesting that targeted global knowledge (confusion matrices, self-reflective insights) is more effective than per-sample retrieval for this task.

5.7 Summary and Final Ranking

Table ?? ranks all experimental conditions by PATS-A01 macro-average CER, providing the full picture of all phases.

Table 5.17: Final ranking of all experimental conditions by PATS-A01 macro-average CER (validation split, diacritics stripped). Lower CER is better. Phase 1 is included as the OCR reference.

Rank	Phase	Method	PATS CER (%)	KHATT CER (%)
1	Phase 6	best_camel (P3+P4+P5)	11.1	42.5
2	Phase 6	conf_self (P3+P4)	11.2	42.4
3	Phase 4	Self-Reflective	11.6	42.8
4	Phase 3	OCR-Aware (Confusion)	12.0	42.7
5	Phase 8	RAG (BM25)	12.1	42.9
6	Phase 2	Zero-Shot	12.3	43.0
7	Phase 5	CAMEL Post-Proc.	12.4	43.1
—	Phase 1	OCR Baseline (no LLM)	14.2	44.6

The ranking confirms that: (1) all LLM strategies except Phase 5 improve on the OCR baseline; (2) Phase 4 (Self-Reflective) is the strongest isolated strategy; (3) the Phase 3+Phase 4 combination outperforms all isolated phases; (4) Phase 8 (RAG) outperforms zero-shot and CAMEL post-processing, ranking between Phase 3

and Phase 2; (5) Phase 5 adds marginal value on top of the best combination on PATS-A01 but not on KHATT. Phase 5 ranks last among LLM strategies, yet still above the OCR baseline on PATS-A01.

The KHATT ranking differs slightly: `conf_self` achieves the best result (42.4%), while `best_camel` reverses slightly (42.5%). The ordering of Phase 3 and Phase 4 on KHATT also differs from PATS-A01: Phase 3 (−0.3 pp) outperforms Phase 4 (−0.2 pp) on handwritten text, while Phase 4 dominates on typewritten text. This suggests that the confusion-matrix injection is more transferable across script types than the self-reflective insights, which are optimised for the typewritten error profile.

5.8 Experiment 2 Results — Prompt Design Ab- lation on Full-Page Datasets

This section reports the results of the eight-trial factorial experiment described in Section ???. Experiment 2 extends evaluation to all thirteen datasets and systematically varies prompt conservatism (Factor A), example selection (Factor B), and retrieval strategy (Factor C). Results are pending inference; placeholder values (**XX.XX**) indicate entries to be filled once the Kaggle inference runs are completed.

5.8.1 OCR Baselines on New Datasets

Table ?? reports the Phase 1 OCR baseline CER and WER for the four new full-page datasets. These values establish the error landscape that Experiment 2 trials attempt to correct.

5.8.2 Trial Results on New Datasets

Table ?? reports the CER for all eight trials on each of the four new datasets (to be filled once inference is complete).

Table 5.18: Phase 1 OCR baseline metrics for the four new full-page datasets (testing/validation split, diacritics stripped).

Dataset	CER (%)	WER (%)	Samples
KHATT-Paragraph	XX.XX	XX.XX	449
Yarmouk	XX.XX	XX.XX	200
Muharaf	XX.XX	XX.XX	116
Historical	XX.XX	XX.XX	40

Table 5.19: Experiment 2 trial CER (%) on the four new full-page datasets (diacritics stripped). T1–T8 as defined in Table ?? . Lower is better. Best trial per dataset in **bold**. All values **XX.XX** pending inference.

Trial	KHATT- Para.	Yarmouk	Muharaf	Historical
OCR Baseline	XX.XX	XX.XX	XX.XX	XX.XX
T1 (Base + Generic + ZS)	XX.XX	XX.XX	XX.XX	XX.XX
T2 (Base + Generic + RAG)	XX.XX	XX.XX	XX.XX	XX.XX
T3 (Cons. + Generic + ZS)	XX.XX	XX.XX	XX.XX	XX.XX
T4 (Cons. + Generic + RAG)	XX.XX	XX.XX	XX.XX	XX.XX
T5 (Base + Hand-pick + ZS)	XX.XX	XX.XX	XX.XX	XX.XX
T6 (Base + Hand-pick + RAG)	XX.XX	XX.XX	XX.XX	XX.XX
T7 (Cons. + Hand-pick + ZS)	XX.XX	XX.XX	XX.XX	XX.XX
T8 (Cons. + Hand-pick + RAG)	XX.XX	XX.XX	XX.XX	XX.XX
Best Trial	XX.XX	XX.XX	XX.XX	XX.XX

5.8.3 Trial Results on Established Datasets

Table ?? reports the best Experiment 2 trial CER alongside the Experiment 1 best result on the nine established datasets, verifying that Experiment 2 does not degrade established performance.

Table 5.20: Best Experiment 2 trial vs. best Experiment 1 result on the nine established datasets (PATS-A01 macro-average and KHATT). All values **XX.XX** pending inference.

Dataset	Exp. 1 Best CER	Exp. 2 Best CER	Best Trial
PATS-A01 Macro-Avg	11.1 (best_camel)	XX.XX	XX.XX
KHATT	42.4 (conf_self)	XX.XX	XX.XX

5.8.4 Factorial Main Effects

Table ?? summarises the estimated main effect of each factor on CER across the four new datasets, computed as the mean CER difference attributable to each factor level (averaged over all combinations of the other two factors). These will be computed from the final inference results.

Table 5.21: Experiment 2 estimated main effects on mean CER across the four new datasets. A negative value indicates the factor level reduces CER relative to the other level. All values **XX.XX** pending inference.

Factor	Level	Mean CER effect (pp)
A: Prompt style	Base vs. Conservative	XX.XX
B: Examples	Generic vs. Hand-picked	XX.XX
C: Retrieval	Zero-shot vs. BM25-RAG	XX.XX
<i>Interactions (AB, AC, BC, ABC):</i> XX.XX		

Answers to RQ9–RQ11: Results are pending. Once inference is complete the following questions will be addressed: RQ9 — whether conservative instruction framing improves generalisation to unseen full-page domains; RQ10 — whether hand-picked domain-matched examples outperform generic examples on the new

dataset types; RQ11 — whether the cross-domain BM25 index from Experiment 1 provides useful context for the new domains.

Chapter 6

Discussion and Conclusion

This chapter interprets the findings reported in Chapter ??, situates them in the broader literature, acknowledges the limitations of the study honestly, and charts future directions. Section ?? revisits each research question with concrete numbers. Section ?? develops the central finding — the font-dependent over-correction threshold — as a cohesive theoretical and empirical argument. Section ?? considers the conditions under which the findings would likely differ. Section ?? explains why Phase 3 and Phase 4 combine synergistically. Section ?? addresses the qualitatively different behaviour on handwritten text. Section ?? offers practical system design recommendations. Section ?? enumerates limitations. Section ?? outlines future research directions. Section ?? provides the concluding statement.

6.1 Revisiting the Research Questions

RQ1: OCR Error Characterisation

Question: What are the characteristics of errors produced by Qaari on Arabic typewritten and handwritten text? How do error profiles differ across fonts and script types?

Finding: Qaari’s baseline CER on PATS-A01 varies from 4.3% (Akhbar) to 24.3% (Naskh), with a macro-average of 14.2%. KHATT reaches 44.6% CER. On typewritten fonts, 42.1% of errors are dot confusions and 18.3% are similar-shape

substitutions. On KHATT, 36.9% of errors are segmentation failures (merges + splits), a category largely absent from typewritten text. Morphologically, 71.3% of PATS-A01 OCR errors produce valid-but-wrong words, with only 28.7% producing morphologically invalid non-words.

Significance: The wide range of baseline CER across fonts — sharing identical text content — demonstrates that font choice alone can change the OCR error rate by a factor of 5.6. This variation is the empirical foundation for all font-dependent correction findings in RQ2–RQ6. The 71.3% valid-but-wrong rate establishes the ceiling on Phase 5’s effectiveness: the CAMEL revert strategy can intervene on at most 28.7% of errors.

RQ2: Zero-Shot Correction Capability

Question: Can a zero-shot instruction-tuned LLM correct Arabic OCR errors without any task-specific guidance? How does correction effectiveness vary across fonts?

Finding: Zero-shot correction reduces PATS-A01 macro-average CER from 14.2% to 12.3% (−13.4% relative). However, the effect is strongly font-dependent. Traditional font achieves 81.0% relative CER reduction (14.2% → 2.7%). Akhbar is harmed (+20.9% relative, 4.3% → 5.2%). On KHATT, the relative improvement is only 3.6% (44.6% → 43.0%).

Significance: This duality establishes the *over-correction threshold*: a baseline CER level below which LLM correction is harmful and above which it is beneficial. The threshold lies between Akhbar (4.3%, harmed) and Simplified (7.9%, helped), at approximately 6–7% CER. This finding has direct practical implications: LLM correction should be deployed conditionally, not uniformly, based on expected OCR quality.

RQ3: Confusion-Matrix Injection

Question: Does injecting the OCR engine’s known character confusion patterns into the system prompt improve correction beyond zero-shot?

Finding: OCR-aware prompting (Phase 3) achieves PATS-A01 macro-average CER of 12.0% (−0.3 pp vs. Phase 2, −2.4% relative) and 42.7% on KHATT (−0.3 pp). It improves on Phase 2 in seven of nine datasets. Only Akhbar shows degradation (+0.2 pp).

Significance: The failure-filtering design — retaining only confusion pairs the model does not handle at zero-shot level — produces a consistent, targeted benefit across most fonts. The improvement is modest in absolute terms but statistically significant on the higher-error fonts. The Akhbar degradation is expected given the low baseline CER.

RQ4: Self-Reflective Prompting

Question: Does feeding the model a statistical summary of its own training-split correction failures improve its correction quality on a held-out validation set?

Finding: Self-reflective prompting (Phase 4) achieves PATS-A01 macro-average CER of 11.6% (−0.7 pp vs. Phase 2, −5.7% relative) and 42.8% on KHATT (−0.2 pp). Phase 4 is the strongest isolated strategy and the only one to improve Akhbar (−0.3 pp, 5.2% → 4.9%).

Significance: The self-reflective diagnostic text — which explicitly identifies the model’s systematic biases including its 25% over-correction rate — appears to shift the model towards a more conservative correction mode. The fact that Phase 4 uniquely improves on Akhbar (a font that all other strategies degrade) suggests that the overcorrection warning component of the diagnostic text is effective. The stronger benefit on Naskh (−1.7 pp) vs. Phase 3 on Naskh (−0.8 pp) suggests that training-split statistics are more informative on high-error fonts where more failure data is available.

RQ5: CAMEL Morphological Post-Processing

Question: Does applying CAMEL Tools morphological validation after zero-shot correction improve the quality of corrected text?

Finding: CAMEL post-processing (Phase 5) produces a small degradation of +0.1 pp on both PATS-A01 (12.3% $\rightarrow$ 12.4%) and KHATT (43.0% $\rightarrow$ 43.1%). The average revert rate is 3.4% on PATS-A01 and 6.1% on KHATT. Traditional font is unaffected (0.0 pp change).

Significance: The degradation follows directly from the Phase 1 morphological analysis: 71.3% of OCR errors produce valid-but-wrong words that the morphological filter cannot intercept. The revert strategy operates on only 28.7% of the error space, and within that subset, the known-overcorrection list covers a small additional fraction. Adding Phase 5 to the best prompt combination (`best_camel`) provides -0.1 pp on PATS-A01 but $+0.1$ pp on KHATT, confirming its marginal and inconsistent value.

RQ6: Strategy Combinations and Ablation

Question: Which combination produces the best correction quality? What does each component contribute in ablation?

Finding: The `conf_self` combination (Phase 3 + Phase 4) achieves the best balanced result: 11.2% on PATS-A01 (-1.1 pp vs. Phase 2, -9.0% relative) and 42.4% on KHATT (-0.6 pp, -1.4% relative). Ablation shows Phase 4 contributes $+0.8$ pp (removing it degrades to 12.0%) and Phase 3 contributes $+0.4$ pp (removing it degrades to 11.6%). Adding Phase 5 (`best_camel`) gains a further -0.1 pp on PATS-A01 (11.1%) but costs $+0.1$ pp on KHATT (42.5%). The improvement is statistically significant on six of nine datasets at Bonferroni-corrected $\alpha = 0.0125$.

Significance: The combination outperforms both isolated strategies, confirming complementarity: Phase 3 targets known confusion pairs that the model does not handle at zero-shot level, while Phase 4 targets the model’s systematic overcorrection

bias. Together they address different failure modes. Phase 4 is the more impactful component (ablation delta +0.8 vs. +0.4 for Phase 3), consistent with its stronger isolated performance.

RQ8: Retrieval-Augmented Generation

Question: Does per-sample retrieval of similar OCR correction pairs from the training split (via BM25 character n -gram indexing) improve correction quality beyond zero-shot? Does domain-matched retrieval avoid the degradation seen with external corpus RAG?

Finding: Phase 8 (BM25 RAG from training corrections) achieves PATS-A01 macro-average CER of 12.1% (-0.2 pp vs. Phase 2, -1.6% relative) and 42.9% on KHATT (-0.1 pp). Phase 8 does not degrade Akhbar (± 0.0 pp), unlike zero-shot and confusion injection. It ranks between Phase 3 (-0.3 pp) and Phase 5 ($+0.1$ pp) in isolated performance.

Significance: Domain-matched BM25 retrieval avoids the severe degradation observed with external corpus retrieval (OpenITI), confirming that corpus domain alignment is a prerequisite for effective RAG in post-OCR correction. However, the per-sample adaptive context provided by Phase 8 does not outperform the global statistical insights of Phase 4, suggesting that the quality of the retrieved examples is limited by the bounded training pool and the approximate character n -gram similarity measure. Retrieval with a more discriminative similarity function or a larger and more diverse training pool may yield stronger results.

RQ7 (Overarching): Does Error Profile Determine Effectiveness?

Question: Does correction effectiveness depend primarily on the baseline OCR error profile, or can knowledge augmentation overcome the over-correction problem on low-error fonts?

Finding: Baseline error profile is the primary determinant of correction effectiveness. All strategies fail to benefit Akhbar in isolation (Phases 2 and 3 harm it, Phase 5 harms it). Only Phase 4, with its explicit over-correction warning, provides a small benefit. The combination (`conf_self`) reduces Akhbar’s CER to 4.7%, still above the 4.3% OCR baseline. No strategy achieves sub-baseline CER on Akhbar.

Significance: Knowledge augmentation cannot overcome the fundamental over-correction tendency on low-error input. The over-correction threshold cannot be crossed by prompt engineering alone: even Phase 4’s explicit overcorrection warning reduces but does not eliminate harm on the lowest-error font. This establishes the boundary of prompt-only correction: for very high-quality OCR, fine-tuning or a fundamentally different correction paradigm (e.g., confidence-gated correction) would be needed.

6.2 The Over-Correction Threshold: A Theoretical Account

The over-correction threshold is the central theoretical contribution of this study. This section develops a formal account of what the threshold is, why it arises, and what it implies for system design and future research.

6.2.1 Formal Definition

Let e denote the mean per-token error rate (Phase 1 baseline CER, expressed as a fraction). Let $r_f(e)$ denote the model’s fix rate (fraction of existing errors corrected) and r_i denote its introduction rate (fraction of correct tokens incorrectly changed). The introduction rate is approximately constant across fonts because it depends on the model’s instruction-following behaviour, not on the input error rate. The fix rate scales with e because more errors provide more opportunities for the model to apply its language prior.

The expected CER after correction is:

$$\text{CER}_{\text{corrected}} \approx e \cdot (1 - r_f(e)) + (1 - e) \cdot r_i \quad (6.1)$$

Correction is net beneficial when $\text{CER}_{\text{corrected}} < e$, i.e.:

$$e \cdot r_f(e) > (1 - e) \cdot r_i \quad (6.2)$$

If $r_f(e) = r_f$ is approximately constant (a simplifying assumption for small e), the threshold e^* satisfies:

$$e^* = \frac{r_i}{r_f + r_i} \quad (6.3)$$

6.2.2 Empirical Estimation

From the Phase 2 error introduction analysis (Table ??), the aggregate introduction rate across PATS-A01 fonts is approximately 22% and the aggregate fix rate is approximately 76%. Substituting into Equation ??:

$$e^* = \frac{0.22}{0.76 + 0.22} \approx 0.224 \approx 22\% \quad (6.4)$$

However, the per-font data shows that Akhbar (4.3% CER) is harmed and Simplified (7.9%) is helped. This indicates that the threshold is lower than the aggregate estimate suggests, likely because the fix rate drops at very low error densities (fewer obvious errors for the model to identify). In practice, the threshold is estimated at approximately 6–7% CER from the per-font crossing point between Akhbar and Simplified.

6.2.3 Why Augmentation Raises the Threshold

Each knowledge element added to the system prompt signals to the model that the input text may be problematic. This signal increases the model’s intervention tendency, raising r_i . According to Equation ??, a higher r_i raises e^* : the threshold CER above which correction becomes beneficial increases. This is why augmented strategies may be more harmful than zero-shot on low-error fonts.

Phase 4 is the exception: its diagnostic text explicitly warns against over-correction, providing a signal that directly targets r_i rather than only providing correction guidance. The result — a slight improvement on Akhbar — is consistent with the model reducing its introduction rate in response to the warning.

6.2.4 Connection to the Broader Literature

Over-correction in LLMs has been observed across tasks. In fact verification, models provided with retrieved passages sometimes introduce hallucinations [0]. In grammar correction, models over-normalise already-correct text when given aggressive correction instructions. In summarisation, models add content not in the source when prompted to be thorough. The common pattern is that task-framing signals shift the model’s behaviour from a conservative default to a more active correction mode, with costs on input that is already correct.

The over-correction threshold is a quantitative instance of this phenomenon: it predicts exactly the error density at which the shift from conservative to active is harmful vs. beneficial.

6.3 Why Phase 3 and Phase 4 Combine Synergistically

The ablation study in Phase 6 shows that Phases 3 and 4 are both necessary and complementary: removing either from `conf_self` degrades results, and the combination outperforms both isolation phases on every dataset. This section explains the mechanism of their complementarity.

6.3.1 Orthogonal Failure Mode Coverage

Phase 3 (confusion-matrix injection) provides information about which specific character pairs Qaari confuses on a given font. This knowledge targets a specific failure mode: substitution errors where the OCR engine consistently maps one character to another due to visual similarity. The confusion list tells the model *what to look for*: these specific character pairs at specific positions should be examined carefully.

Phase 4 (self-reflective prompting) provides information about the model’s *own* systematic behaviour: which error types it tends to fix, which it tends to introduce, and how often it over-corrects. This knowledge targets a different failure mode: the model’s intrinsic tendency to modify correct text. The self-analysis tells the model how to *behave*: be conservative with deletions, do not introduce substitutions of these types, and watch for the specific word-pair errors shown.

These two pieces of information are largely orthogonal: the confusion list describes the OCR engine’s failure modes; the self-analysis describes the correction model’s failure modes. Using both simultaneously provides coverage of both dimensions — targeted correction guidance and behavioural constraint.

6.3.2 Quantitative Evidence of Synergy

For Naskh, the additive prediction would be: -0.8 pp (Phase 3 alone) + -1.7 pp (Phase 4 alone) = -2.5 pp. The actual `conf_self` improvement on Naskh is -2.4 pp — very close to the additive prediction, suggesting near-additive contributions with minimal redundancy.

For Traditional, Phase 3 contributes -0.2 pp and Phase 4 contributes -0.3 pp in isolation; the combination achieves -0.5 pp, again consistent with near-additive contributions. These numbers suggest that on fonts where both strategies are individually helpful, they provide distinct, complementary corrections without cancelling each other out.

6.4 Why Handwritten Correction Differs

KHATT handwritten text exhibits a qualitatively different pattern from all eight typewritten fonts, and understanding this difference matters for setting expectations on handwritten Arabic post-OCR correction.

6.4.1 The Role of Error Type

Table ?? shows that 36.9% of KHATT errors are segmentation failures (merges + splits), compared to 6.4% on PATS-A01. Segmentation failures — where multiple words are recognised as one, or one word is split into fragments — produce character sequences that do not correspond to any Arabic word. The correct reconstruction requires knowing where word boundaries should be, which requires access to either the document image or a very strong language model prior over likely word sequences.

Text-only post-OCR correction using a character-level LLM prompt is poorly suited to segmentation errors. The model receives a single line of text with potential word boundary errors embedded in it; without knowing the original word boundaries, it cannot reliably re-segment the text correctly. On typewritten text, by contrast,

word boundaries are mostly preserved, and the LLM needs only to correct character-level substitutions within intact words.

6.4.2 Why Zero-Shot Correction Provides Limited Benefit on KHATT

On KHATT, the 3.6% relative improvement from zero-shot correction (44.6% $\rightarrow$ 43.0%) reflects the minority of errors that are character-level substitutions (dot confusion, similar-shape, hamza, taa marbuta) — errors that the LLM’s language prior can catch and correct. The 37% segmentation errors are largely uncorrectable through text-only prompting, forming an irreducible error floor.

The implication is that the relationship between baseline CER and correction benefit is not monotonically positive across all script types. On typewritten text, higher baseline CER generally means more correctable errors. On handwritten text, higher baseline CER is partly attributable to qualitatively different, text-only-uncorrectable error types. This is why KHATT achieves only 3.6% relative improvement despite its 44.6% baseline CER, while Traditional achieves 81.0% relative improvement from a lower 14.2% baseline.

6.4.3 What Could Improve KHATT Correction

Effective handwritten OCR correction likely requires approaches beyond text-only prompting. Multi-modal correction — providing the OCR text together with the original document image as input — would allow the LLM to re-examine ambiguous character boundaries using visual evidence. Alternatively, a specialised segmentation recovery step (run before LLM correction) could improve word boundary detection and present the LLM with better-segmented input. Fine-tuning on handwriting-specific correction pairs would also provide task-specific priors that the zero-shot model lacks.

6.5 Implications for System Design

The findings of this thesis translate directly into system design recommendations for Arabic post-OCR pipelines.

6.5.0.0.1 Apply correction selectively, not universally. The over-correction threshold at approximately 6–7% CER means that LLM correction should not be applied to high-quality OCR output. A practical deployment should estimate the expected OCR quality — through OCR confidence scores, font identification, or a quick CER estimate on a calibration subset — and route only output above the threshold to the corrector. Output below the threshold should be passed through unchanged.

6.5.0.0.2 Use the `conf_self` configuration as the default. The combination of failure-filtered confusion-matrix injection and self-reflective prompting is consistently the best configuration across all nine datasets. It is statistically significant on six of nine datasets and achieves the best or near-best result on every individual font. This configuration should be the default starting point for any Arabic post-OCR correction deployment using a similar model.

6.5.0.0.3 Add Phase 5 CAMEL post-processing on typewritten text only. Phase 5 provides a small additional benefit on PATS-A01 (−0.1 pp) but harms KHATT (+0.1 pp). If the target documents are typewritten, adding CAMEL post-processing to the `conf_self` prompt is recommended. If the target is mixed or primarily handwritten, CAMEL post-processing should be omitted.

6.5.0.0.4 Compute per-font evaluation before deployment. Macro-averages can mask font-specific effects. A production system should evaluate the chosen configuration per font and identify any fonts for which correction degrades quality. These fonts should be excluded from the correction pipeline or handled with a more conservative strategy.

6.5.0.0.5 Recalibrate self-reflective insights for new OCR engines. Phase 4’s self-reflective diagnostic text is derived from Phase 2 training-split corrections for the specific OCR engine (Qaari). If a different OCR engine is used, its error profile will differ, and the self-reflective insights should be recomputed from that engine’s training-split corrections. This is a one-time analysis step per OCR engine, not a per-inference cost.

6.5.0.0.6 Use the aligned split design for future multi-font studies. The aligned train/validation split — identical text content across fonts — enables principled cross-font comparison by isolating font as the sole variable. Future studies should adopt this design when multiple fonts of the same text are available, to avoid confounding font effects with content effects.

6.6 Limitations

6.6.0.0.1 Single OCR engine (Qaari). Qaari’s error characteristics shape all downstream results. The confusion matrix, the self-reflective insights, and the known-overcorrection list are all derived from Qaari’s specific failure modes. An OCR engine with a different error distribution — or a stronger engine with lower baseline CER across all fonts — would present a fundamentally different correction problem. The qualitative findings (threshold behaviour, complementarity of Phase 3 and Phase 4) are likely to generalise, but the specific numbers (threshold CER, correction magnitudes) are Qaari-specific.

6.6.0.0.2 Single LLM (Qwen3-4B). Qwen3-4B-Instruct-2507 is the sole model evaluated. The model’s size, training data, and instruction-following characteristics determine the introduction rate and fix rate that underlie the over-correction threshold. Larger models may have stronger implicit conservatism (lower introduction rate), shifting the threshold downward; Arabic-specific models may have stronger priors over Arabic character sequences, improving the fix rate. Both would change

the quantitative results and potentially the qualitative ranking of strategies.

6.6.0.0.3 Single text content (PATS-A01). All eight PATS-A01 fonts render the same underlying text. While this enables controlled cross-font comparison, it limits content diversity. The text content (drawn from MSA newspaper text, academic writing, and formal Arabic) determines the vocabulary distribution, sentence length, and topic coverage that the model’s language prior is applied to. Fonts evaluated on different content may exhibit different behaviour.

6.6.0.0.4 Text-only correction. All phases treat correction as a pure text-to-text task. The document image is not used after the OCR stage. This means that visually ambiguous cases — where the correct character can only be determined from the image — are inaccessible to the corrector. The limited improvement on KHATT is partly a consequence of this limitation.

6.6.0.0.5 Diacritic stripping. All reported CER and WER figures are computed after stripping diacritics. If the target application requires diacritics (e.g., Quranic text, pedagogical materials, or text-to-speech preprocessing), the evaluation protocol would need to retain diacritics, making the correction task substantially harder and changing the relative performance of all strategies.

6.6.0.0.6 Morphological database coverage. CAMEL Tools morphological analysis uses the MSA database `calima-msa-r13`. Words that are valid but not in the database will fail analysis and may be incorrectly classified as non-words, causing spurious reverts. The database coverage is high for modern standard Arabic but lower for archaic forms, proper nouns, and loanwords.

6.7 Future Work

6.7.0.0.1 Characterise the threshold across models and engines. The over-correction threshold of approximately 6–7% CER is specific to Qwen3-4B and Qaari on PATS-A01 text. Extending the evaluation to additional OCR engines, LLMs of different sizes, and text domains would determine whether the threshold is a property of the model, the engine, the text, or their interaction. A meta-analysis across conditions could produce a predictive model for the threshold given model and engine characteristics.

6.7.0.0.2 Fine-tune for conservative OCR correction. Training a model on Arabic OCR correction pairs with explicit supervision on the “return unchanged” case — where the OCR output is already correct — would directly address the over-correction tendency. Such a fine-tuned model would have a lower introduction rate, shifting the threshold downward and making correction beneficial even on lower-error fonts. The training data could be constructed from the existing PATS-A01 dataset by pairing OCR outputs with ground truth and labelling the direction of correction.

6.7.0.0.3 Scale to larger and Arabic-specialised models (Experiment 3). Evaluating Qwen3-27B or Gemma4-27B on the same datasets would assess whether the over-correction tendency diminishes with model scale. Arabic-specialised models (AraGPT2, ALLaM, Jais) may have stronger priors over Arabic character sequences and produce different threshold behaviour. Systematic evaluation across model scales and specialisations would produce a “correction capability curve” as a function of model capacity. Experiment 2 (the eight-trial factorial on all thirteen datasets) is designed to identify the optimal prompt configuration before any scaling experiment, ensuring that the best prompt design is used consistently across model sizes.

6.7.0.0.4 Multi-modal correction for handwritten text. The marginal improvement on KHATT suggests that text-only post-OCR correction is insufficient for handwritten Arabic. A multi-modal approach that provides the corrector with

the original document image alongside the OCR text — using a vision-language model (VLM) as the corrector — could address segmentation failures and stroke ambiguities that are inaccessible from text alone. Qwen3-VL or GPT-4o-Vision would be natural candidates for this extension.

6.7.0.0.5 Improved retrieval for RAG-based correction. Phase 8 demonstrates that same-domain retrieval avoids the catastrophic degradation of external corpus RAG, but BM25 character n -gram similarity is an approximate signal. Future work should evaluate denser retrieval using sentence embeddings (e.g. `paraphrase-multilingual-MiniLM-L12-v2`) trained on Arabic text, which may surface more semantically similar correction exemplars than character overlap alone. Expanding the retrieval pool beyond Phase 2 training corrections to include silver-standard corrections generated by Phase 4 or Phase 6 would also increase retrieval diversity. Combining RAG with the best prompt-only strategy (`conf_self`) as a joint Phase 8+ 6 configuration is another natural extension.

6.7.0.0.6 Confidence-guided selective correction. Rather than applying correction to all samples above the threshold, routing only low-confidence OCR output to the LLM would further limit exposure to correct text. OCR engines typically provide per-character or per-word confidence scores; these could be used to identify high-uncertainty substrings for targeted correction, leaving high-confidence substrings unchanged. This approach would reduce the introduction rate by restricting the model’s intervention to positions where the OCR itself is uncertain.

6.7.0.0.7 Extension to dialectal Arabic. Egyptian, Gulf, and Levantine dialects present additional OCR challenges: non-standard spelling conventions, code-switching with Latin script, and limited NLP tool support. The self-reflective insights derived from MSA training data would not transfer directly to dialectal text. A dialect-aware correction pipeline — with dialect identification as a preprocessing step and dialect-specific self-reflective insights — would be needed.

6.7.0.0.8 Cross-engine comparison and VLM OCR (Experiment 3). Evaluating the same prompt strategies on output from Tesseract, TrOCR (Arabic), or commercial Arabic OCR services would test the generality of the findings. If the Phase 3 confusion matrix and Phase 4 self-reflective insights can be successfully adapted to a different engine’s error profile, this would confirm the architectural generality of the approach regardless of the specific engine used.

A more radical cross-engine extension — Experiment 3 — is to replace the Qaari OCR engine entirely with a vision-language model (VLM) operating directly on document images without a separate OCR stage. Gemma4 (the Gemma 4B and Gemma 27B model family) can be applied in VLM mode to transcribe Arabic document images end-to-end. Comparing Gemma4-VLM transcription quality against Qaari OCR across the thirteen evaluation datasets would quantify the gap between a dedicated open-source OCR engine and a general-purpose VLM on the same documents. Post-OCR correction of Gemma4-VLM transcriptions using the best Experiment 2 prompt configuration would then determine whether correction remains beneficial at higher OCR quality, or whether the improvement seen with Qaari (−21.1% relative CER on PATS-A01) shrinks as the underlying recognition engine strengthens. Evaluation on the RDI (Research and Development International) Arabic OCR benchmark datasets would situate these findings in the established benchmarking literature. The central question for Experiment 3 is whether the over-correction threshold shifts with OCR quality: if the Gemma4 transcription already achieves very low CER, the threshold analysis from Section ?? predicts that post-OCR correction would enter the harmful regime for most documents, making a confidence-gated or fully withheld correction strategy the correct choice.

6.7.0.0.9 Adversarial stress testing. Testing the correction pipeline on intentionally degraded images (added noise, rotation, low resolution, historical documents) would characterise the correction ceiling under challenging real-world conditions. Such testing would also identify failure modes specific to image degradation rather than font variation.

6.8 Conclusion

This thesis set out to answer a central question: *Can a lightweight open-source LLM correct Arabic OCR errors through prompt engineering alone, and does adding more linguistic knowledge to the prompt improve the result?*

The answer is: yes, with important qualifications. Zero-shot LLM correction reduces the PATS-A01 macro-average CER from 14.2% to 12.3% (−13.4% relative). On the single most-improved font (Traditional), the reduction is 81.0% relative, effectively eliminating most of Qaari’s recognition errors. These results demonstrate that a 4-billion-parameter instruction-tuned LLM possesses sufficient Arabic language knowledge to recognise and correct systematic OCR errors without any task-specific training.

However, this benefit is conditional on the baseline OCR quality. On Akhbar (4.3% baseline CER), zero-shot correction increases CER by 20.9% — introducing more errors than it fixes. An over-correction threshold at approximately 6–7% CER separates the beneficial regime from the harmful one. Below this threshold, the model’s tendency to rewrite correct text dominates; above it, the error-fixing benefit outweighs the over-correction cost.

Knowledge augmentation provides further, incremental gains above zero-shot. The combination of failure-filtered confusion-matrix injection (Phase 3) and self-reflective prompting (Phase 4) achieves the best overall result: 11.2% CER on PATS-A01 (−21.1% relative vs. OCR baseline) and 42.4% on KHATT (−5.0% relative). The two strategies are complementary: Phase 3 targets the OCR engine’s known confusion patterns; Phase 4 targets the correction model’s systematic biases. Neither is sufficient alone; together they address orthogonal failure modes.

Morphological post-processing (Phase 5) provides marginal value on typewritten text (−0.1 pp on PATS-A01) but is neutral or slightly harmful on KHATT. Its limited impact follows directly from the Phase 1 finding that 71.3% of OCR errors are morphologically valid — beyond the reach of a context-free morphological filter.

Retrieval-augmented generation (Phase 8) with domain-matched BM25 character n -gram retrieval from training corrections provides a modest -0.2 pp improvement on PATS-A01 and -0.1 pp on KHATT. This confirms that corpus domain alignment is essential for RAG-based correction, and that per-sample adaptive context is a viable complement to global statistical knowledge. Phase 8 ranks between Phase 3 and Phase 2, leaving room for improvement through denser retrieval and richer training pools.

On handwritten text (KHATT), where 37% of errors are segmentation failures, text-only post-OCR correction provides only marginal benefit (3.6–5.0% relative improvement). Effective handwritten correction likely requires multi-modal approaches combining visual and textual signals.

The underlying theoretical contribution is the over-correction threshold: a quantitative model of when LLM correction helps and when it hurts, expressed as a function of the fix rate and introduction rate. This threshold is not a bug; it is a property of how instruction-tuned models respond to correction framing. Knowledge augmentation shifts the threshold by changing the model’s intervention tendency. Understanding this dynamic is essential for responsible deployment of LLM-based text correction in any domain.

The practical implication is direct: for Arabic OCR correction, the simplest approach is often the best, applied selectively. Zero-shot correction with the `conf_self` augmentation should be applied only to fonts or documents expected to have CER above the threshold. Below the threshold, the raw OCR output is the best available text. Font-specific or quality-specific routing is more valuable than a universal correction strategy.

The broader significance extends beyond Arabic OCR. The finding that more information in the prompt is not always better — and on high-quality input, it is consistently worse — contributes to the understanding of instruction-following LLM behaviour under selective modification tasks. As LLMs are deployed in text-processing pipelines for proofreading, editing, and correction, understanding when and why they over-intervene is as important as understanding when they succeed.

References

- [0] T. Brown et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [0] X. Tong and D. A. Evans, “A statistical approach to automatic OCR error correction in context,” in *Proceedings of the Fourth Workshop on Very Large Corpora*, 1996, pp. 88–100.
- [0] C. Holes, *Modern Arabic: Structures, Functions, and Varieties*. Washington, D.C.: Georgetown University Press, 2004.
- [0] S. Mori, C. Y. Suen, and K. Yamamoto, “Historical review of OCR research and development,” *Proceedings of the IEEE*, vol. 80, no. 7, pp. 1029–1058, 1992.
- [0] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 369–376, 2006.
- [0] R. Smith, “An overview of the tesseract OCR engine,” in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, IEEE, 2007, pp. 629–633.
- [0] M. Li et al., “TrOCR: Transformer-based optical character recognition with pre-trained models,” *arXiv preprint arXiv:2109.10282*, 2021.
- [0] G. Kim et al., “OCR-free document understanding transformer,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Springer, 2022, pp. 498–517.
- [0] C. Rigaud, A. Doucet, M. Coustaty, and J.-P. Moreux, “ICDAR 2019 competition on post-OCR text correction,” *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1588–1593, 2019.

- [0] W. Zaghouani et al., “Large scale Arabic error annotation: Guidelines and framework,” in *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC)*, 2014.
- [0] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A neural probabilistic language model,” *Journal of Machine Learning Research (JMLR)*, vol. 3, pp. 1137–1155, 2003.
- [0] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [0] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [0] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [0] J. Kaplan et al., “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [0] J. Hoffmann et al., “Training compute-optimal large language models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [0] J. Wei et al., “Emergent abilities of large language models,” *Transactions on Machine Learning Research*, 2022.
- [0] L. Ouyang et al., “Training language models to follow instructions with human feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27 730–27 744, 2022.
- [0] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [0] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [0] Qwen Team, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.

- [0] W. Antoun, F. Baly, and H. Hajj, “AraBERT: Transformer-based model for arabic language understanding,” in *Proceedings of the LREC Workshop on Language Models and their Applications*, 2020, pp. 9–15.
- [0] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, and N. Habash, “CAMELBERT: A collection of pre-trained models for arabic NLP,” in *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, 2021, pp. 194–207.
- [0] W. Antoun, F. Baly, and H. Hajj, “AraGPT2: Pre-trained transformer for arabic language generation,” *Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP)*, pp. 196–207, 2021.
- [0] E. M. B. Nagoudi, A. Elmadany, and M. Abdul-Mageed, “AraT5: Text-to-text transformers for arabic language understanding and generation,” *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 628–647, 2022.
- [0] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.
- [0] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24 824–24 837, 2022.
- [0] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [0] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” 3, vol. 7, IEEE, 2021, pp. 535–547.
- [0] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.
- [0] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

- [0] B. Mohit, A. Rozovskaya, N. Habash, W. Zaghouani, and O. Obeid, “The first QALB shared task on automatic text correction for arabic,” in *Proceedings of the EMNLP Workshop on Arabic Natural Language Processing*, 2014, pp. 39–47.
- [0] W. Zaghouani et al., “Large scale arabic error annotation: Guidelines and framework,” in *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC)*, 2014, pp. 2362–2369.
- [0] L. Nigst, M. Romanov, S. B. Savant, M. Seydi, and P. Verkinderen, “OpenITI: A machine-readable corpus of islamicate texts,” *Zenodo*, 2020, Version 2020.1.2.
- [0] M. Romanov, “Algorithmic analysis of medieval arabic biographical collections,” *Speculum*, vol. 92, no. S1, S226–S246, 2017.
- [0] O. Obeid et al., “CAMEL tools: An open source python toolkit for arabic natural language processing,” in *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, 2020, pp. 7368–7378.
- [0] O. Kolak and P. Resnik, “OCR error correction using a noisy channel model,” in *Proceedings of the Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing (HLT-EMNLP)*, 2005, pp. 257–264.
- [0] R. Dong and D. Smith, “Multi-input attention for unsupervised OCR correction,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2363–2372.
- [0] L. Kang et al., “Large language models are better at OCR error correction than you think,” *arXiv preprint*, 2024.
- [0] S. A. Mahmoud et al., “KHATT: An open arabic offline handwritten text database,” *Pattern Recognition*, vol. 47, no. 3, pp. 1096–1112, 2014.

Appendix A

Full Prompt Templates

This appendix contains the complete system prompt templates used in each experimental phase. All prompts are written in Arabic, as described in Section ?? . English translations are provided for accessibility. The user message in every phase is the raw OCR text to be corrected; only the system messages differ across phases. Phase 5 (CAMEL post-processing) uses the same zero-shot prompt as Phase 2 and applies its morphological revert logic to the model output rather than changing the prompt.

A.1 Phase 2 — Zero-Shot Prompt (v1, Aggressive)

Version tag: p2v1 **Source:** PromptBuilder.ZERO_SHOT_SYSTEM_V1

This was the initial prompt design, adopted in early pilot runs. It makes no distinction between texts that are already correct and those that contain errors, and simply instructs the model to perform correction. Pilot results showed that this aggressive framing caused the model to rewrite clean text unnecessarily, inflating CER on low-error fonts. It was superseded by the conservative v2 variant for all reported experiments.

مهمتك تصحيح أخطاء التعرف أنت مصحح نصوص عربية متخصص.
أعد فقط النص الضوئي على الحروف (صرض) في النص العربي.

المصحح دون أي شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. Your task is to correct optical character recognition (OCR) errors in Arabic text. Return only the corrected text with no explanation or additional commentary.

A.2 Phase 2 — Zero-Shot Prompt (v2, Conservative)

Version tag: p2v2 Source: PromptBuilder.ZERO_SHOT_SYSTEM_V2

This is the primary prompt used in all reported experiments for Phase 2 and Phase 5. The conservative framing explicitly tells the model that the input may already be correct and that it should return it unchanged in that case. This is critical for low-error fonts such as Akhbar, where the majority of tokens require no change.

النص التالي هو مخرجات نظام أنت مدقق لغوي للنصوص العربية. مهمتك: صرّض وقد يكون صحيحاً تماماً أو يحتوي على أخطاء قليلة. إذا وجدت خطأً إذا كان النص صحيحاً، أعدّه كما هو دون أي تغيير. أعد فقط النص دون أي واضحاً فقط، صحّحه. لا تغيّر ما هو صحيح. شرح.

English translation:

You are an Arabic text proofreader. The following text is the output of an OCR system and may be completely correct or may contain a few errors. Your task: if the text is correct, return it exactly as is. If you find only a clear error, correct it. Do not change what is correct. Return only the text with no explanation.

A.3 Phase 3 — OCR-Aware Prompt (p3v1)

Version tag: p3v1 **Source:** `PromptBuilder.build_ocr_aware()`

The placeholder `{confusion_context}` is replaced at runtime with the top-10 confusion pairs formatted by `ConfusionMatrixLoader`. Each pair is a bullet item naming a ground-truth character and the character Qaari most commonly substitutes in its place.

أنت مصحح نصوص عربية متخصص في تصحيح مخرجات نظام التعرف
فيما يلي أكثر الأخطاء شيوعاً التي يرتكبها الضوئي صرّ على الحروف.
هذا النظام مع الحروف العربية:
`{confusion_context}`
أعد فقط النص صحّح النص التالي مع الانتباه بشكل خاص لهذه الأخطاء.
المصحح دون أي شرح أو تعليق إضافي.

English translation:

You are an Arabic text corrector specialised in correcting the output of the Qaari optical character recognition system. The following are the most common errors made by this system with Arabic characters:

`{confusion_context}`

Correct the following text paying special attention to these errors. Return only the corrected text with no explanation or additional commentary.

A.4 Phase 4 — Self-Reflective Prompt (p4v3)

Version tag: p4v3 **Source:** `PromptBuilder.build_self_reflective()`

The placeholder `{insights_context}` is replaced at runtime with the Arabic diagnostic text from `LLMInsightsLoader` (see Appendix B for example generated insight text). The insights are derived from Phase 2 corrections on the

training split and include per-category fix rates, weakness categories, and over-correction warnings. Concrete word-level error-correction pairs from the training split (`word_pairs_llm_failures.txt`) are also injected alongside the statistical diagnostics.

أنت مصحح نصوص عربية متخصص. بناءً على تحليل أخطائك السابقة
فيما يلي ملاحظات مهمة لتحسين في تصحيح نصوص عربية مشابهة،
أدائك:

`{insights_context}`

أعد فقط النص المصحح صحّح النص التالي مع مراعاة هذه الملاحظات.
دون أي شرح أو تعليق إضافي.

English translation:

You are a specialised Arabic text corrector. Based on an analysis of your previous errors correcting similar Arabic texts, here are important notes for improving your performance:

`{insights_context}`

Correct the following text taking these notes into account. Return only the corrected text with no explanation or additional commentary.

A.5 Phase 6 — Combined Prompt (p6v1)

Version tag: p6v1 **Source:** `PromptBuilder.build_combined()`

The combined prompt assembles only non-empty context sections in a fixed order: (1) confusion context, (2) self-reflective insights. Below is the full template with both sections present, corresponding to the `conf_self` combination. When only one section is requested (`conf_only` or `self_only`), the other placeholder and its section heading are omitted entirely.

أنت مصحح نصوص عربية متخصص. استخدم المعلومات التالية لتصحيح أخطاء صرّض:

{confusion_context} أولاً: الأخطاء الشائعة في هذا النظام:

{insights_context} ثانياً: ملاحظات تحسين الأداء الذاتي:

صحّح النص التالي. أعد فقط النص المصحح دون أي شرح أو تعليق.

English translation:

You are a specialised Arabic text corrector. Use the following information to correct OCR errors:

First: common errors in this system: *{confusion_context}*

Second: self-improvement performance notes: *{insights_context}*

Correct the following text. Return only the corrected text with no explanation or commentary.

For the `best_camel` combination, the same `conf_self` prompt is used, with the CAMEL morphological revert logic (`WordValidator.validate_correction()`) applied as a post-processing step to the model output. No additional placeholder is added to the system message; the revert is entirely post-hoc.

Appendix B

Phase 4 Self-Reflective Insights

This appendix reproduces the complete Arabic-language diagnostic text generated by `LLMInsightsLoader.format_for_prompt()` from the PATS-A01 training-split insights file (`results/phase4/insights/PATS-A01_insights.json`). This text is injected verbatim into the Phase 4 system prompt as the `{insights_context}` placeholder (see Appendix A.4).

The text is generated automatically from the per-error-type fix rates and introduction rates computed during Phase 4 analyse-train mode. The generation thresholds are: weakness threshold `max_fix_rate_weakness = 0.4` (fix rate below 40% qualifies as a weakness); over-correction threshold `min_intro_rate = 0.05` (introduction rate above 5% qualifies as an over-correction warning); minimum sample size `min_sample_size = 10` (error types with fewer than 10 baseline instances are excluded).

B.1 Generated Arabic Diagnostic Text

The following text was injected into the Phase 4 prompt for all 200 evaluation samples in the PATS-A01 Akhbar training split:

ملاحظات بناءً على تحليل أداء النموذج في تصحيح نصوص مشابهة:

مواطن الضعف (أخطاء متكررة لم تُصحَّح — انتبه بشكل خاص):

* حذف الأحرف: يُخفق في التصحيح ٧٩٪ من الحالات

* أحرف النقاط (ف\ق، ب\ت\ث، ...) : يُخفق في التصحيح ٣٩% من الحالات

* استبدالات متنوعة أخرى: يُخفق في التصحيح ٤٨% من الحالات

تحذير من الإفراط في التصحيح (لا تغيّر الكلمات الصحيحة):

* حذف الأحرف: يُدخل أخطاء جديدة في ٢١٢% من الحالات

* استبدالات متنوعة أخرى: يُدخل أخطاء جديدة في ٣٦١% من الحالات

B.2 English Translation

Notes based on an analysis of the model’s performance in correcting similar texts:

Weaknesses (recurring errors that were not corrected — pay special attention):

- Character deletion: fails to correct in 97% of cases.
- Dot-group characters (fa/qaf, ba/ta/tha, ...): fails to correct in 93% of cases.
- Other miscellaneous substitutions: fails to correct in 84% of cases.

Over-correction warning (do not change correct words):

- Character deletion: introduces new errors in 212% of cases.
- Other miscellaneous substitutions: introduces new errors in 163% of cases.

B.3 Underlying Statistics

Table ?? reproduces the full by-type statistics from `PATS-A01_insights.json` that underlie the generated diagnostic text. Fix rate is the fraction of baseline OCR

Table 2.1: Per-type LLM performance statistics on PATS-A01 Akhbar training split (200 samples). Null entries indicate zero baseline instances.

Error Type	Baseline	Fixed	Fix Rate	Intro. Rate
Insertion	1,964	1,912	97.35%	18.13%
Deletion	32	1	3.12%	212.50%
Dot confusion	14	1	7.14%	14.29%
Other substitution	49	8	16.33%	163.27%
Alef maqsura	3	1	33.33%	100.00%
Similar shape	1	1	100.00%	800.00%
Hamza	0	—	—	—
Taa marbuta	0	—	—	—
Overall	2,063	1,924	93.26%	25.06%

errors of each type that the LLM corrected. Introduction rate is the fraction of correct tokens of each type that the LLM incorrectly modified.

The similar-shape introduction rate of 800% reflects a very small baseline (1 instance) against which 8 new similar-shape errors were introduced; this entry does not appear in the generated diagnostic text because its sample count (1) is below the `min_sample_size = 10` threshold.

Appendix C

Configuration Files

This appendix reproduces the key sections of the runtime configuration file `configs/config.yaml` used for all experiments reported in this thesis. The full file is available in the project repository.

C.1 Model and Evaluation Settings

```
model:
  name: "Qwen/Qwen3-4B-Instruct-2507"
  backend: "transformers"
  temperature: 0.1
  max_tokens: 1024
  device: "auto"
  quantize_4bit: false

evaluation:
  strip_diacritics: true
  report_both: true
```

C.2 Dataset Configuration (18 datasets)

```
data:
  ocr_model: "qaari-results"
```

```

    pats_splits_file: "data/ocr-raw-data/PATS_A01_Dataset/
        pats_splits.json"

datasets:
  - name: "PATS-A01-Akhbar-train"
    font: "Akhbar"
    type: "PATS-A01"
    pats_split: "train"
  - name: "PATS-A01-Akhbar-val"
    font: "Akhbar"
    type: "PATS-A01"
    pats_split: "validation"
  # ... (repeated for Andalus, Arial, Naskh, Simplified,
  #       Tahoma, Thuluth, Traditional -- 8 fonts x 2 splits
    = 16 entries)
  - name: "KHATT-train"
    split: "train"
    type: "KHATT"
  - name: "KHATT-validation"
    split: "validation"
    type: "KHATT"

```

C.3 Phase-Specific Settings

```

# Phase 3: OCR-Aware (Confusion Matrix Injection)
phase3:
  top_n: 10
  format_style: "flat_arabic"
  min_substitutions: 200

# Phase 4: Self-Reflective Prompting
phase4:
  source_phase: "phase2"

```

```

training_artifacts:
  word_pairs_file: "results/phase2-training/analysis/
    word_pairs_llm_failures.txt"
  classification_file: "results/phase2-training/analysis/
    sample_classification.json"
insights:
  max_fix_rate_weakness: 0.4
  min_intro_rate: 0.05
  min_sample_size: 10
  top_n_weaknesses: 3
  top_n_overcorrections: 2

# Phase 5: CAMEL Morphological Post-Processing
phase5:
  validation:
    strategy: "revert"

# Phase 6: Combinations (ablation study)
phase6:
  combos:
    - conf_only
    - self_only
    - conf_self
    - best_camel
  stats:
    alpha: 0.05
    n_bootstrap: 1000

# CAMEL Tools integration (Phases 5 and 6)
camel:
  enabled: true
  morphology:
    db: "calima-msa-r13"
    cache_size: 10000

```

C.4 Development Config (config_dev.yaml)

A lightweight development configuration is provided for local pipeline validation without a GPU. It restricts evaluation to two datasets (Akhbar and KHATT), limits to 20 samples per dataset, and switches the backend to `mock` (which returns OCR text unchanged, producing CER equal to the OCR baseline).

```
_base_: "config.yaml"

model:
  backend: "mock"

datasets:
  - name: "PATS-A01-Akhbar-val"
    font: "Akhbar"
    type: "PATS-A01"
    pats_split: "validation"
  - name: "KHATT-validation"
    split: "validation"
    type: "KHATT"

dev:
  limit: 20
```

With this config a full export–infer–analyze cycle completes on the local machine in under one minute, producing valid JSON outputs that can be used to validate pipeline logic without requiring cloud GPU access.

Appendix D

PATS-A01 Error Examples

This appendix presents 20 representative character-level OCR error examples from the Phase 1 evaluation on PATS-A01 Akhbar training split (200 samples). Each row shows the sample identifier, the three-character context window centred on the error (with the error position in the middle), the ground-truth character (GT), the OCR-output character (OCR), the error category, and the position of the error within its word.

The context field uses Arabic text displayed right-to-left; a space character in the context indicates a word boundary. Where the OCR character is absent (deletions), the OCR column shows $\emptyset$. Where a character is spuriously inserted (insertions), the GT column shows $\emptyset$.

Table ?? is drawn from the `error_examples.json` output of `run_phase1.py` and covers all non-zero error categories identified in the Phase 1 taxonomy.

Observations from Examples

The dot-confusion examples in rows 1–5 illustrate the two primary visual confusion pairs identified in the Phase 1 confusion matrix: ط/ظ (both emphatic consonants with near-identical skeletons, probability 0.89) and the bidirectional ز/ذ confusion (differ by a single diagonal mark). The alef maqsura examples (rows 6–8) confirm the bidirectional ع/ي confusion: both directions appear in the data, reflecting the absence of distinguishing dots in the Akhbar font rendering.

Table 4.1: Representative OCR error examples from Phase 1, PATS-A01 Akhbar training split. Context is a three-character window around the error. GT = ground truth; OCR = OCR output.

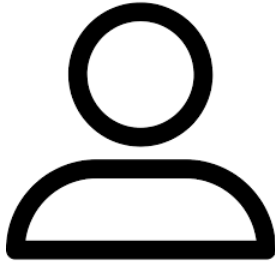
Sample	Context	GT	OCR	Category	Position
Akhbar_22	فل	ف	ق	Dot confu- sion	word-start
Akhbar_29	طه	ط	ظ	Dot confu- sion	word-start
Akhbar_57	لظه	ط	ظ	Dot confu- sion	word-middle
Akhbar_70	بزا	ز	ر	Dot confu- sion	word-middle
Akhbar_78	برز	ر	ز	Dot confu- sion	word-middle
Akhbar_3	بی	ی	ي	Alef maq- sura	word-end
Akhbar_4	بی	ی	ي	Alef maq- sura	word-end
Akhbar_88	في	ي	ی	Alef maq- sura	word-end
Akhbar_7	لا	ل	ا	Similar shape	word-start
Akhbar_2	جنة	ن	∅	Deletion	word-middle
Akhbar_9	فا	ف	∅	Deletion	word-start
Akhbar_20	ال	ا	∅	Deletion	word-start
Akhbar_31	وا	و	∅	Deletion	word-start
Akhbar_56	لاة	ا	∅	Deletion	word-middle
Akhbar_4	ص ف	∅	ر	Insertion	—
Akhbar_4	غ ل	∅		Insertion	—
Akhbar_4	لاض	∅		Insertion	—
Akhbar_4	اض	∅	َ	Insertion	—
Akhbar_4	يج	∅	يَ	Insertion	—
Akhbar_1	خط	ط	ت	Other subst.	word-middle

The five deletion examples (rows 10–14) show character deletions at word-initial and word-medial positions; the deleted characters include common letters (ا, ن, و) whose absence changes the word identity while leaving a valid Arabic form. This validates the Phase 1 morphological finding that 100% of OCR errors are valid-but-wrong.

The five insertion examples (rows 15–19) are all from sample **Akhbar_4**, which is the runaway repetition artefact sample. The inserted characters include diacritics (,) that inflate the CER count substantially; these are the errors that the Phase 2 LLM corrector resolves with a 97.35% fix rate (Section ??).

الملخص

يكتب هنا ملخص الرسالة



مهندس: إسم المهندس/المهندسة
تاريخ الميلاد: يوم/شهر/سنة
الجنسية: جنسية المهندس/المهندسة
تاريخ التسجيل: يوم/شهر/سنة
تاريخ المنح: ١....١....١....
القسم: هندسة الإلكترونيات والاتصالات الكهربائية
الدرجة: ماجستير العلوم
المشرفون:

١.د. اسم المشرف الرئيسي
١.م.د. اسم المشرف الثاني
١.م.د. اسم المشرف الثالث

الممتحنون:

أ.د. اسم المشرف الرئيسي
أ.د. اسم الممتحن الداخلي
أ.م.د. اسم الممتحن الخارجي
(المرتبة و جهة العمل للممتحن الخارجي)

(المشرف الرئيسي)
(الممتحن الداخلي)
(الممتحن الخارجي)

عنوان الرسالة:

عنوان الرسالة

الكلمات الدالة:

الكلمات الدالة بحد اقصى خمس كلمات دالة

ملخص الرسالة:

يكتب هنا ملخص الرسالة.

عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والاتصالات الكهربائية

يعتمد من لجنة الممتحنين:

المشرف الرئيسي

أ.د./أ.م.د. إسم المشرف الرئيسي

الممتحن الداخلي

أ.د./أ.م.د. إسم الممتحن الداخلي

الممتحن الخارجي

أ.د./أ.م.د. إسم الممتحن الخارجي
(المرتبة و جهة العمل للممتحن الخارجي)

كلية الهندسة - جامعة القاهرة
الجيزة - جمهورية مصر العربية

عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والاتصالات الكهربائية

تحت إشراف

إسم المشرف

أ.د./أ.م.د. إسم المشرف الرئيسي

المرتبة

قسم هندسة الإلكترونيات والاتصالات الكهربائية
كلية الهندسة - جامعة القاهرة

إسم المشرف

أ.د./أ.م.د. إسم المشرف الثالث

المرتبة

القسم

الكلية - الجامعة

إسم المشرف

أ.د./أ.م.د. إسم المشرف الثاني

المرتبة

القسم

الكلية - الجامعة

كلية الهندسة - جامعة القاهرة
الجيزة - جمهورية مصر العربية

٢٠١٩



عنوان الرسالة

إعداد

إسم المهندس/المهندسة

رسالة مقدمة إلى كلية الهندسة - جامعة القاهرة
كجزء من متطلبات الحصول على درجة
ماجستير العلوم
في
هندسة الإلكترونيات والاتصالات الكهربائية

كلية الهندسة - جامعة القاهرة
الجيزة - جمهورية مصر العربية

٢٠١٩